RAJIV GANDHI COLLEGE OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Computer Networks Laboratory [CS P52]

B.Tech V-Semester [2022-2023 ODD Semester]

Syllabus

1. Implementation of a socket program for Echo/Ping/Talk commands.
2. Creation of a socket between two computers and enable file transfer between them. Using (a.) TCP (b.) UDP
3. Implementation of a program for Remote Command Execution (Two M/Cs may be used).
4. Implementation of a program for CRC and Hamming code for error handling.
5. Writing a code for simulating Sliding Window Protocols.
6. Create a socket for HTTP for web page upload & Download.
7. Write a program for TCP module Implementation. (TCP services).
8. Write a program to implement RCP (Remote Capture Screen).
9. Implementation (using NS2/Glomosim) and Performance evaluation of the following routing protocols:
    a. Shortest path routing
    b. Flooding
    c. Link State
    d. Hierarchical
    e. Broadcast /Multicast routing.
10. Implementation of ARP.
11. Throughput comparison between 802.3 and 802.11.
12. Study of Key distribution and Certification schemes.
13. Design of an E-Mail system
14. Implementation of Security Compromise on a Node using NS2 / Glomosim
15. Implementation of Various Traffic Sources using NS2 / Glomosim

Table of Contents

Ex No:1 **Study of Socket Programming in Java**

Aim: To learn the basics of Socket Programming in Java

Socket Programming in Java

Java Socket programming is used for communication between the applications running on different JRE.

The java.net package of the J2SE APIs contains a collection of classes and interfaces that provide the low-level communication details, allowing you to write programs that focus on solving the problem at hand.

The java.net package provides support for the two common network protocols −

- **TCP** − TCP stands for Transmission Control Protocol, which allows for reliable communication between two applications. TCP is typically used over the Internet Protocol, which is referred to as TCP/IP.
- **UDP** − UDP stands for User Datagram Protocol, a connection-less protocol that allows for packets of data to be transmitted between applications.

Socket and ServerSocket classes are used for connection-oriented socket programming and DatagramSocket and DatagramPacket classes are used for connection-less socket programming.

The client in socket programming must know two information:

1. IP Address of Server, and
2. Port number.

TCP Socket Programming

- Sockets provide the communication mechanism between two computers using TCP. A client program creates a socket on its end of the communication and attempts to connect that socket to a server. When the connection is made, the server creates a socket object on its end of the communication. The client and the server can now communicate by writing to and reading from the socket.

- The java.net.Socket class represents a socket, and the java.net.ServerSocket class provides a mechanism for the server program to listen for clients and establish connections with them.

The following steps occur when establishing a TCP connection between two computers using sockets –

- The server instantiates a ServerSocket object, denoting which port number communication is to occur on.

- The server invokes the accept() method of the ServerSocket class. This method waits until a client connects to the server on the given port.
- After the server is waiting, a client instantiates a Socket object, specifying the server name and the port number to connect to.
- The constructor of the Socket class attempts to connect the client to the specified server and the port number. If communication is established, the client now has a Socket object capable of communicating with the server.
- On the server side, the accept() method returns a reference to a new socket on the server that is connected to the client's socket.

After the connections are established, communication can occur using I/O streams. Each socket has both an OutputStream and an InputStream. The client's OutputStream is connected to the server's InputStream, and the client's InputStream is connected to the server's OutputStream.

TCP is a two-way communication protocol, hence data can be sent across both streams at the same time. Following are the useful classes providing complete set of methods to implement sockets.

ServerSocket Class Methods

The **java.net.ServerSocket** class is used by server applications to obtain a port and listen for client requests.The ServerSocket class has four constructors –

Sr.No.	Method & Description
1	**public ServerSocket(int port) throws IOException** Attempts to create a server socket bound to the specified port. An exception occurs if the port is already bound by another application.
2	**public ServerSocket(int port, int backlog) throws IOException** Similar to the previous constructor, the backlog parameter specifies how many incoming clients to store in a wait queue.
3	**public ServerSocket(int port, int backlog, InetAddress address) throws IOException** Similar to the previous constructor, the InetAddress parameter specifies the local IP address to bind to. The InetAddress is used for servers that may have multiple IP addresses, allowing the server to specify which of its IP addresses to accept client requests on.
4	**public ServerSocket() throws IOException** Creates an unbound server socket. When using this constructor, use the bind() method when you are ready to bind the server socket.

If the ServerSocket constructor does not throw an exception, it means that your application has successfully bound to the specified port and is ready for client requests.

Following are some of the common methods of the ServerSocket class –

Sr.No.	Method & Description
1	**public int getLocalPort()** Returns the port that the server socket is listening on. This method is useful if you passed in 0 as the port number in a constructor and let the server find a port for you.
2	**public Socket accept() throws IOException** Waits for an incoming client. This method blocks until either a client connects to the server on the specified port or the socket times out, assuming that the time-out value has been set using the setSoTimeout() method. Otherwise, this method blocks indefinitely.
3	**public void setSoTimeout(int timeout)** Sets the time-out value for how long the server socket waits for a client during the accept().
4	**public void bind(SocketAddress host, int backlog)** Binds the socket to the specified server and port in the SocketAddress object. Use this method if you have instantiated the ServerSocket using the no-argument constructor.

When the ServerSocket invokes accept(), the method does not return until a client connects. After a client does connect, the ServerSocket creates a new Socket on an unspecified port and returns a reference to this new Socket. A TCP connection now exists between the client and the server, and communication can begin.

Socket Class Methods

The **java.net.Socket** class represents the socket that both the client and the server use to communicate with each other. The client obtains a Socket object by instantiating one, whereas the server obtains a Socket object from the return value of the accept() method.

The Socket class has five constructors that a client uses to connect to a server –

Sr.No.	Method & Description
1	**public Socket(String host, int port) throws UnknownHostException, IOException.** This method attempts to connect to the specified server at the specified port. If this constructor does not throw an exception, the connection is successful and the client is connected to the server.
2	**public Socket(InetAddress host, int port) throws IOException** This method is identical to the previous constructor, except that the host is denoted by an InetAddress object.
3	**public Socket(String host, int port, InetAddress localAddress, int localPort) throws IOException.** Connects to the specified host and port, creating a socket on the local host at the specified address and port.
4	**public Socket(InetAddress host, int port, InetAddress localAddress, int localPort) throws IOException.**

	This method is identical to the previous constructor, except that the host is denoted by an InetAddress object instead of a String.
5	**public Socket()** Creates an unconnected socket. Use the connect() method to connect this socket to a server.

When the Socket constructor returns, it does not simply instantiate a Socket object but it actually attempts to connect to the specified server and port.Some methods of interest in the Socket class are listed here. Notice that both the client and the server have a Socket object, so these methods can be invoked by both the client and the server.

Sr.No.	Method & Description
1	**public void connect(SocketAddress host, int timeout) throws IOException** This method connects the socket to the specified host. This method is needed only when you instantiate the Socket using the no-argument constructor.
2	**public InetAddress getInetAddress()** This method returns the address of the other computer that this socket is connected to.
3	**public int getPort()** Returns the port the socket is bound to on the remote machine.
4	**public int getLocalPort()** Returns the port the socket is bound to on the local machine.
5	**public SocketAddress getRemoteSocketAddress()** Returns the address of the remote socket.
6	**public InputStream getInputStream() throws IOException** Returns the input stream of the socket. The input stream is connected to the output stream of the remote socket.
7	**public OutputStream getOutputStream() throws IOException** Returns the output stream of the socket. The output stream is connected to the input stream of the remote socket.
8	**public void close() throws IOException** Closes the socket, which makes this Socket object no longer capable of connecting again to any server.

InetAddress Class Methods

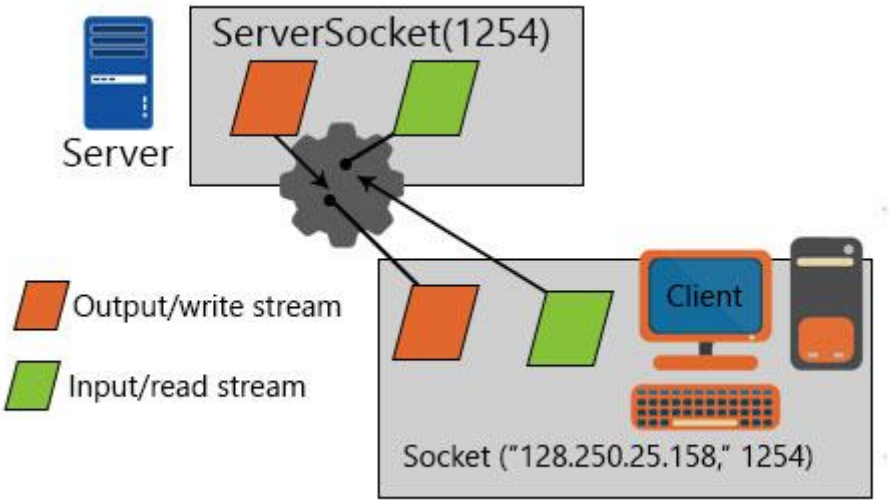
This class represents an Internet Protocol (IP) address. Here are following usefull methods which you would need while doing socket programming –

Sr.No.	Method & Description
1	**static InetAddress getByAddress(byte[] addr)** Returns an InetAddress object given the raw IP address.
2	**static InetAddress getByAddress(String host, byte[] addr)** Creates an InetAddress based on the provided host name and IP address.

3	**static InetAddress getByName(String host)** Determines the IP address of a host, given the host's name.
4	**String getHostAddress()** Returns the IP address string in textual presentation.
5	**String getHostName()** Gets the host name for this IP address.
6	**static InetAddress InetAddress getLocalHost()** Returns the local host.
7	**String toString()** Converts this IP address to a String.

Example

A server program communicates through input and output streams. If there is a request for connection, then there is a new socket that comes into play and here is a connection established with it.



Creating a Server Socket Program

The simple steps of creating a server socket program are as follows:

Step 1 – The Socket Server is Opened
Server Socket General= new Server Socket(PO)
Here PO is the port number.
Here the Port number is assigned to the server network through which it will communicate using Input/ Output streams.

Step 2 – There is a Client Request for which we have to Patiently Wait
Socket General= server. accept()
Here the Server. accept() waits for the client and the name of the socket is Client here.

Step 3 – I/O Streams are Created so that a Connection is Established
Data Input Stream is = new Data Input Stream(client. Get Input Stream());
Data Output Stream os = new Data Output Stream(client. get Output Stream());
The input stream is and the output stream os are assigned their Get Input Stream() and they are called respectively.

Step 4 – Contact with the Client is Created
Receive from client: String hello = br. Read Line();
Send it to the client: br. Write Bytes("How are you\n");
The following code communicates with the client receiving and sending to a client the requests.
Step 5 – Finally, the Socket is made to Exit
Finally, the close socket function is used to close and end the socket programming. A simple example of a server socket is shown below:

```
// A simple program for connecting the server.
import java.net.*;
import java.io.*;
public class SimpleMachine {
public static void main(String args[]) throws IOException {
// On port 1362 server port is registered
ServerSocket soc = new ServerSocket(1362);
Socket soc1=soc.accept(); // Link is accepted after waiting
// Linked with the socket there should be a connection
OutputStream s1out = soc1.getOutputStream();
DataOutputStream dosH = new DataOutputStream (s1out);
// A string command is sent
dosH.writeUTF("Hello how are you");
// The connection can be closed but the server socket cannot.
dosH.close();
s1out.close();
soc1.close();        }
}
```

Creating a Simple Client Program

The steps for creating a simple client program in Java is shown below:

Step 1 – Socket Object is Made
Socket client= new Socket(server, port_id)

The server and the Port ID are connected, that is, the server is connected to the Port ID.

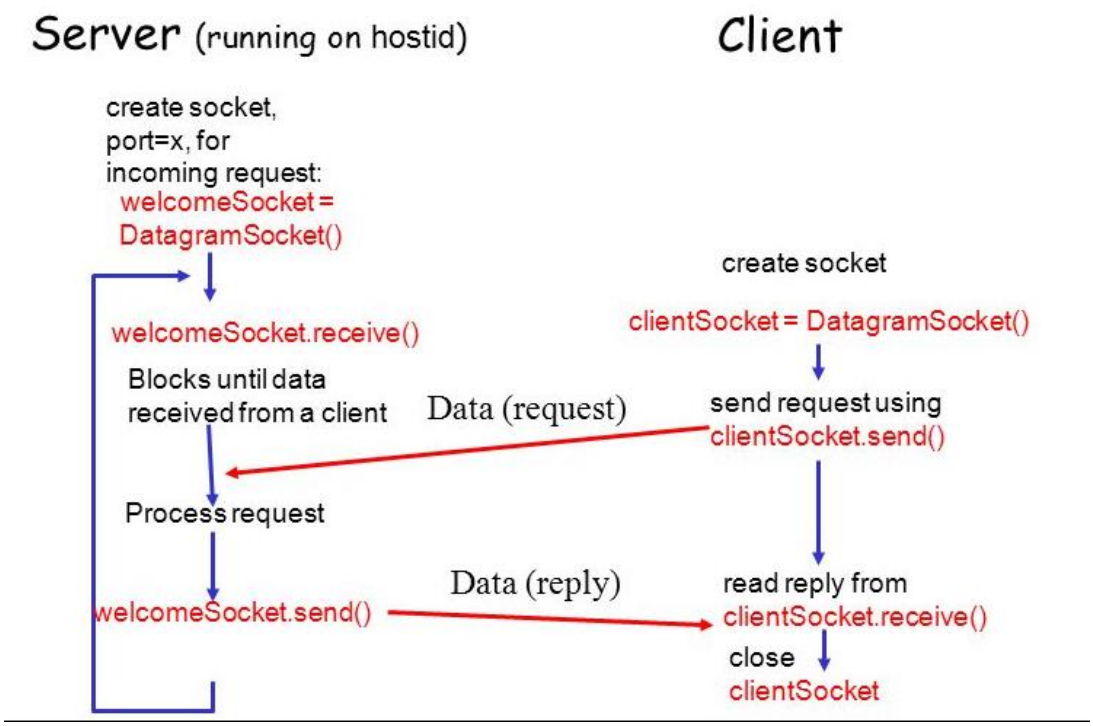
Step 2 – Input/Output Streams are Created
is = new Data Input Stream(client.getInputStream());
os = new Data Output Stream(client.getOutputStream());
The Input Stream is and Output Stream os is used for communicating with the client.

Step 3 – Input/Output streams are made for talking to the Client
Data is read from the server: String hello = br. readLine();
Send data to the server: br.writeBytes("How are you\n")
This step communicates with the server. The input stream and output stream both communicates with the server.

Step 4 – Close the Socket when done
This function will close the client when it is finally done.
An example of a simple server socket program is shown below

```
// A simple program for the client
import java.net.*;
import java.io.*;
public class SimpleMachineClient {
public static void main(String args[]) throws IOException
{
// At port 1325, connection to the server is opened
Socket s1H = new Socket("host",1325);
// Read the input stream by getting an input file from the socket
Input Stream s1I = s1. getInputStream();
Data Input Stream disH = new Data Input Stream(s1In);
String str = new String (disH.readUTF());
System.out.println(str);
// After it is done, we can close the connection.
disH.close();
s1I.close();
s1H.close();
}
```

UDP Socket Programming
In UDP's terms, data transferred is encapsulated in a unit called datagram. A datagram is an independent, self-contained message sent over the network whose arrival, arrival time, and content are not guaranteed. And in Java, DatagramPacket represents a datagram.



Java DatagramSocket class

Java DatagramSocket class represents a connection-less socket for sending and receiving datagram packets.

Commonly used Constructors of DatagramSocket class

- **DatagramSocket() throws SocketEeption:** it creates a datagram socket and binds it with the available Port Number on the localhost machine.
- **DatagramSocket(int port) throws SocketEeption:** it creates a datagram socket and binds it with the given Port Number.
- **DatagramSocket(int port, InetAddress address) throws SocketEeption:** it creates a datagram socket and binds it with the specified port number and host address.

Java DatagramPacket class

Java DatagramPacket is a message that can be sent or received. If you send multiple packet, it may arrive in any order. Additionally, packet delivery is not guaranteed.

Commonly used Constructors of DatagramPacket class

- **DatagramPacket(byte[] barr, int length):** it creates a datagram packet. This constructor is used to receive the packets.
- **DatagramPacket(byte[] barr, int length, InetAddress address, int port):** it creates a datagram packet. This constructor is used to send the packets.

Example of Sending DatagramPacket by DatagramSocket

```
1. //DSender.java
2. import java.net.*;
3. public class DSender{
4.   public static void main(String[] args) throws Exception {
5.     DatagramSocket ds = new DatagramSocket();
6.     String str = "Welcome java";
7.     InetAddress ip = InetAddress.getByName("127.0.0.1");
8.
9.     DatagramPacket dp = new DatagramPacket(str.getBytes(), str.length(), ip, 3000);
10.    ds.send(dp);
11.    ds.close();
12.  }
13. }
14.
```

Example of Receiving DatagramPacket by DatagramSocket

```
1. //DReceiver.java
2. import java.net.*;
3. public class DReceiver{
4.   public static void main(String[] args) throws Exception {
5.     DatagramSocket ds = new DatagramSocket(3000);
6.     byte[] buf = new byte[1024];
7.     DatagramPacket dp = new DatagramPacket(buf, 1024);
8.     ds.receive(dp);
9.     String str = new String(dp.getData(), 0, dp.getLength());
10.    System.out.println(str);
11.    ds.close();
12.  }
13. }
```

Result : Thus the Socket programming in Java is learnt

Ex No:2 **Implementation of ECHO / PING / DATE & TIME commands through Sockets**

Aim: To implement the following commands through Sockets

a) ECHO
b) PING
c) DATE & TIME

a) Implementation of ECHO Command

This command displays messages, or turns command echoing on or off.

CLIENT SIDE

1. Start the program.
2. Create a socket which binds the Ip address of server and the port address to acquire service.
3. After establishing connection send a data to server.
4. Receive and print the same data from server.
5. Close the socket.
6. End the program.

SERVER SIDE

1. Start the program.
2. Create a server socket to activate the port address.
3. Create a socket for the server socket which accepts the connection.
4. After establishing connection receive the data from client.
5. Print and send the same data to client.
6. Close the socket.
7. End the program.

Implementation of ECHO

SERVER :

```
import java.io.*;
import java.net.*;
public class EchoServer
{
public EchoServer(int portnum)
{
try
{
Server=new ServerSocket(portnum);
}
catch(IOException e)
{
System.out.println("Error"+e);
}
}
public void Server()
{
try
{
while(true)
{
Socket client=Server.accept();
BufferedReader r=new BufferedReader(new
InputStreamReader(client.getInputStream()));
PrintWriter w=new PrintWriter(client.getOutputStream(),true);
w.println("welcome to the Java EchoServer...Type 'bye' to close the window");
String line;
do
{
line=r.readLine();
if(line!=null)
w.println("Echoes Message is:"+line);
System.out.println("The client message is:"+line);
}
while(!line.trim().equals("Bye"));
client.close();
}
}
catch(IOException e)
```

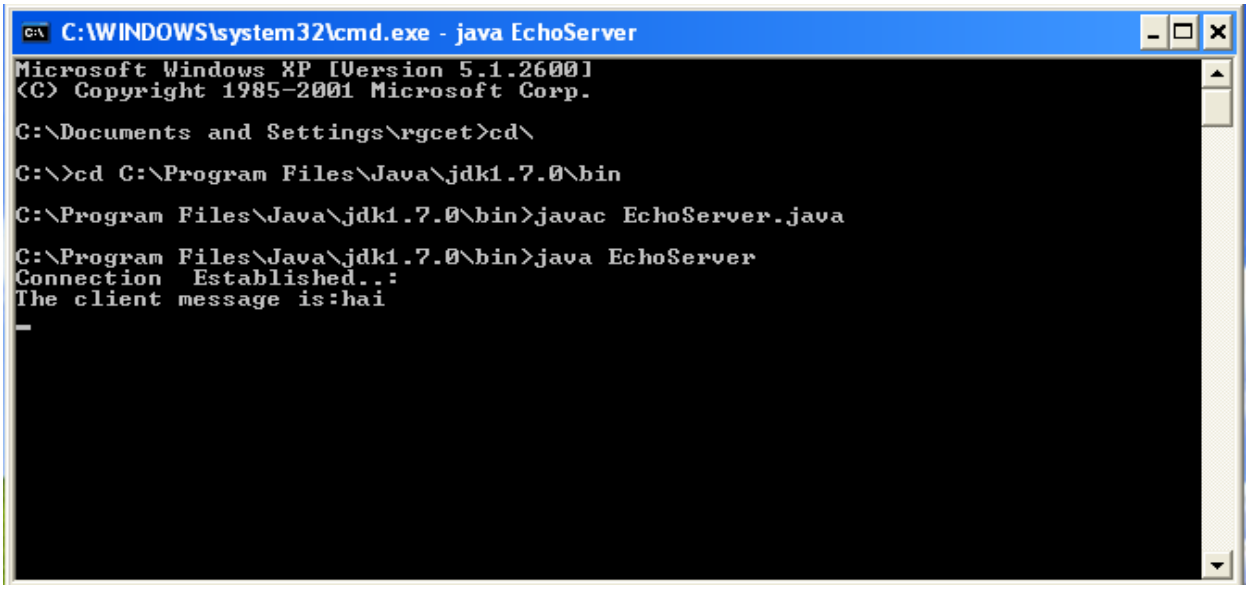
```
{
System.out.println("Error"+e);
}
}
public static void main(String args[])
{
EchoServer s=new EchoServer(8888);
System.out.println("Connection  Established..:");
s.Server();
}
private ServerSocket Server;
}
```

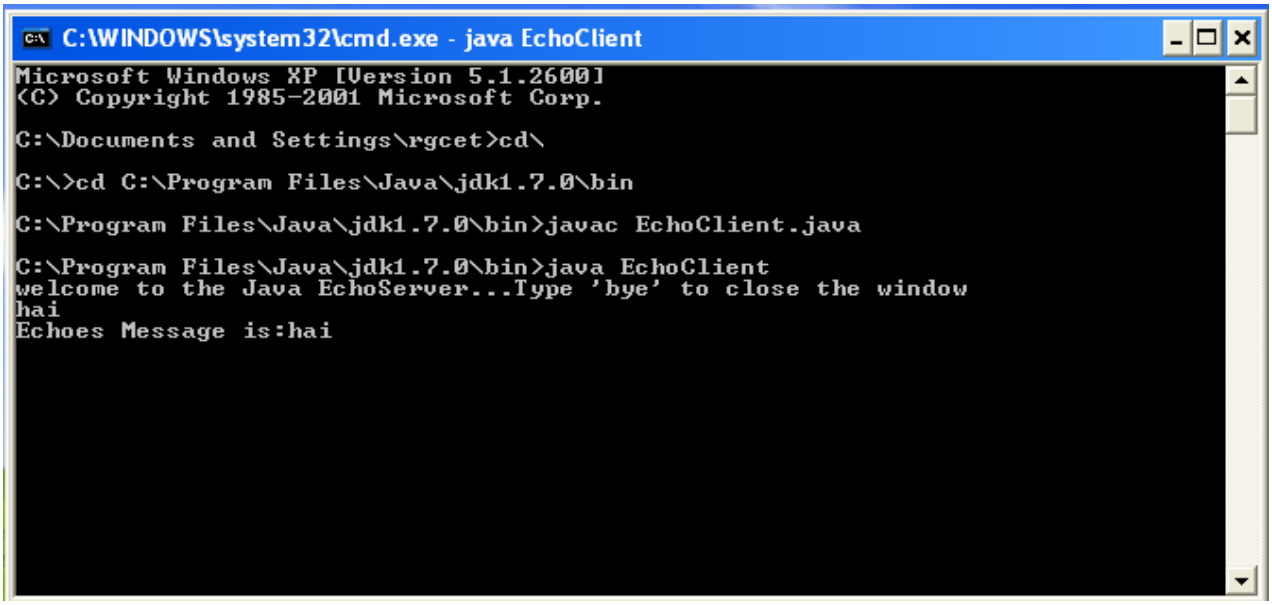
CLIENT:

```
import java.io.*;
import java.net.*;
public class EchoClient
{
public static void main(String args[])
{
try
{
Socket s=new Socket("127.0.0.1",8888);
BufferedReader r=new BufferedReader(new InputStreamReader(s.getInputStream()));
PrintWriter w=new PrintWriter(s.getOutputStream(),true);
BufferedReader con=new BufferedReader(new InputStreamReader(System.in));
String line;
do
{
line=r.readLine();
if(line!=null)
System.out.println(line);
line=con.readLine();
w.println(line);
}
while(!line.trim().equals("Bye"));
}
catch(IOException e)
{
System.out.println(e);
}
}
}
```

Ex No 2A Implementation of Echo

OUTPUT:



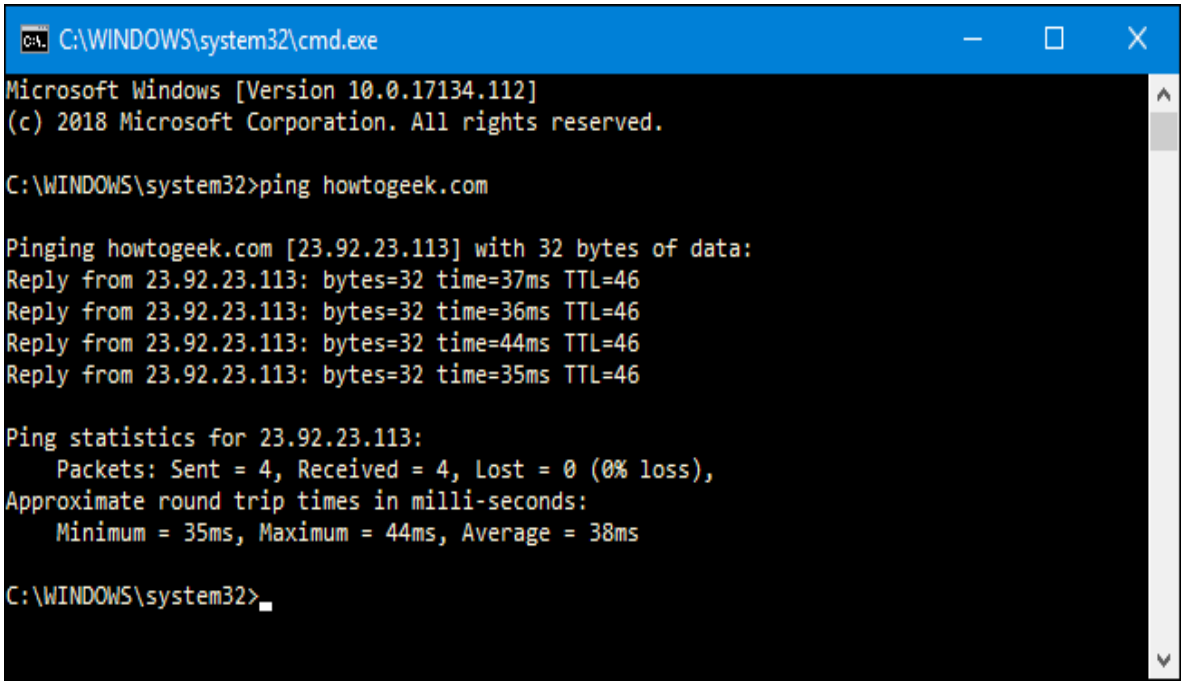


Result: Thus, the program to implement ECHO command is executed and verified

b) Implementation of PING Command

PING [Packet InterNet Gropher] is a command-line utility, available on virtually any operating system with network connectivity, that acts as a test to see if a networked device is reachable.

The ping command sends a request over the network to a specific device. A successful ping results in a response from the computer that was pinged back to the originating computer.



Algorithm

Ping Server:

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Use the RunTime class to capture the IP Address given from client
7. The Process class is instantiated and ping command is executed.
8. Close the socket connections
9. End of the program

Ping Client

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define Socket with required details
4. Get connected with the server program through common port address
5. Declare and Define the input and output stream objects
6. Send the IP Address to the Server
7. Get the PING Statistics from the Server and display
8. Close the socket connections
9. End of the program

Ex No 2b - Implementation of PING

SERVER :

```
import java.io.*;
import java.net.*;
class Pingserver
{
public static void main(String args[])throws IOException
{
```

```
try
{
ServerSocket ss=new ServerSocket(1100);
Socket s=ss.accept();
PrintStream ps=new PrintStream(s.getOutputStream());
BufferedReader br1=new BufferedReader(new InputStreamReader(s.getInputStream()));
String ip;
ip=br1.readLine();
Runtime r=Runtime.getRuntime();
Process p=r.exec("ping "+ip);
BufferedReader br2=new BufferedReader(new InputStreamReader(p.getInputStream()));
String str;
while((str=br2.readLine())!=null)
{
ps.println(str);
}
}
catch(IOException e)
{
System.out.println("Error" +e);
}
}
}
```

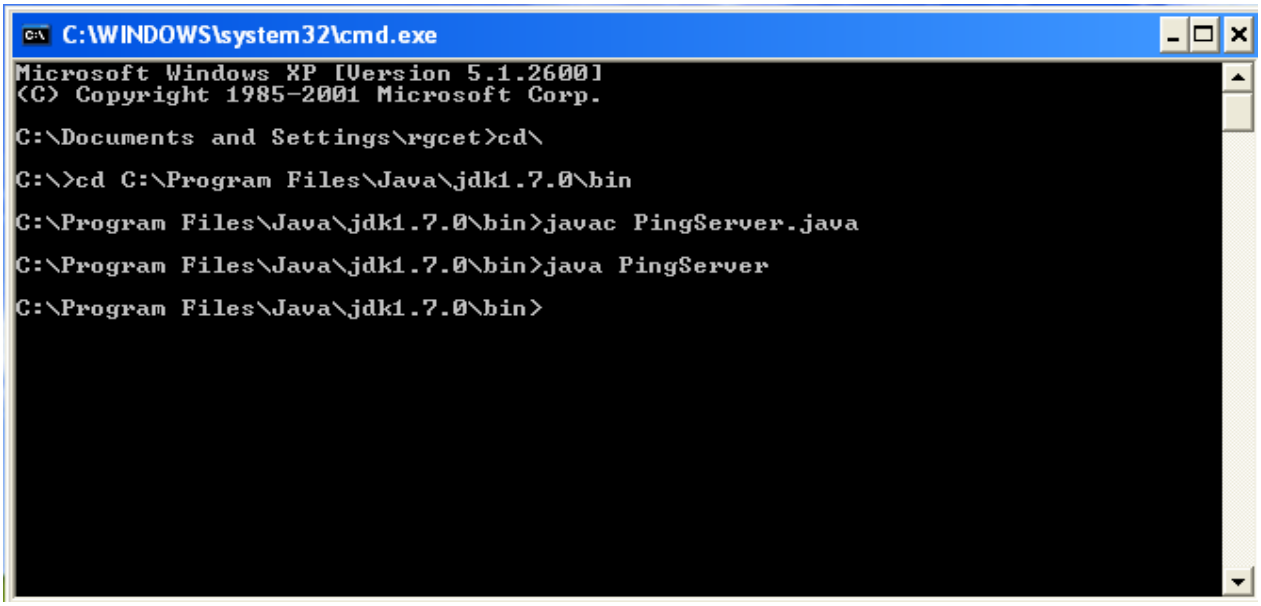
CLIENT:

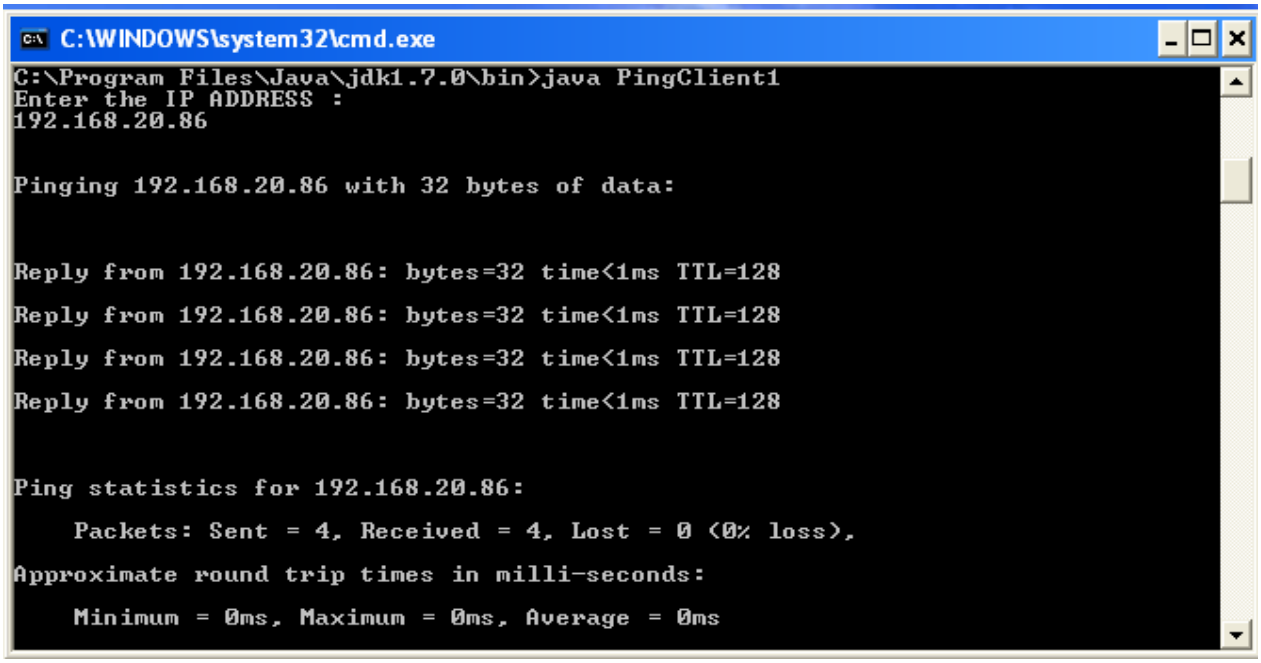
```
import java.io.*;
import java.net.*;
class PingClient
{
public static void main(String args[])throws IOException
{
try
{
Socket ss=new Socket(InetAddress.getLocalHost(),1100);
PrintStream ps=new PrintStream(ss.getOutputStream());
```

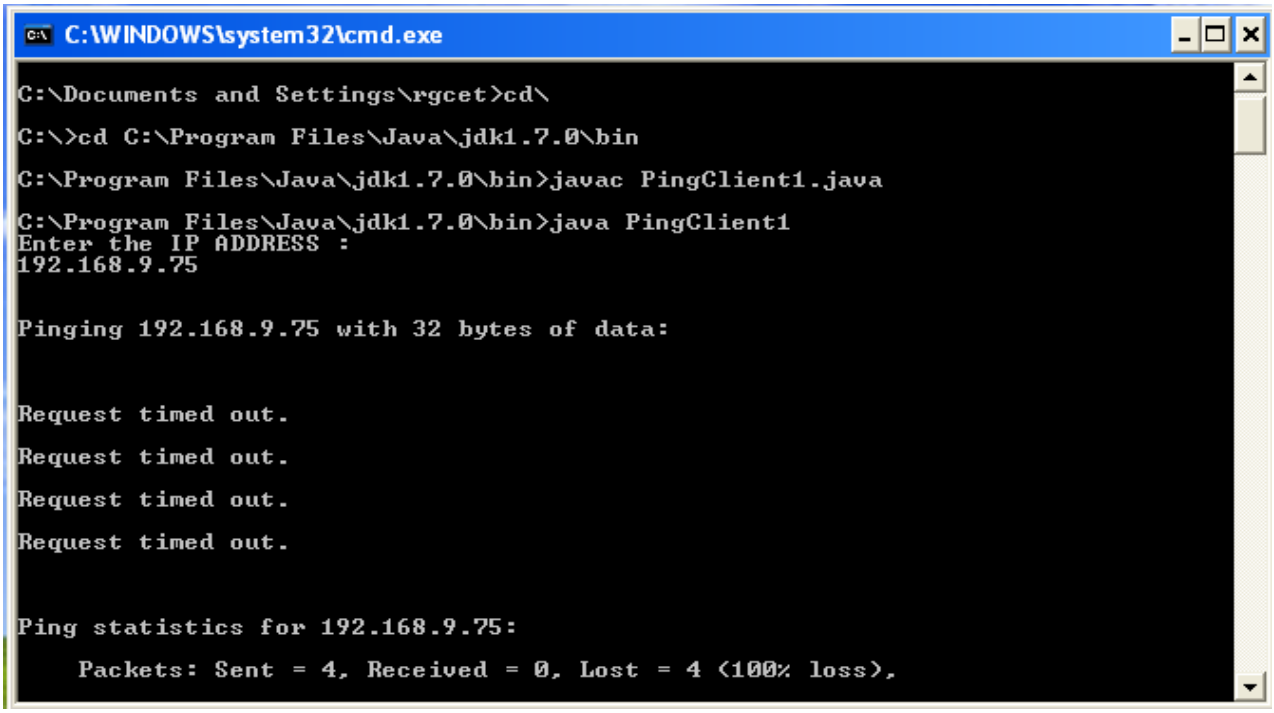
```
BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
String ip;
System.out.println("Enter the IP ADDRESS :");
ip=br.readLine();
ps.println(ip);
String str,data;
BufferedReader br2=new BufferedReader(new InputStreamReader(ss.getInputStream()));
do
{
str=br2.readLine();
System.out.println(str);
}
while(!(str.equalsIgnoreCase("end")));
}
catch(IOException e)
{
System.out.println("Error"+e);
}
}
}
```

OUTPUT:

Ex No 2b - Implementation of PING


C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [Version 5.1.2600]
(C) Copyright 1985-2001 Microsoft Corp.
C:\Documents and Settings\rgcet>cd\
C:\>cd C:\Program Files\Java\jdk1.7.0\bin
C:\Program Files\Java\jdk1.7.0\bin>javac PingServer.java
C:\Program Files\Java\jdk1.7.0\bin>java PingServer
C:\Program Files\Java\jdk1.7.0\bin>


C:\WINDOWS\system32\cmd.exe
C:\Program Files\Java\jdk1.7.0\bin>java PingClient1
Enter the IP ADDRESS :
192.168.20.86
Pinging 192.168.20.86 with 32 bytes of data:
Reply from 192.168.20.86: bytes=32 time<1ms TTL=128
Reply from 192.168.20.86: bytes=32 time<1ms TTL=128
Reply from 192.168.20.86: bytes=32 time<1ms TTL=128
Reply from 192.168.20.86: bytes=32 time<1ms TTL=128
Ping statistics for 192.168.20.86:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms

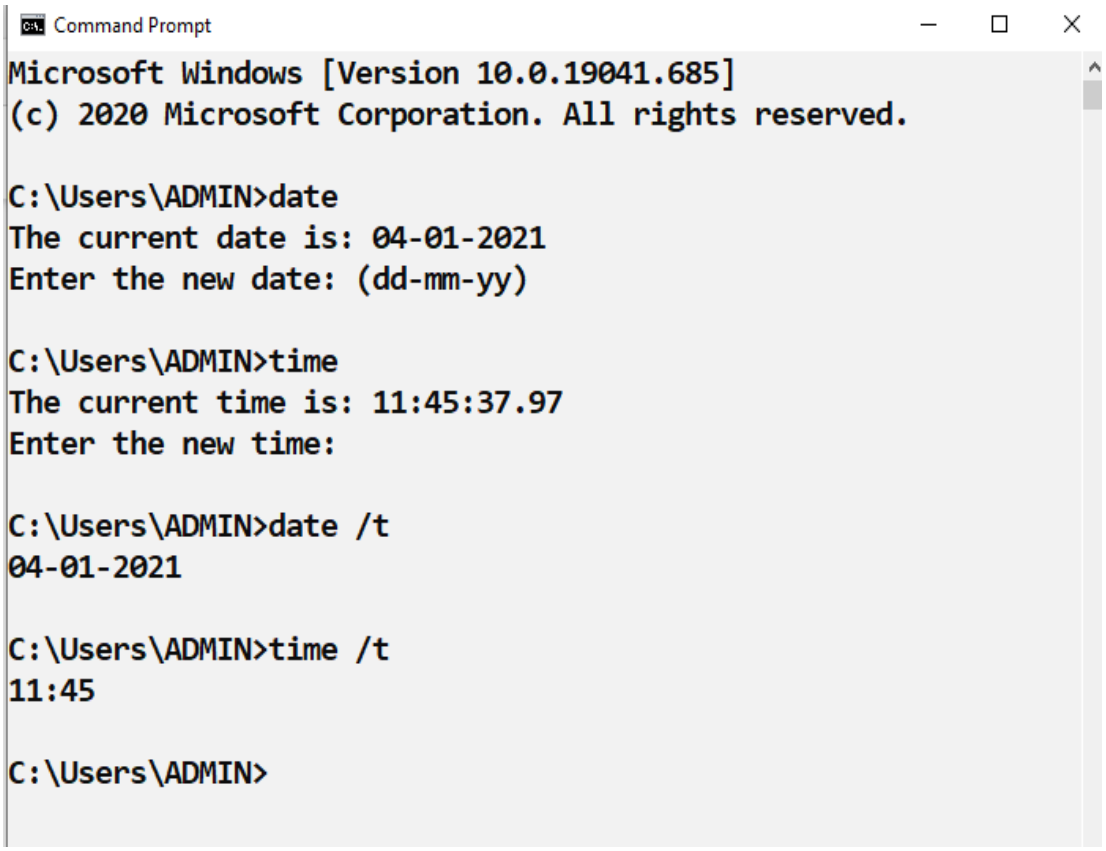


Result:

Thus, the program to implement PING command is executed and verified

c) Implementation of DATE & TIME Command

The **date** command can view or change the current date of the system clock. The Time command sets or displays the time.



Algorithm

DateTimeServer

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Use the DATE class to find the current date & time of the system
7. Send the current date & time to the client
8. Close the socket connections
9. End of the program

DateTimeClient

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define Socket with required details
4. Get connected with the server program through common port address
5. Declare and Define the input and output stream objects
6. Get the DATE and TIME from the Server and display
7. Close the socket connections
8. End of the program

Ex No 2c - Implementation of DATE & TIME

Server

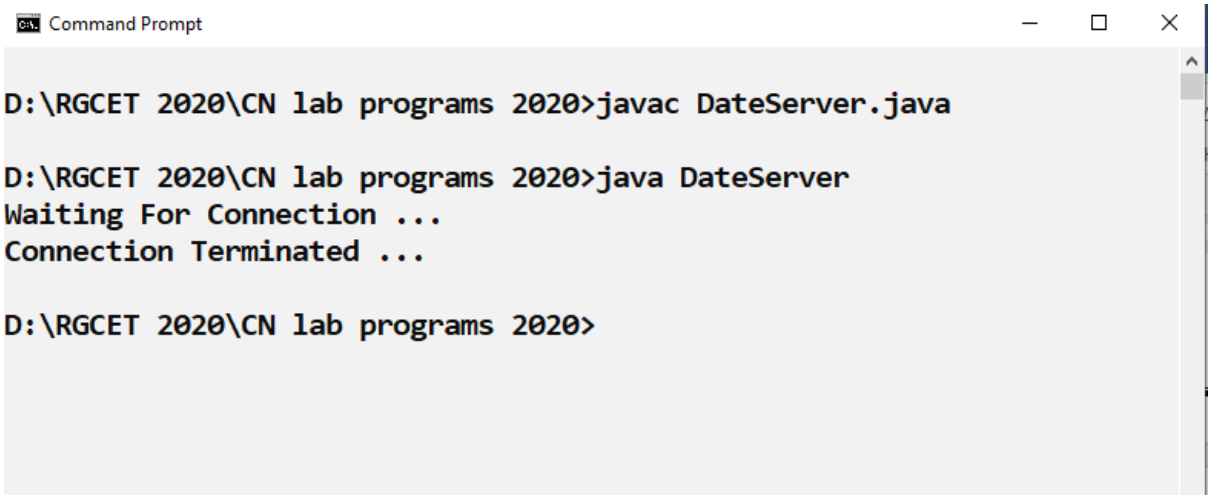
```
import java.net.*;
import java.io.*;
import java.util.*;
class DateServer
{
   public static void main(String args[]) throws Exception
   {
      ServerSocket s=new ServerSocket(5217);
         System.out.println("Waiting For Connection ...");
         Socket soc=s.accept();
      DataOutputStream out=new DataOutputStream(soc.getOutputStream());
      out.writeBytes("Server Date: " + (new Date()).toString() + "\n");
      out.close();
         System.out.println("Connection Terminated ...");
         soc.close();

   }
}
```

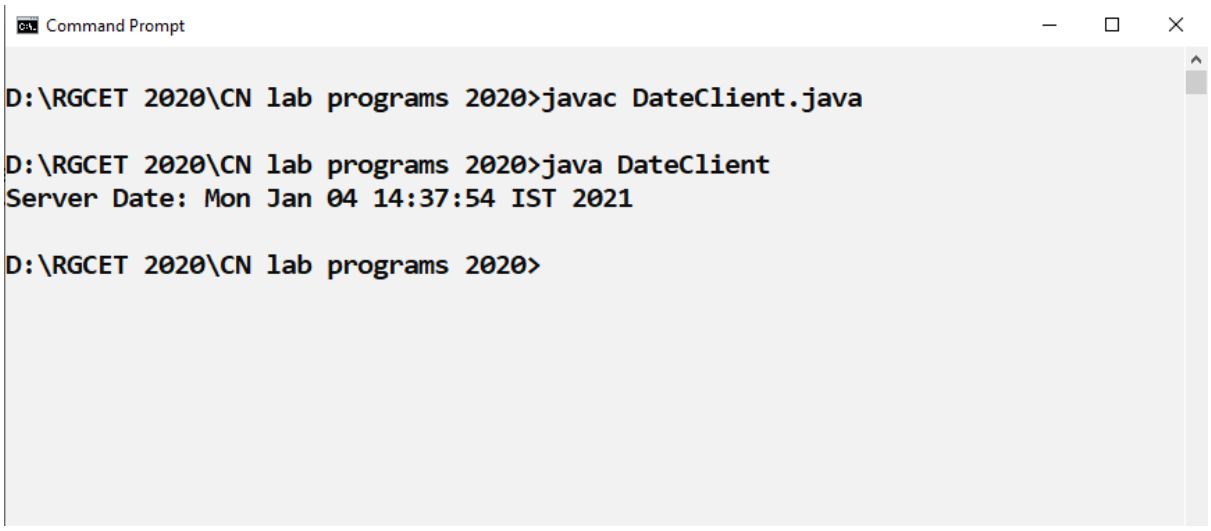
Client

```
import java.io.*;
import java.net.*;
class DateClient
{
   public static void main(String args[]) throws Exception
   {
      Socket soc=new Socket(InetAddress.getLocalHost(),5217);
      BufferedReader in=new BufferedReader(new InputStreamReader(soc.getInputStream()
));
      System.out.println(in.readLine());
   }
}
```

Ex No 2c - Implementation of DATE & TIME

OUTPUT





Result:

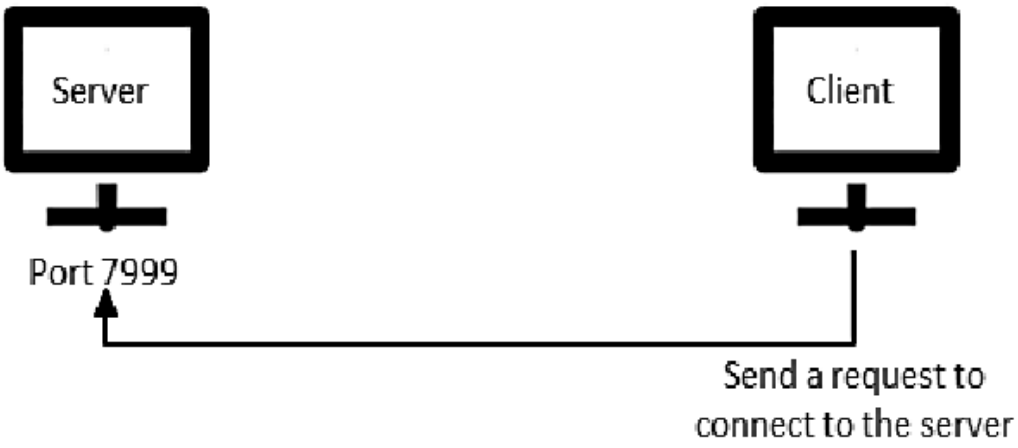
Thus, the program to implement DATE & TIME command is executed and verified

Ex No:3 **Implementation of ONE-WAY COMMUNICATION through TCP-Client & Server Sockets**

Aim : To implement the one way communication through TCP Sockets

a) **Client to Server**
b) **Server to Client**

3 a) Client to Server One-way Communication



Algorithm

WishesServer

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Display the message received from client
7. Close the socket connections
8. End of the program

WishesClient

1. Start the Program
2. Import JAVA.NET and other necessary packages

3. Declare and define Socket with required details
4. Get connected with the server program through common port address
5. Declare and Define the input and output stream objects
6. Send Greetings messages to server
7. Close the socket connections
8. End of the program

Ex No 3a – Client to Server One-way communication

WishesServer

```
import java.net.ServerSocket;
import java.net.Socket;
import java.io.InputStream;
import java.io.DataInputStream;

public class WishesServer
{
  public static void main(String args[]) throws Exception
  {
    ServerSocket sersock = new ServerSocket(5000);
    System.out.println("server is ready");
   Socket sock = sersock.accept();

    InputStream istream = sock.getInputStream();
    DataInputStream dstream = new DataInputStream(istream);

    String message2 = dstream.readLine();
    System.out.println(message2);
    dstream .close(); istream.close(); sock.close(); sersock.close();
  }
}
```

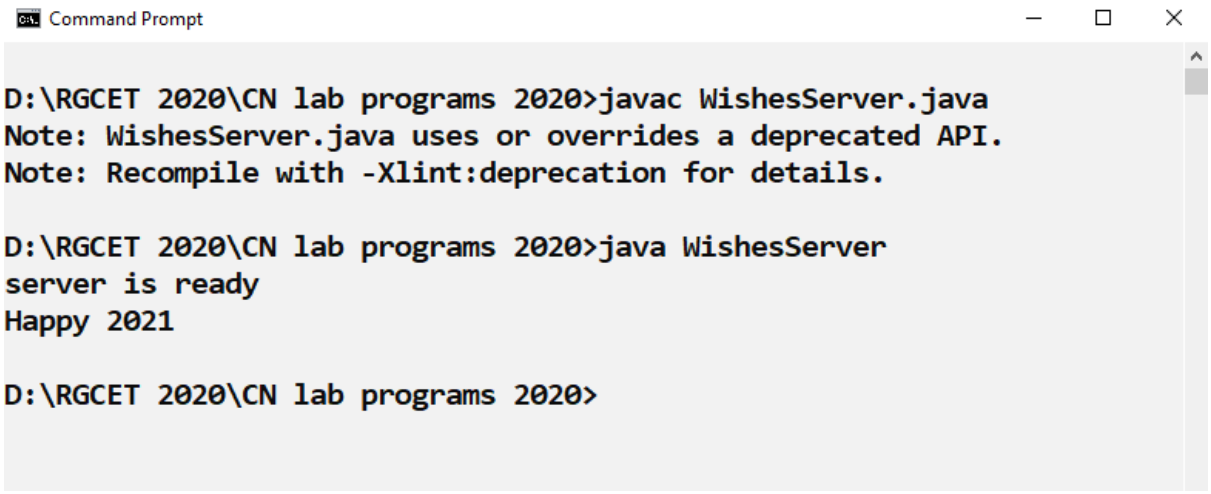
WishesClient

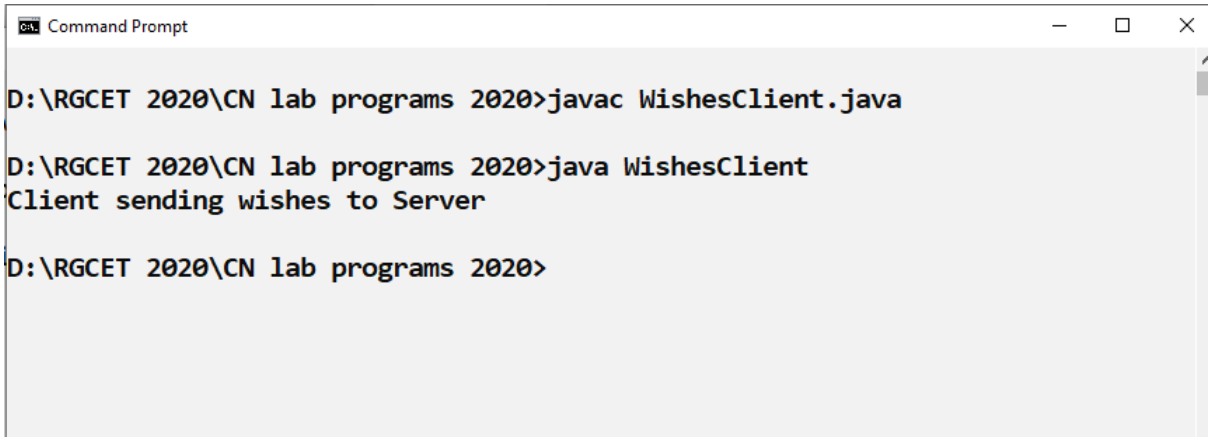
```
import java.net.Socket;
import java.io.OutputStream;
import java.io.DataOutputStream;

public class WishesClient
{
  public static void main(String args[]) throws Exception
  {
    Socket sock = new Socket("127.0.0.1", 5000);
    System.out.println("Client sending wishes to Server");
    String message1 = "Happy 2021";
```

```
    OutputStream ostream = sock.getOutputStream();
    DataOutputStream dos = new DataOutputStream(ostream);
    dos.writeBytes(message1);
    dos.close();
    ostream.close();
    sock.close();
  }
}
```

Ex No 3a – Client to Server One-way communication

OUTPUT

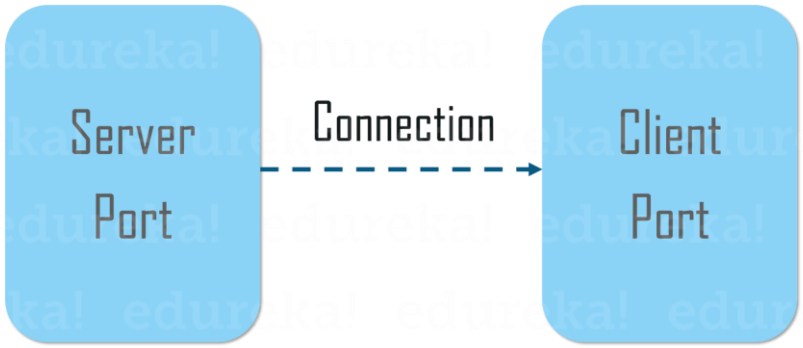




Result:

Thus, the program to implement Client to Server One-way communication is executed and verified

3 b) Server to Client One-way Communication



Algorithm

DateInfoServer

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Sent message to client
7. Close the socket connections
8. End of the program

DateInfoClient

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define Socket with required details
4. Get connected with the server program through common port address
5. Declare and Define the input and output stream objects
6. Display the message received from Server
7. Close the socket connections
8. End of the program

Ex No 3b – Server to Client One-way communication

DateInfoServer

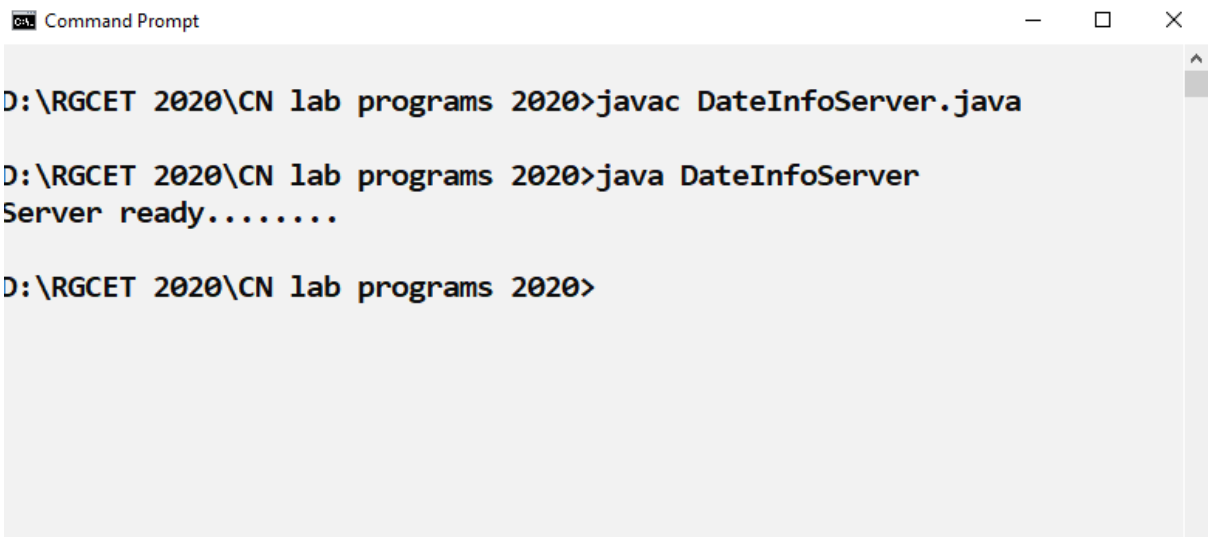
```
import java.net.*;
import java.io.*;
public class DateInfoServer
{
  public static void main(String args[]) throws Exception
  {
    ServerSocket sersock = new ServerSocket(7000);
    System.out.println("Server ready.....");
    Socket sock = sersock.accept( );

    OutputStream ostream = sock.getOutputStream();
    BufferedWriter bw1 = new BufferedWriter(new OutputStreamWriter(ostream));
    String s2 = "Thanks client for your calling on " + new java.util.Date();
    bw1.write(s2);

    bw1.close();    ostream.close();   sock.close();   sersock.close( );
  }
}
```

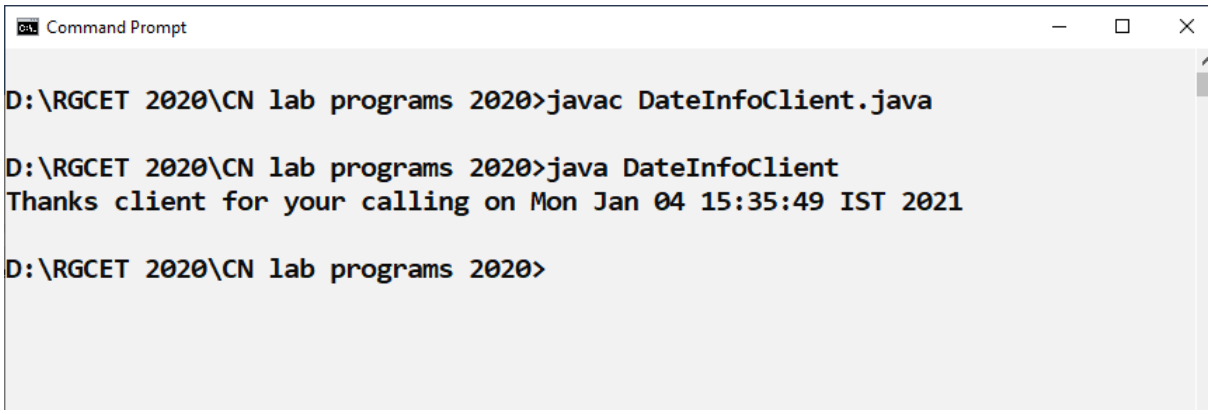
DateInfoClient

```
import java.io.*;
import java.net.*;
public class DateInfoClient
{
  public static void main(String args[]) throws Exception
  {
    Socket sock = new Socket("127.0.0.1", 7000);

    InputStream istream = sock.getInputStream();
    BufferedReader br1 = new BufferedReader(new InputStreamReader(istream));

    String s1 = br1.readLine();
    System.out.println(s1);

    br1.close();    istream.close();   sock.close( );
  }
}
```

Ex No 3b – Server to Client One-way communication

OUTPUT





Result:

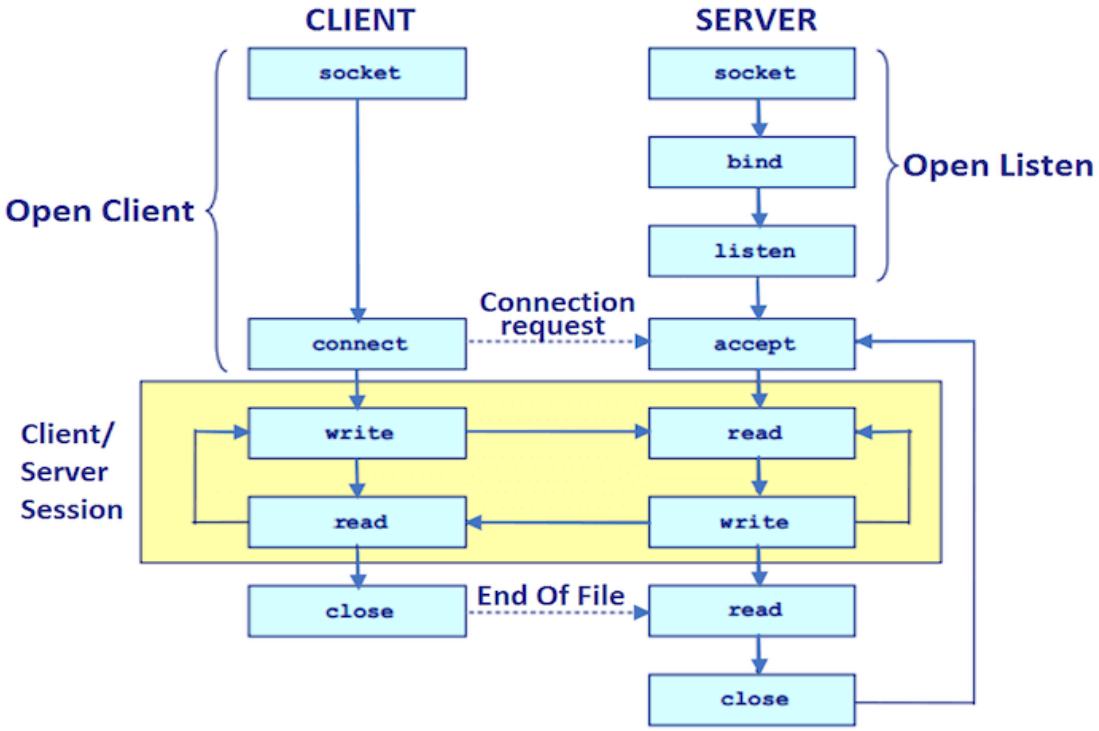
Thus, the program to implement Server to Client One way communication is executed and verified

Ex No:4 **Implementation of Two-way chat communication**

Aim: To implement the Two-way communication through

a) **TCP Sockets**
b) **UDP Sockets**

4 a) Two-Way communication through TCP Sockets



Algorithm

TCPChatServer

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Send message to client and receive the response also
7. Continue chatting till STOP is received.
8. End of the program

TCPChatClient

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Sent message to Server and receive the response from Client
7. Continue chatting till STOP is received.
8. End of the program

Ex No 4a – Two Way CHAT communication using TCP Sockets

TCPChatServer

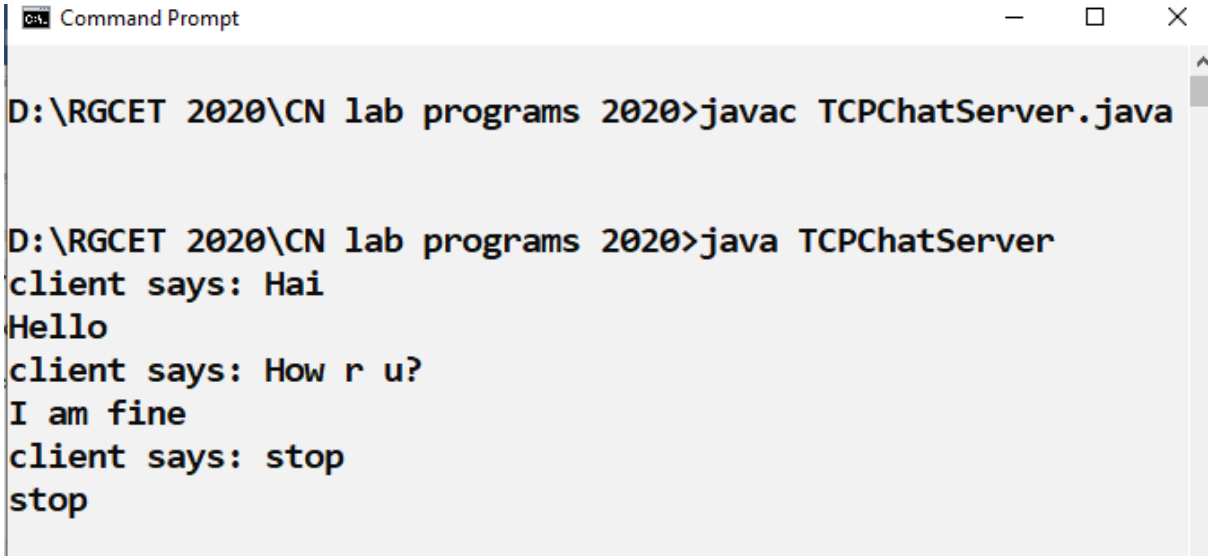
```
import java.net.*;
import java.io.*;
class TCPChatServer{
public static void main(String args[])throws Exception{
ServerSocket ss=new ServerSocket(3333);
Socket s=ss.accept();
DataInputStream din=new DataInputStream(s.getInputStream());
DataOutputStream dout=new DataOutputStream(s.getOutputStream());
BufferedReader br=new BufferedReader(new InputStreamReader(System.in));
 String str="",str2="";
while(!str.equals("stop")){
str=din.readUTF();
System.out.println("client says: "+str);
str2=br.readLine();
dout.writeUTF(str2);
dout.flush();
}
din.close();
s.close();
ss.close();
}}
```

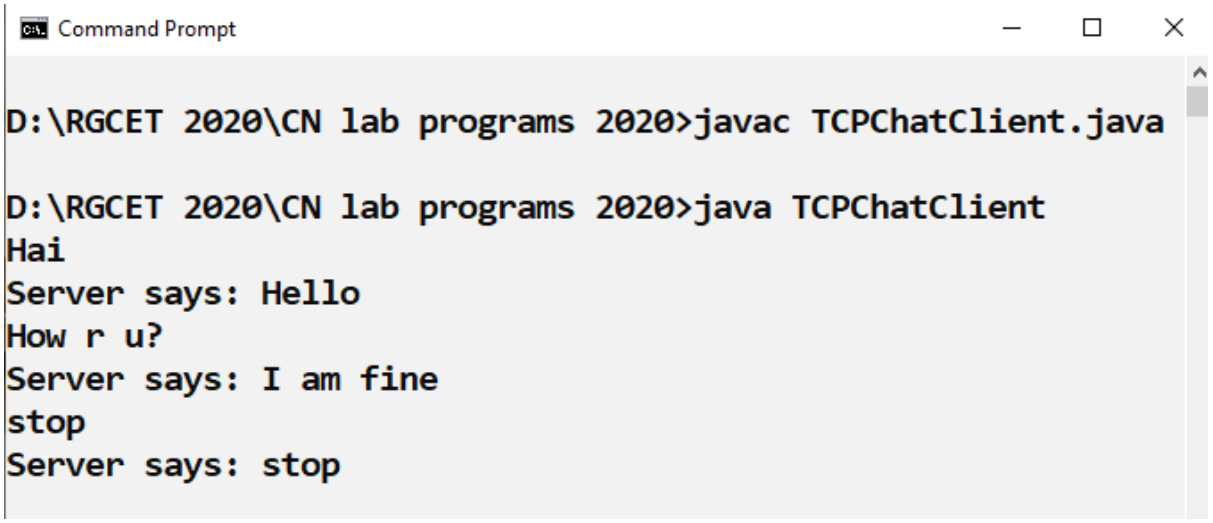
TCPChatClient

```
import java.net.*;
import java.io.*;
class TCPChatClient{
public static void main(String args[])throws Exception{
```

```
Socket s=new Socket("localhost",3333);
DataInputStream din=new DataInputStream(s.getInputStream());
DataOutputStream dout=new DataOutputStream(s.getOutputStream());
BufferedReader br=new BufferedReader(new InputStreamReader(System.in));

String str="",str2="";
while(!str.equals("stop")){
str=br.readLine();
dout.writeUTF(str);
dout.flush();
str2=din.readUTF();
System.out.println("Server says: "+str2);
}

dout.close();
s.close();
}}
```

Ex No 4a – Two Way CHAT communication using TCP Sockets

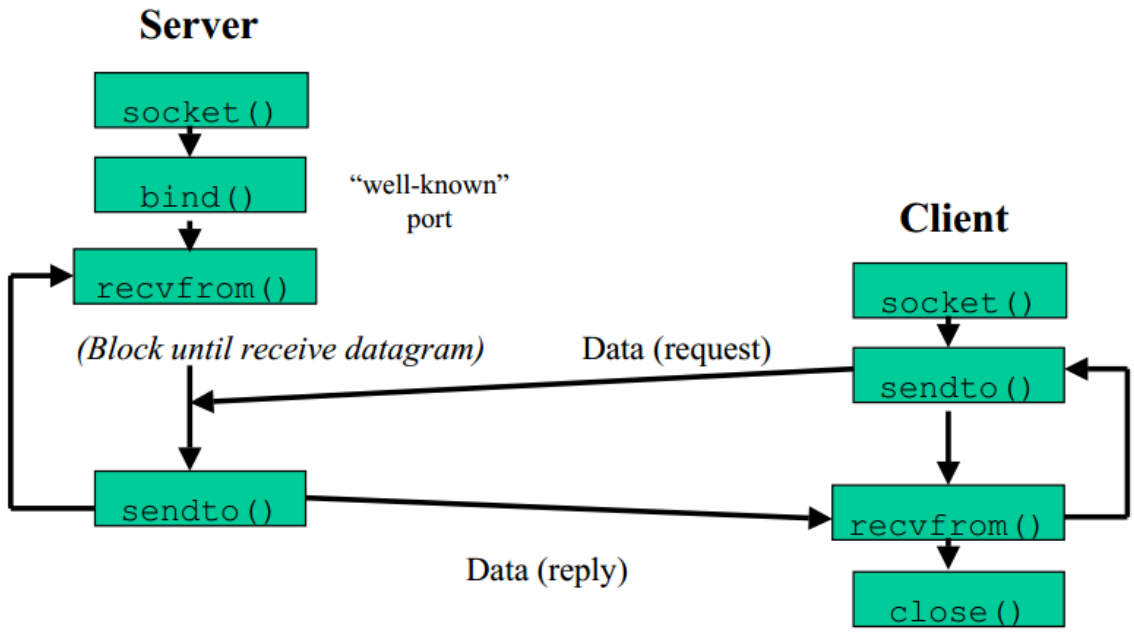




Result:

Thus, the program to implement Two way Chat communication using TCP Sockets is executed and verified

4 b) Two-Way communication through UDP Sockets



Algorithm

UDPServer

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define DatagramSocket & DatagramPacket Objects with required details
4. Declare and Define the input and output stream objects
5. Sent message to client and receive the response from server
6. Continue chatting till STOP is received.
7. End of the program

UDPClient

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define DatagramSocket & DatagramPacket Objects with required details
4. Declare and Define the input and output stream objects
5. Sent message to Server and receive the response from Client
6. Continue chatting till STOP is received.
7. End of the program

Ex No 4b – Two Way CHAT communication using UDP Sockets

UDPServer

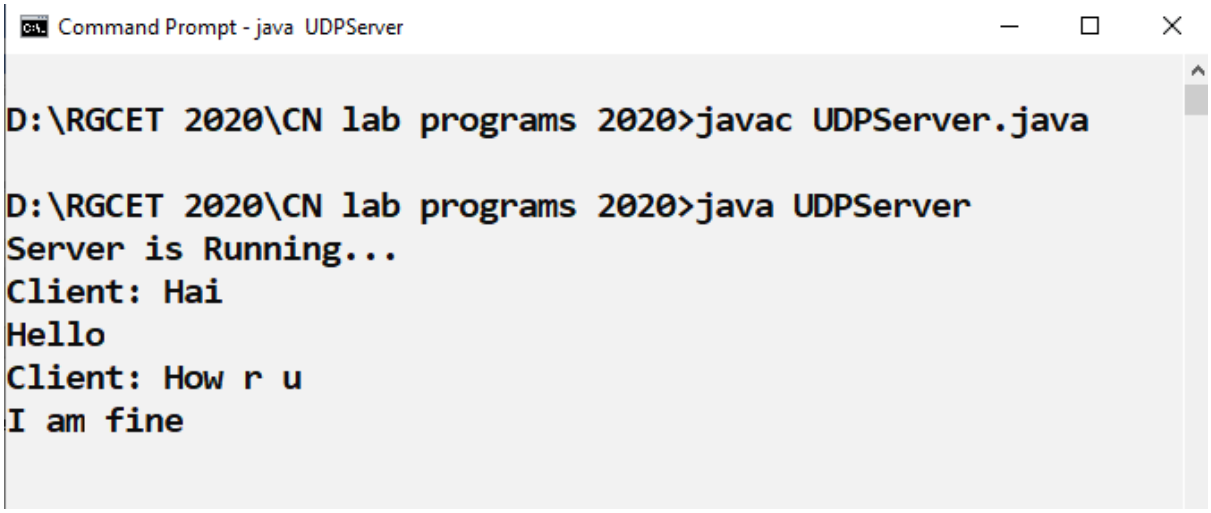
```
import java.io.*;
import java.net.*;
class UDPServer
{
public static DatagramSocket serversocket;
public static DatagramPacket dp;
public static BufferedReader dis;
public static InetAddress ia;
public static byte buf[] = new byte[1024];
public static int cport = 789,sport=790;
public static void main(String[] a) throws IOException
{
serversocket = new DatagramSocket(sport);
dp = new DatagramPacket(buf,buf.length);
dis = new BufferedReader (new InputStreamReader(System.in));
ia = InetAddress.getLocalHost();
System.out.println("Server is Running...");
while(true)
{
serversocket.receive(dp);
String str = new String(dp.getData(), 0,dp.getLength());
if(str.equals("STOP"))
{
System.out.println("Terminated...");
break;
}
System.out.println("Client: " + str);
String str1 = new String(dis.readLine());
buf = str1.getBytes();
serversocket.send(new DatagramPacket(buf,str1.length(), ia, cport));
}
}
}
```

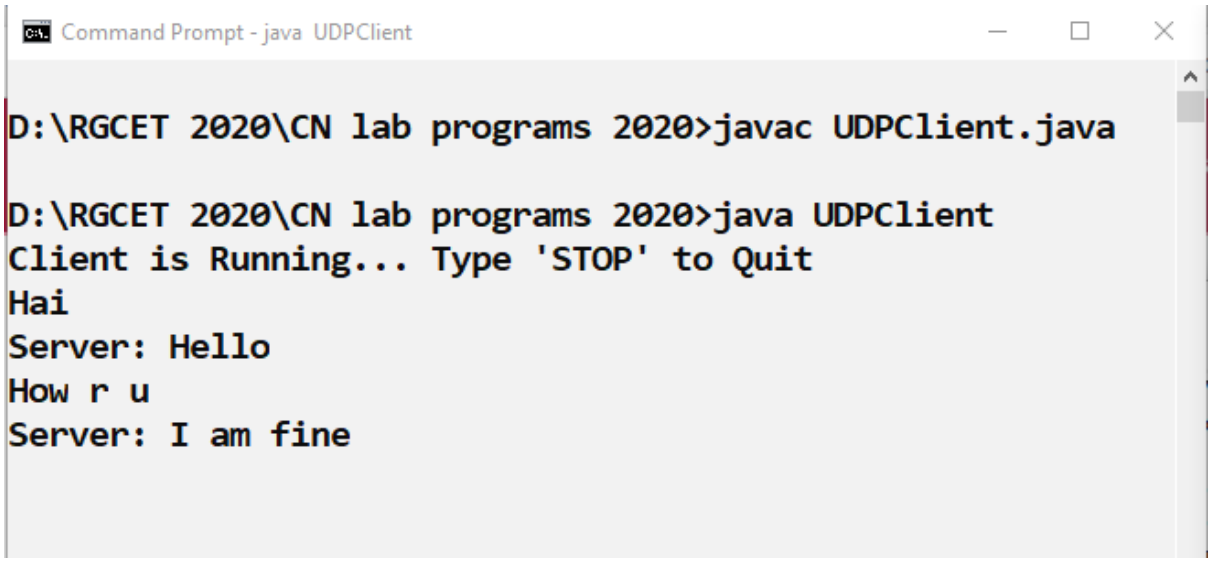
UDPClient

```
import java.io.*;
import java.net.*;
class UDPClient
{
public static DatagramSocket clientsocket;
public static DatagramPacket dp;
public static BufferedReader dis;
public static InetAddress ia;
public static byte buf[] = new byte[1024];
public static int cport = 789, sport = 790;
public static void main(String[] a) throws IOException
{
clientsocket = new DatagramSocket(cport);
dp = new DatagramPacket(buf, buf.length);
dis = new BufferedReader(new InputStreamReader(System.in));
ia = InetAddress.getLocalHost();
System.out.println("Client is Running... Type 'STOP' to Quit");
while(true)
{
String str = new String(dis.readLine());
buf = str.getBytes();
if(str.equals("STOP"))
{
System.out.println("Terminated...");
clientsocket.send(new DatagramPacket(buf,str.length(), ia,sport));
break;
}
clientsocket.send(new DatagramPacket(buf,str.length(), ia, sport));
clientsocket.receive(dp);
String str2 = new String(dp.getData(), 0,dp.getLength());
System.out.println("Server: " + str2);
}
}
}
```

Ex No 4b – Two Way CHAT communication using UDP Sockets

OUTPUT





Result:

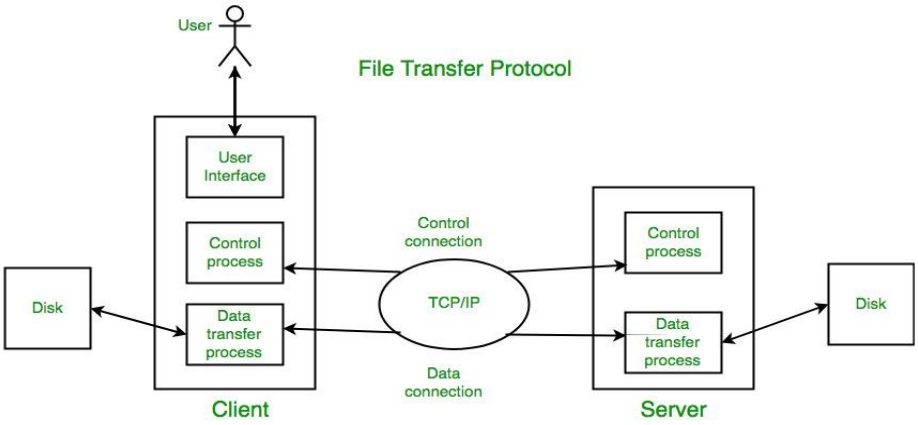
Thus, the program to implement Two way Chat communication using UDP Sockets is executed and verified

Ex No:5

Implementation of File Transfer Protocol

Aim: To implement File transfer protocols using a) TCP and b) UDP

5a) FTP using TCP Sockets



Algorithm

Server

1. Import java packages and create class file server.
2. Create a new server socket and bind it to the port.
3. Accept the client connection
4. Get the file name and stored into the BufferedReader.
5. Create a new object class file and realine.
6. If file is existing then FileReader read the content until EOF is reached.
7. Stop the program.

Client

1. Import java packages and create class file server.
2. Create a new server socket and bind it to the port.
3. Now connection is established.
4. The object of a BufferReader class is used for storing data content which has been retrieved from socket object.
5. The connection is closed.
6. Stop the program.

Ex No 5b – FTP using TCP Sockets

FileTCPServer

```
import java.io.BufferedInputStream;
import java.io.File;
import java.io.FileInputStream;
import java.io.OutputStream;
import java.net.InetAddress;
import java.net.ServerSocket;
import java.net.Socket;

public class FileTCPServer
{
 public static void main(String[] args) throws Exception
{
ServerSocket ssock = new ServerSocket(5000);
Socket socket = ssock.accept();
InetAddress IA = InetAddress.getByName("localhost");
File file = new File("Welcome.txt");
FileInputStream fis = new FileInputStream(file);
BufferedInputStream bis = new BufferedInputStream(fis);
OutputStream os = socket.getOutputStream();
Read File Contents into contents array
byte[] contents;
long fileLength = file.length();
long current = 0;
long start = System.nanoTime();
while(current!=fileLength){
int size = 10000;
if(fileLength - current >= size)
current += size;
else {
size = (int)(fileLength - current);
current = fileLength;
}
contents = new byte[size];
 bis.read(contents, 0, size);
 os.write(contents);
 System.out.print("Sending file ... "+(current*100)/fileLength+"% complete!");
 }
 os.flush();
```

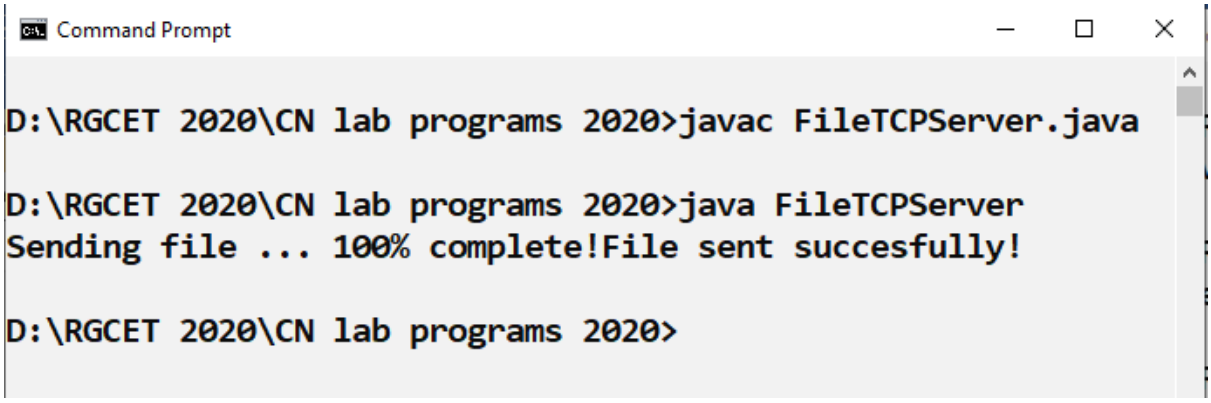
```
 //File transfer done. Close the socket connection!
 socket.close();
 ssock.close();
 System.out.println("File sent succesfully!");
 }
}
```

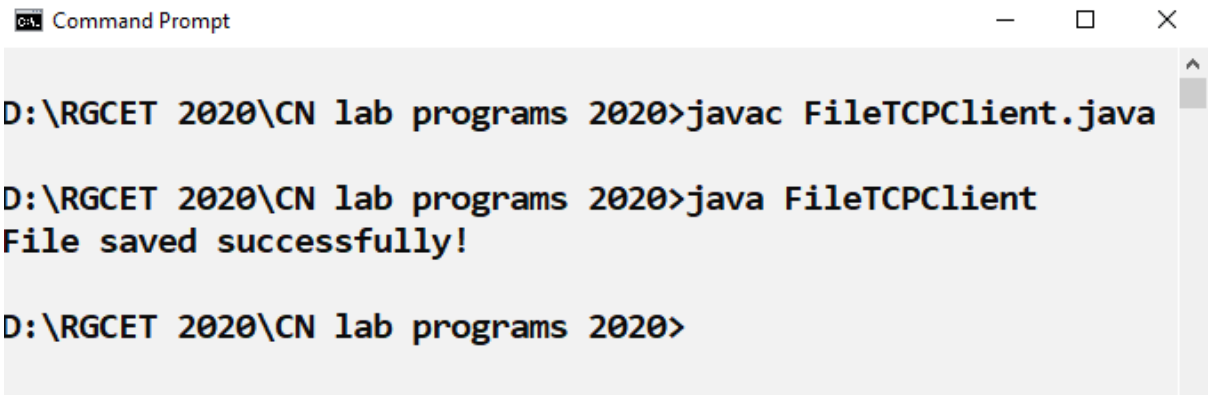
FileTCPClient

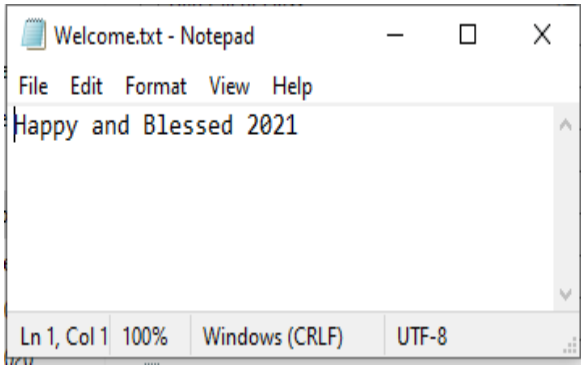
```
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.InputStream;
import java.net.InetAddress;
import java.net.Socket;
public class FileTCPClient {

public static void main(String[] args) throws Exception{
Socket socket = new Socket(InetAddress.getByName("localhost"), 5000);
byte[] contents = new byte[10000];
FileOutputStream fos = new FileOutputStream("Welcome1.txt");
BufferedOutputStream bos = new BufferedOutputStream(fos);
InputStream is = socket.getInputStream();
int bytesRead = 0;
while((bytesRead=is.read(contents))!=-1)
bos.write(contents, 0, bytesRead);
bos.flush();
socket.close();
System.out.println("File saved successfully!");
}
}
```

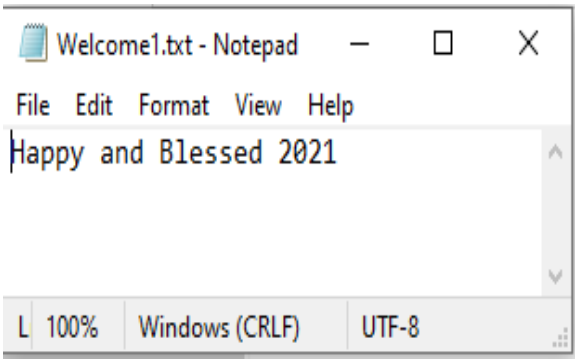
Ex No 5b – FTP using TCP Sockets

OUTPUT



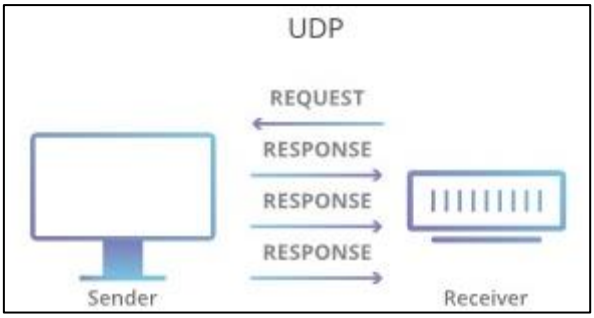






Result: Thus, FTP is implemented and verified using TCP Sockets

5b) FTP using UDP Sockets



Algorithm

Server

1. Import java packages and create class file server.
2. Create a new Datagram socket and bind it to the port.
3. Accept the client connection
4. Use the objects of File class to open the file for reading
5. The object of a File stream input and output classes are used for storing data content which has been retrieved from socket object.
6. Stop the program.

Client

1. Import java packages and create class file server.
2. Create a new Datagram socket and bind it to the port.
3. Request the file transfer
4. Use the objects of File class to open the file for writing
5. The object of a File stream input and output classes are used for storing data content which has been retrieved from socket object.
6. Stop the program.

Ex No 5b – FTP using UDP Sockets

FileUDPserver

```
import java.net.*;
import java.io.*;
public class FileUDPserver
{
        public static void main(String args[])throws IOException
         {
                byte b[]=new byte[3072];
                DatagramSocket dsoc=new DatagramSocket(1000);
                FileOutputStream f=new FileOutputStream("greetings1.txt");
                while(true)
                {
                        DatagramPacket dp=new DatagramPacket(b,b.length);
                        dsoc.receive(dp);
                        String r=new String(dp.getData());
                        byte[] n=r.getBytes();
                        int len=dp.getLength();
                        while(len!=0)
                        {
                        f.write(n);
                        break;
                        }
                       System.out.println(new String(dp.getData(),0,dp.getLength()));
                      f.close();
                      break;
                }
                dsoc.close();
        }
}
```

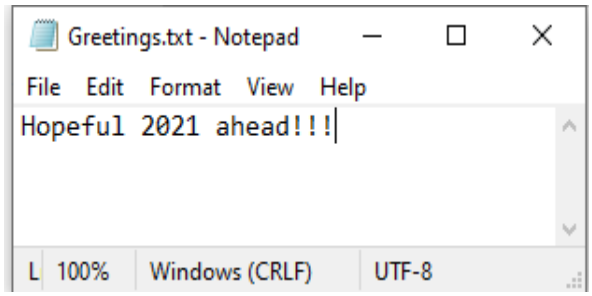
FileUDPclient

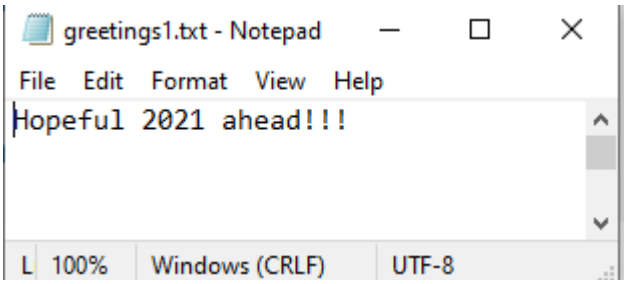
```
import java.net.*;
import java.io.*;
public class FileUDPclient
{
        public static void main(String args[])throws Exception
        {
                byte b[]=new byte[1024];
                FileInputStream f=new FileInputStream("Greetings.txt");
                DatagramSocket dsoc=new DatagramSocket(2000);
```

```
                int i=0;
                while(f.available()!=0)
                {
                        b[i]=(byte)f.read();
                        i++;
                }
                f.close();
                dsoc.send(new DatagramPacket(b,i,InetAddress.getLocalHost(),1000));
                System.out.println("File Sent successfully");
        }

}
```

Ex No 5b – FTP using UDP Sockets

OUTPUT





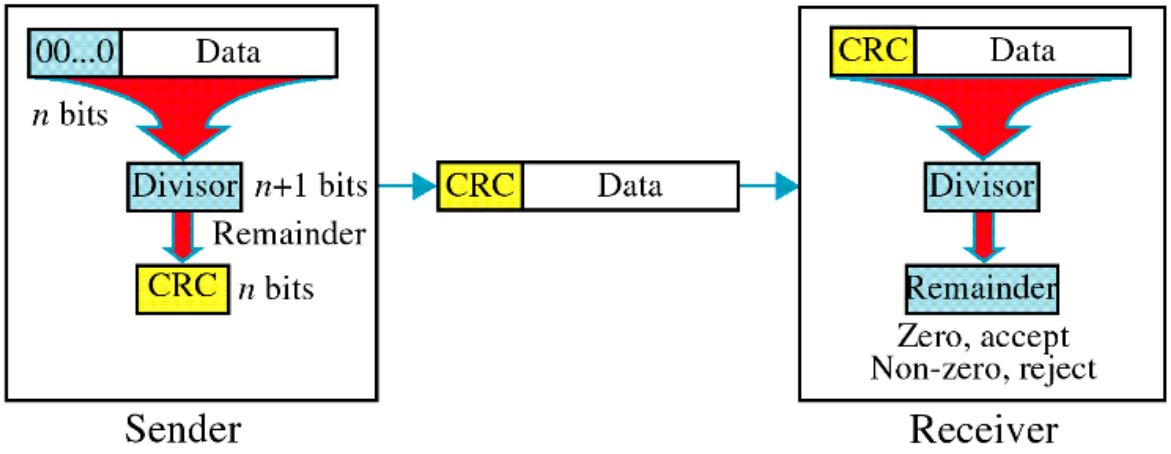
Result: Thus, FTP is implemented and verified using UDP Sockets

Ex No:6 **Implementation of Cyclic Redundancy Check**

Aim: To implement the simulation of Cyclic Redundancy Check

Cyclic Redundancy code

CRC or Cyclic Redundancy Check is a method of detecting accidental changes/errors in the communication channel. CRC uses Generator Polynomial which is available on both sender and receiver side. An example generator polynomial is of the form like x3 + x + 1. This generator polynomial represents key 1011. Another example is x2 + 1 that represents key 101.



Algorithm

CRC

1. Start the Program
2. Import necessary packages for the program
3. Declare and define input variables for generator, divisor and data with required details
4. Get the input in the form of bits.
5. Append the redundancy bits.
6. Divide the appended data using a divisor polynomial.
7. The resulting data should be transmitted to the receiver.
8. At the receiver the received data is entered.
9. The same process is repeated at the receiver.
10. If the remainder is zero there is no error otherwise there is some error in the received bits
11. End of the program

Ex No. 6 – Cyclic Redundancy Code

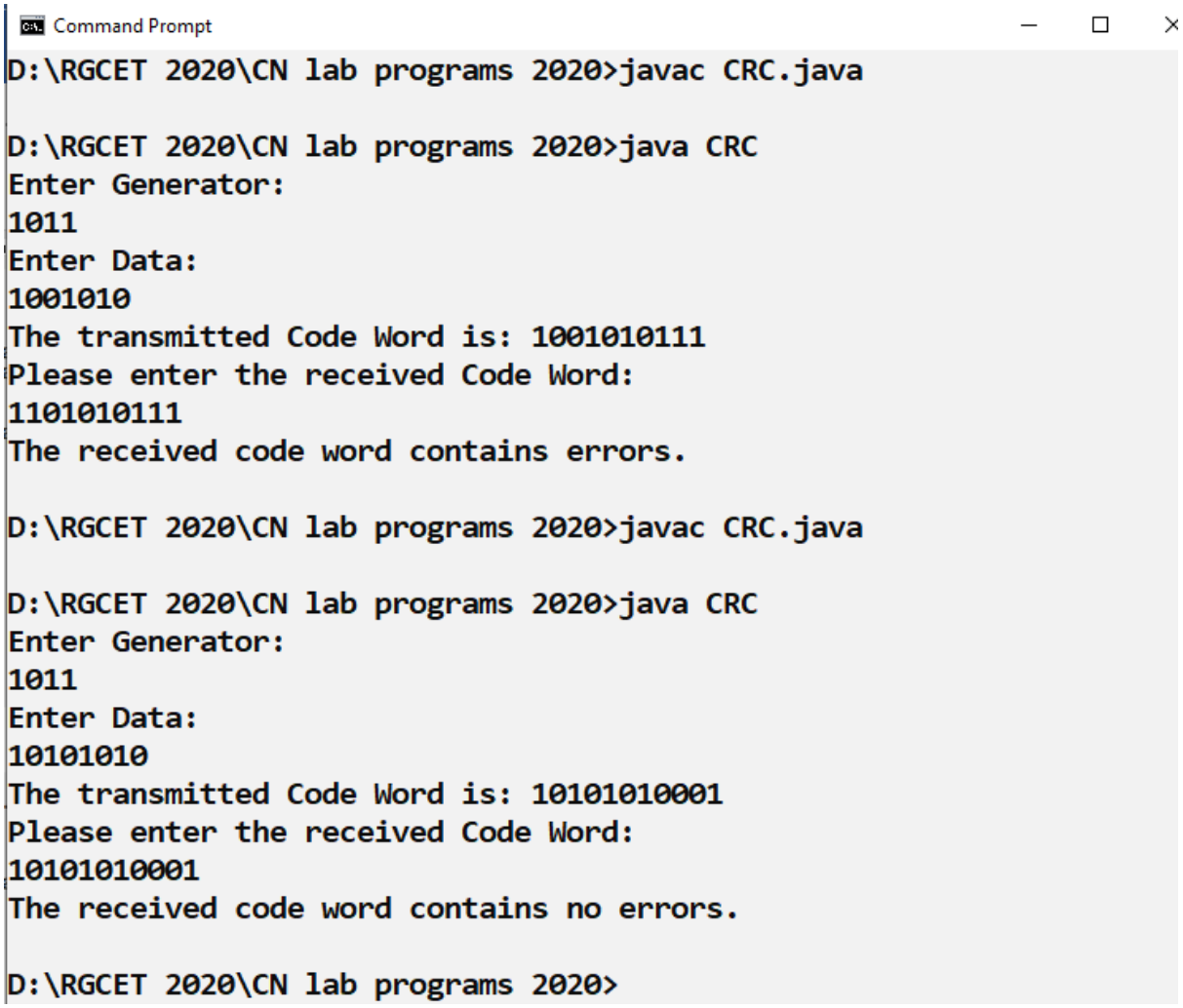
CRC

```
import java.io.*;
class CRC
{
 public static void main(String args[]) throws IOException
 {
  BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
  System.out.println("Enter Generator:");
  String gen = br.readLine();
  System.out.println("Enter Data:");
  String data = br.readLine();
  String code = data;
  while(code.length() < (data.length() + gen.length() - 1))
   code = code + "0";
  code = data + div(code,gen);
  System.out.println("The transmitted Code Word is: " + code);
  System.out.println("Please enter the received Code Word: ");
  String rec = br.readLine();
  if(Integer.parseInt(div(rec,gen)) == 0)
   System.out.println("The received code word contains no errors.");
  else
   System.out.println("The received code word contains errors.");
 }

 static String div(String num1,String num2)
 {
  int pointer = num2.length();
  String result = num1.substring(0, pointer);
  String remainder = "";
  for(int i = 0; i < num2.length(); i++)
  {
   if(result.charAt(i) == num2.charAt(i))
    remainder += "0";
   else
    remainder += "1";
  }
  while(pointer < num1.length())
  {
   if(remainder.charAt(0) == '0')
   {
    remainder = remainder.substring(1, remainder.length());
    remainder = remainder + String.valueOf(num1.charAt(pointer));
```

```
    pointer++;
   }
   result = remainder;
   remainder = "";
   for(int i = 0; i < num2.length(); i++)
   {
    if(result.charAt(i) == num2.charAt(i))
     remainder += "0";
    else
     remainder += "1";
   }
  }
  return remainder.substring(1,remainder.length());
 }
}
```

Ex No. 6 – Cyclic Redundancy Code

OUTPUT



Result:

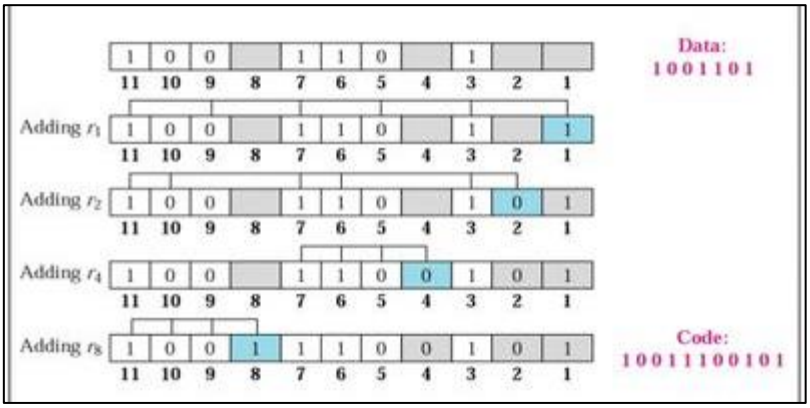
Thus, the CRC is implemented and successfully executed.

Ex No:7 **Implementation of Hamming code**

Aim: To simulate the Hamming code implementation

Hamming Code:

Hamming code is a liner code that is useful for error detection up to two immediate bit errors. It is capable of single-bit errors. In Hamming code, the source encodes the message by adding redundant bits in the message. These redundant bits are mostly inserted and generated at certain positions in the message to accomplish error detection and correction process.



Algorithm

1. Start the Program
2. Declare the necessary input variables for getting the data
3. Calculate the parity bits
4. Calculate the Hamming code
5. Introduce an error and check for error detection logic
6. Correct the error detected and display
7. End the program

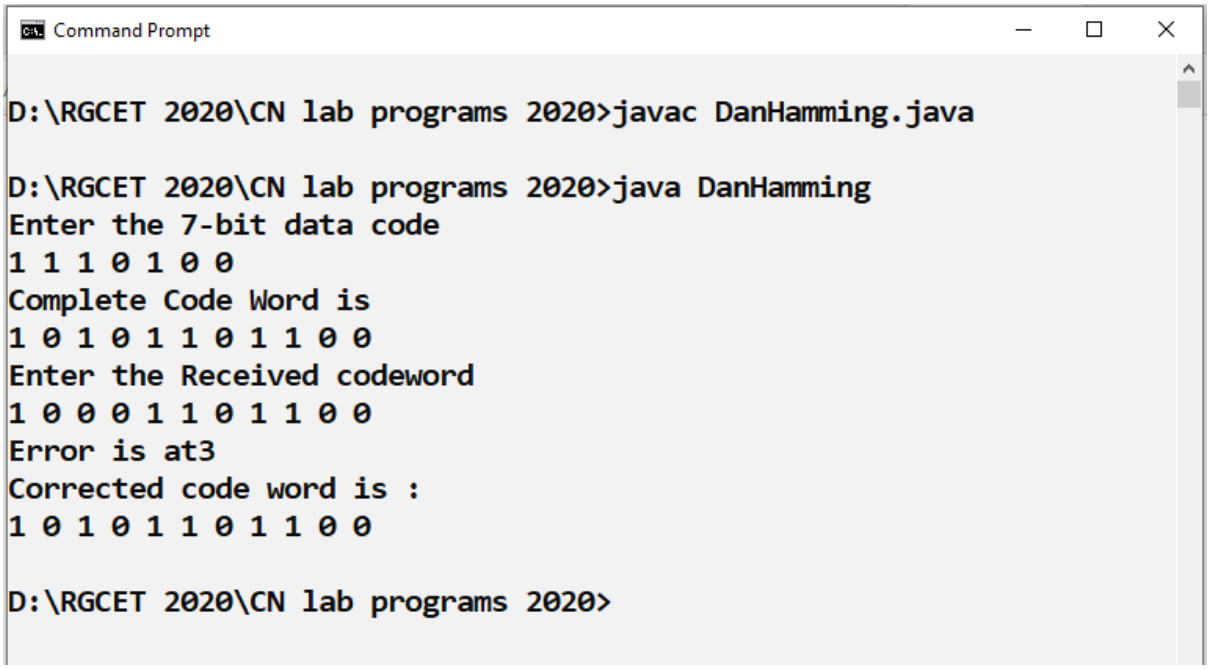
Ex No. 7 – Hamming Code

```
import java.util.*;
class DanHamming
{
```

```
public static void main(String arg[])
{
Scanner sc=new Scanner(System.in);
System.out.println("Enter the 7-bit data code");
int d[]=new int[7];
for(int i=0;i<7;i++)
{
d[i]=sc.nextInt();
}
int p[]=new int[4];
p[0]=d[0]^d[1]^d[3]^d[4]^d[6];
p[1]=d[0]^d[2]^d[3]^d[5]^d[6];
p[2]=d[1]^d[2]^d[3];
p[3]=d[4]^d[5]^d[6];

int c[]=new int[11];
System.out.println("Complete Code Word is ");

c[0]=p[0];
c[1]=p[1];
c[2]=d[0];
c[3]=p[2];
c[4]=d[1];
c[5]=d[2];
c[6]=d[3];
c[7]=p[3];
c[8]=d[4];
c[9]=d[5];
c[10]=d[6];

for(int i=0; i<11;i++)
{
System.out.print(c[i]+ " ");
}
System.out.println();
System.out.println("Enter the Received codeword");
int r[]=new int[11];
for(int i=0;i<11;i++)
{
r[i]=sc.nextInt();
}

int pr[]=new int[4];
int rd[]=new int[7];
```

```
pr[0]=r[0];
pr[1]=r[1];
rd[0]=r[2];
pr[2]=r[3];
rd[1]=r[4];
rd[2]=r[5];
rd[3]=r[6];
pr[3]=r[7];
rd[4]=r[8];
rd[5]=r[9];
rd[6]=r[10];
int s[]=new int[4];
s[0]=pr[0]^rd[0]^rd[1]^rd[3]^rd[4]^rd[6];
s[1]=pr[1]^rd[0]^rd[2]^rd[3]^rd[5]^rd[6];
s[2]=pr[2]^rd[1]^rd[2]^rd[3];
s[3]=pr[3]^rd[4]^rd[5]^rd[6];

int dec=(s[0]*1)+(s[1]*2)+(s[2]*4)+(s[3]*8);
if(dec==0)
System.out.println("No error");
else
{
System.out.println("Error is at"+dec);
if(r[dec-1]==0)
r[dec-1]=1;
else
r[dec-1]=0;
}
System.out.println("Corrected code word is : ");
for(int i=0;i<11;i++)
System.out.print(r[i]+" ");
System.out.println();
}
}
```

Ex No. 7 – Hamming Code



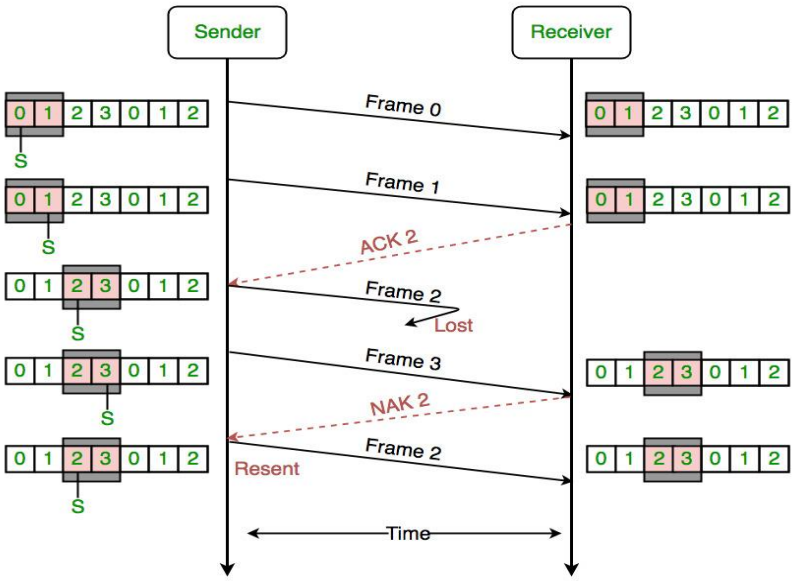
Result: Thus, the Hamming code error detection and correction is implemented and verified.

Ex No:8 **Implementation of Sliding Window Protocol**

Aim: To implement Sliding window protocol using TCP Sockets

Sliding Window Protocol

Sliding window protocols are data link layer protocols for reliable and sequential delivery of data frames. In this protocol, multiple frames can be sent by a sender at a time before receiving an acknowledgment from the receiver. The term sliding window refers to the imaginary boxes to hold frames. Sliding window method is also known as windowing.



Algorithm

BitServer

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Receive the data from client and send acknowledgement
7. Display if any errors in data are detected
8. End of the program

BitClient

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define Socket Objects with required details
4. Get connected with Server
5. Declare and Define the input and output stream objects
6. Sent message to Server and receive the acknowledgements
7. End.

Ex No. 8 – Sliding Window Protocol

SERVER

```
import java.lang.System;
import java.net.*;
import java.io.*;
class BitServer
{
public static void main(String args[])throws IOException
{
try
{
BufferedInputStream in;
ServerSocket  Serversocket=new ServerSocket(500);
System.out.println("waiting for connection:");
Socket client=Serversocket.accept();
System.out.println("reaceived request from sending frames:");
in=new BufferedInputStream(client.getInputStream());
DataOutputStream out=new DataOutputStream(client.getOutputStream());
int p=in.read();
```

```
System.out.println("sending");
for(int i=1;i<=p;i++)
{
System.out.println("sending frame no"+i);
out.write(i);
out.flush();
System.out.println("waiting for acknowledgement");
Thread.sleep(5000);
int a=in.read();
System.out.println("received acknowledgement for frame no:" +i);
System.out.print("as"+a);
}
out.flush();
in.close();
out.close();
System.out.println("quiting");
}
catch(IOException e)
{
System.out.println(e);
}
catch(InterruptedException e)
{
}
}
}
```

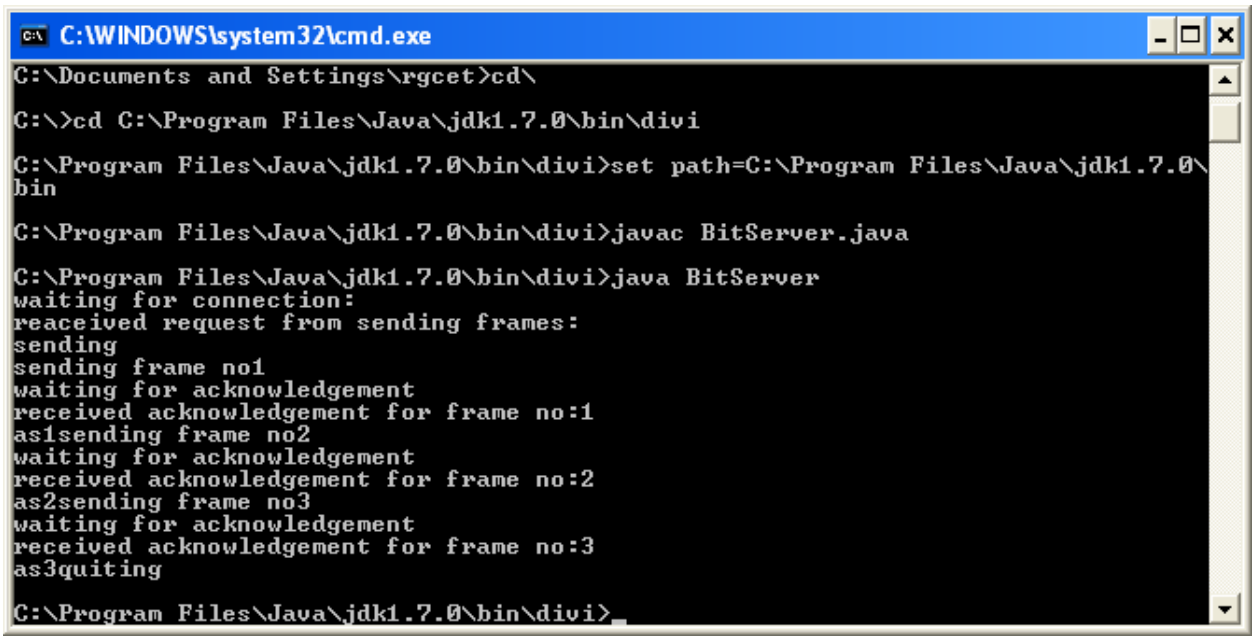
CLIENT

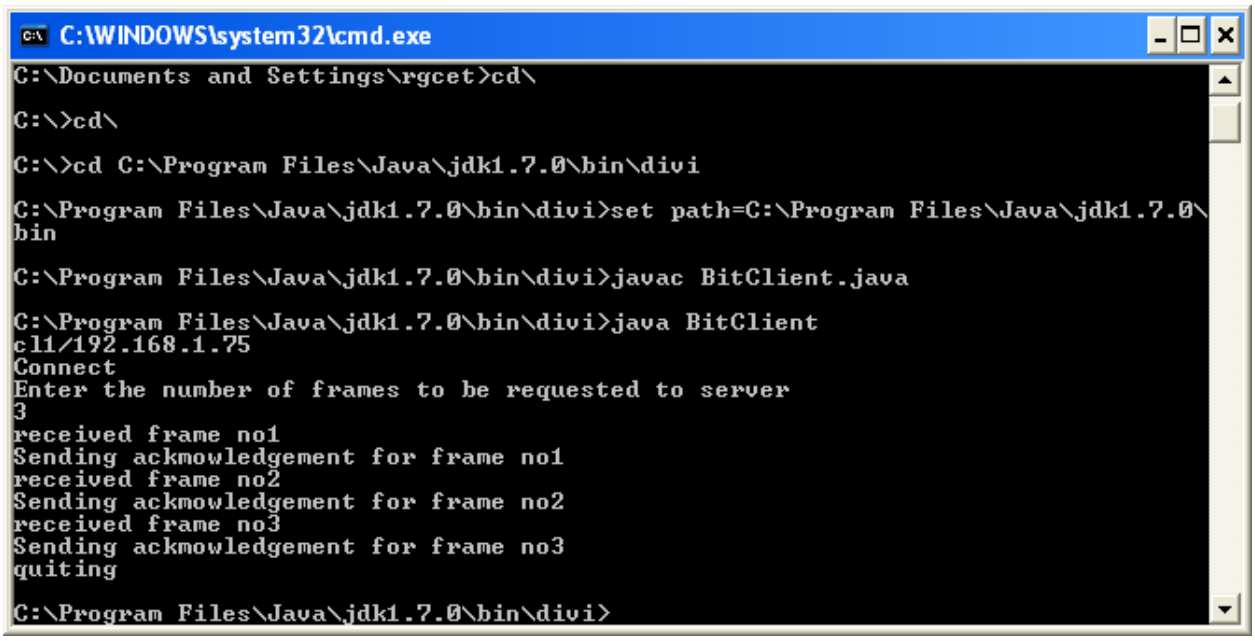
```
import java.lang.System;
import java.net.*;
import java.io.*;
import java.math.*;
class BitClient
{
public static void main(String args[])
{
try
{
InetAddress addr=InetAddress.getLocalHost();
System.out.println(addr);
Socket connection=new Socket(addr,500);
DataOutputStream out=new DataOutputStream(connection.getOutputStream());
BufferedInputStream in=new BufferedInputStream(connection.getInputStream());
BufferedReader ki=new BufferedReader(new InputStreamReader(System.in));
int flag=0;
System.out.println("Connect");
System.out.println("Enter the number of frames to be requested to server");
int c=Integer.parseInt(ki.readLine());
out.write(c);
out.flush();
int i,jj=0;
while(jj<c)
{
i=in.read();
System.out.println("received frame no"+i);
```

```
System.out.println("Sending ackmowledgement for frame no"+i);
out.write(i);
out.flush();
jj++;
}
out.flush();
in.close();
System.out.println("quiting");
}
catch(Exception e)
{
System.out.println(e);
}}}
```

Ex No. 8 – Sliding Window Protocol

OUTPUT





Result: Thus, the sliding window protocol between client and server is implemented and verified

Ex No:9 **Implementation of Remote Command execution**

Aim: To implement the remote command execution task from client to server

Remote command execution

Remote command execution is when a process on a host causes a program to be executed on another host. Usually, the invoking process wants to pass data to the remote program capture its output also.

Algorithm

RemoteServer

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define ServerSocket and Socket Objects with required details

4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
6. Receive the command from client
7. Get the runtime object of the runtime class using getruntime ().
8. Execute the command using the exec () method of runtime
9. End of the program

RemoteClient

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define Socket Objects with required details
4. Get connected with the server
5. Declare and Define the input and output stream objects
6. In the client side the remote command to be executed is given as input.
7. End of the program

Ex No. 9 – Remote Command Execution

RemoteServer

```
import java.io.*;
import java.net.*;
class RemoteServer
{
public static void main(String args[])
{
try
{
int Port;
BufferedReader Buf =new BufferedReader(new
InputStreamReader(System.in));
System.out.print(" Enter the Port Address : " );
Port=Integer.parseInt(Buf.readLine());
ServerSocket ss=new ServerSocket(Port);
System.out.println(" Server is Ready To Receive a Command. ");
System.out.println(" Waiting ..... ");
Socket s=ss.accept();
```

```
if(s.isConnected()==true)
        System.out.println(" Client Socket is Connected Succecfully. ");
InputStream in=s.getInputStream();
OutputStream ou=s.getOutputStream();
BufferedReader buf=new BufferedReader(new InputStreamReader(in));
String cmd=buf.readLine();
PrintWriter pr=new PrintWriter(ou);
pr.println(cmd);
Runtime H=Runtime.getRuntime();
Process P=H.exec(cmd);
System.out.println(" The " + cmd + " Command is Executed Successfully. ");
pr.flush();
pr.close();
ou.close();
in.close();
}
catch(Exception e)
{
System.out.println(" Error : " + e.getMessage());
}
}
}
```

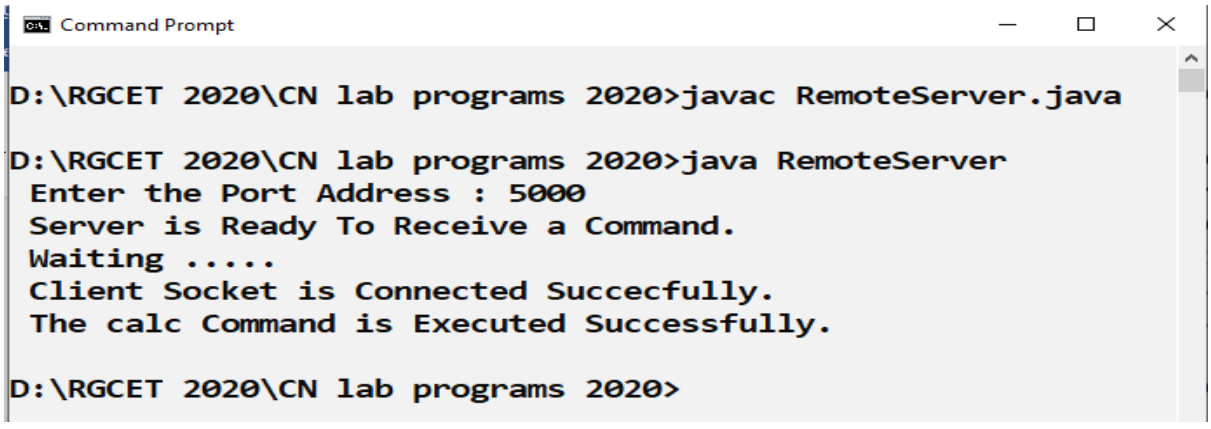
RemoteClient

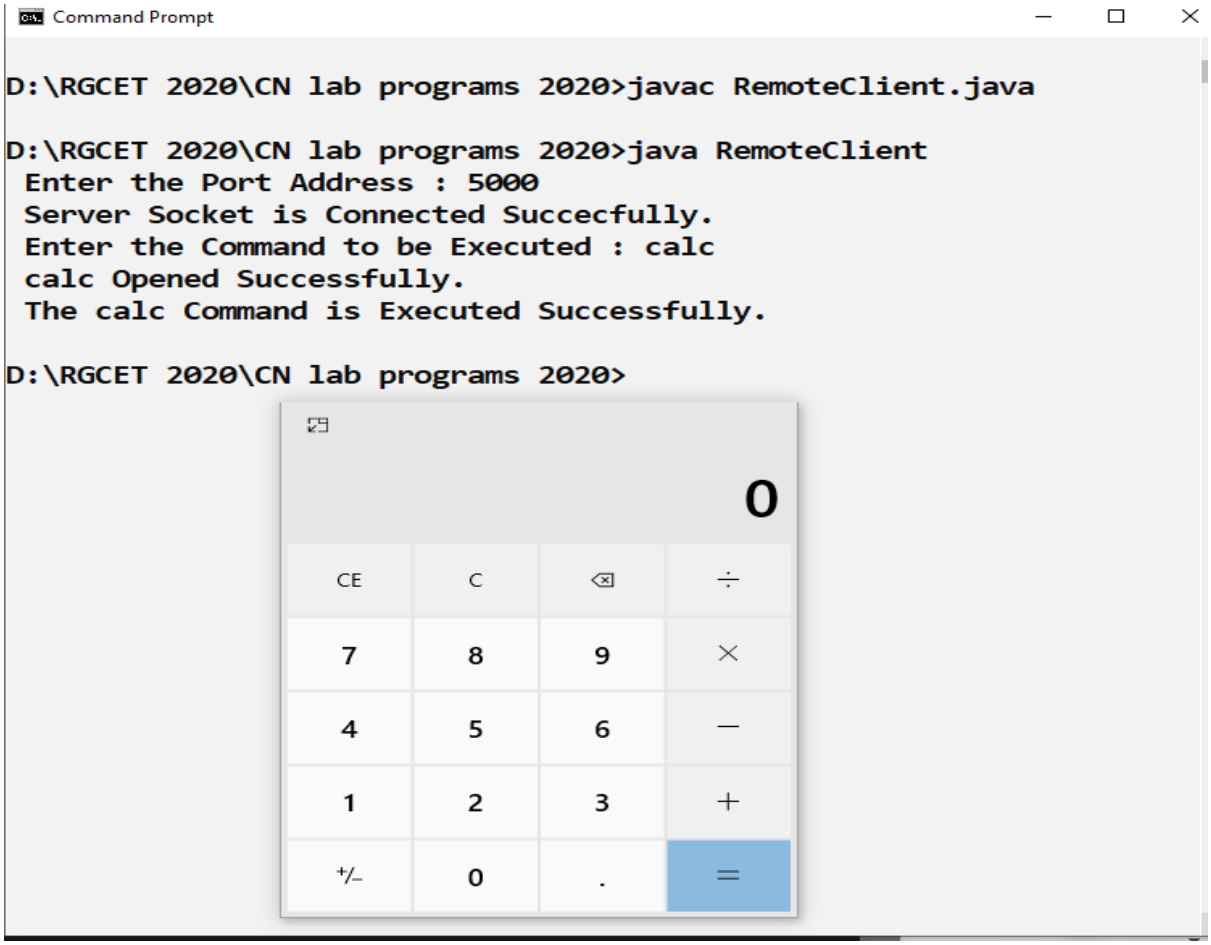
```
import java.io.*;
import java.net.*;

class RemoteClient
{
public static void main(String args[])
{
try
{
int Port;
BufferedReader Buf =new BufferedReader(new
InputStreamReader(System.in));
System.out.print(" Enter the Port Address : " );
Port=Integer.parseInt(Buf.readLine());
Socket s=new Socket("localhost",Port);
if(s.isConnected()==true)
        System.out.println(" Server Socket is Connected Succecfully. ");
InputStream in=s.getInputStream();
OutputStream ou=s.getOutputStream();
BufferedReader buf=new BufferedReader(new
```

```
InputStreamReader(System.in));
BufferedReader buf1=new BufferedReader(new
InputStreamReader(in));
PrintWriter pr=new PrintWriter(ou);
System.out.print(" Enter the Command to be Executed : " );
pr.println(buf.readLine());
pr.flush();
String str=buf1.readLine();
System.out.println(" " + str + " Opened Successfully. ");
System.out.println(" The " + str + " Command is Executed Successfully. ");
pr.close();
ou.close();
in.close();
}
catch(Exception e)
{
System.out.println(" Error : " + e.getMessage());
}
}
}
```

Ex No. 9 – Remote Command Execution

OUTPUT





Result: Thus, the Remote command execution task between client and server is implemented and verified

Ex No:10 **Implementation of Remote Screen Capture**

Aim : To implement Remote Screen Capture between Client and Server using TCP sockets

Algorithm

RemoteDesktopServer

1. Start of the program
2. Import the packages java.net, java.io, java.awt, java.image and javax.image.io.
3. Declare and define ServerSocket and Socket Objects with required details
4. Listen to the client and accept the client requests
5. Declare and Define the input and output stream objects
3. Inside the main method declare a new object robot, Robot class provides a useful method for capturing a screenshot.
4. The screen size as a Rectangle object to display the getScreenSize() as full image.

5. To capture the whole screen using createScreenCapture() method.
6. Assign the size and width of the image.
7. Here using image io.write the image Desktop_screen.png is displayed.
8. Then close() and flush() method close all the server, client and out.
9. Stop the program

RemoteDesktopClient

1. Start the Program
2. Import JAVA.NET and other necessary packages
3. Declare and define Socket Objects with required details
4. Get connected with the server
5. Declare and Define the input and output stream objects
6. The BufferedImage subclass describes an Image with an accessible buffer of image data.
7. The method setRGB() can be used to set the color of a pixel of an already existing image.
8. imageio package, to load images from an external image format into the internal BufferedImage format used by Java 2D
9. Here close the server and in object using close() method.
10. Stop the program

Ex No. 10 – Remote Screen Capture

RemoteDesktopServer

```
import java.net.*;
import java.awt.*;
import java.awt.image.*;
import java.io.*;
import javax.imageio.ImageIO;

public class RemoteDesktopServer
{
public static void main(String[] args) throws Exception
{
try
{
int x;
Robot robot = new Robot();
System.out.println("Server Ready....");
while(true)
{
ServerSocket ss = new ServerSocket(5000); Socket soc = ss.accept();
Rectangle size = new Rectangle(Toolkit.getDefaultToolkit().getScreenSize());

BufferedImage screen = robot.createScreenCapture(size);
int[] rgbData = new int[(int) (size.getWidth()*size.getHeight())];
screen.getRGB(0,0, (int) size.getWidth(), (int) size.getHeight(), rgbData, 0, (int) size.getWidth());
OutputStream baseOut = soc.getOutputStream();
ObjectOutputStream out = new ObjectOutputStream(baseOut);
ImageIO.write(screen, "png", new File("Desktop_screen.png"));
out.writeObject(size);
for (x = 0; x <rgbData.length; x++)
{
out.writeInt(rgbData[x]);
}
out.flush();
ss.close();
soc.close();
out.close();
baseOut.close();
}
```

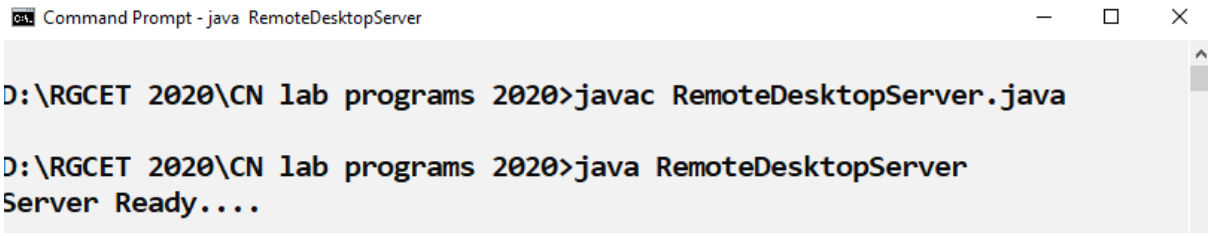
```
}
catch(IOException e)
{
System.out.println(e);
} } }
```

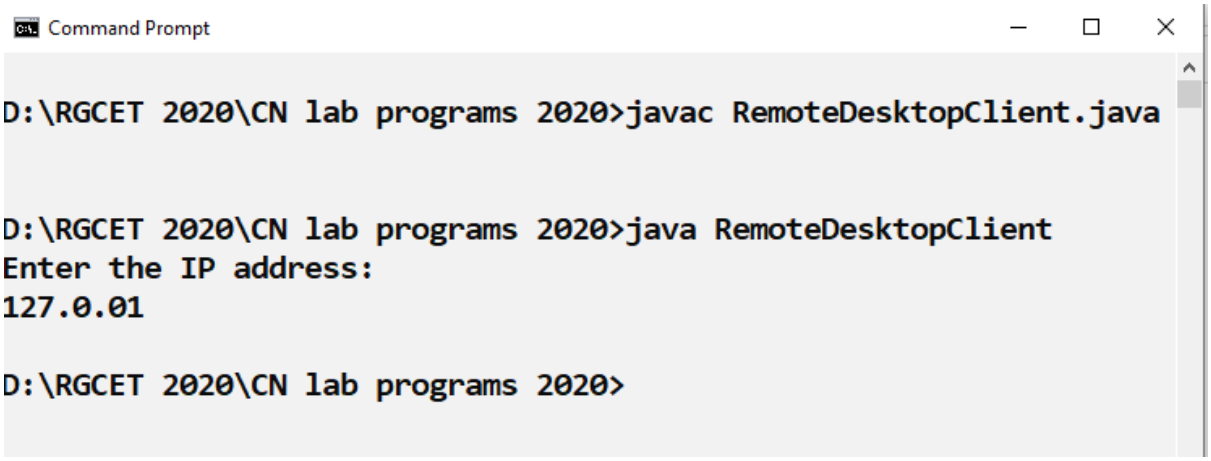
RemoteDesktopClient

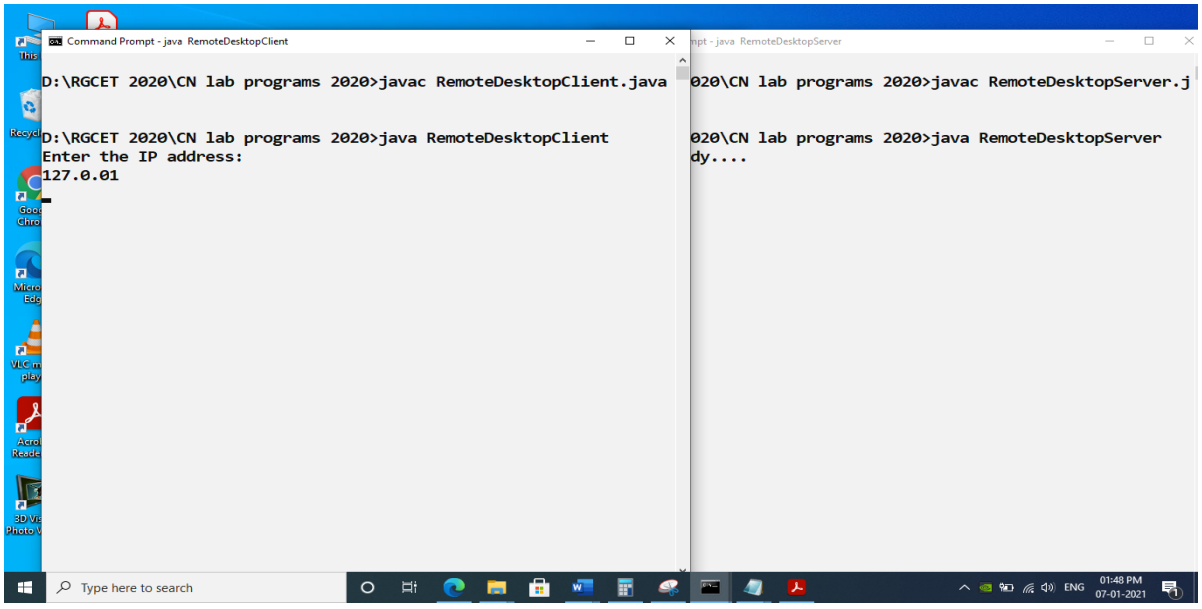
```
import java.net.*;
import java.awt.*;
import java.awt.image.*;
import java.io.*;
import java.util.*;
import javax.imageio.ImageIO;

public class RemoteDesktopClient
{
public static void main(String[] args) throws Exception
{
try
{
int x; String ip;
Scanner keyIn = new Scanner(new InputStreamReader(System.in));
System.out.println("Enter the IP address: ");
ip = keyIn.nextLine();
Socket soc = new Socket(ip, 5000);
ObjectInputStream in = new ObjectInputStream(soc.getInputStream());
Rectangle size = (Rectangle) in.readObject();
int[] rgbData = new int[(int)(size.getWidth() * size.getHeight())];
for (x = 0; x <rgbData.length;x++)
{
rgbData[x] = in.readInt();
}
BufferedImage screen = new BufferedImage((int) size.getWidth(), (int) size.getHeight(),
BufferedImage.TYPE_INT_ARGB);
screen.setRGB(0,0, (int) size.getWidth(), (int) size.getHeight(), rgbData,
0,(int)size.getWidth()); ImageIO.write(screen, "png", new File("Desktop_screen"));
}
catch(IOException e)
{
System.out.println(e);
}
}
}
```

Ex No. 10 – Remote Screen Capture

OUTPUT







Result: Thus, the Remote screen capture task between client and server is implemented and verified

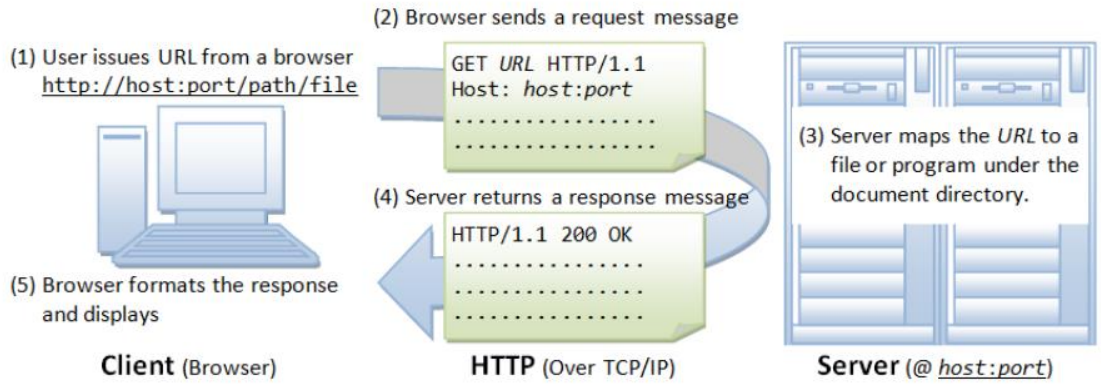
Ex No:11 **Implementation of Web Page Transfer using HTTP**

Aim : To implement Web Page transfer using HTTP

Hyper Text Transfer Protocol

The Hypertext Transfer Protocol (HTTP) is an application-level protocol for distributed, collaborative, hypermedia information systems. HTTP is a generic and stateless protocol which can be used for other purposes as well using extensions of its request methods, error codes, and headers.

Basically, HTTP is a TCP/IP based communication protocol, that is used to deliver data (HTML files, image files, query results, etc.) on the World Wide Web. The default port is TCP 80, but other ports can be used as well. It provides a standardized way for computers to communicate with each other. HTTP specification specifies how clients' request data will be constructed and sent to the server, and how the servers respond to these requests.



Algorithm

Download Page

1. Start the program
2. Create a URL object and pass URL as string and download the webpage
3. Input and OutputStreams are instantiated for downloading the image from the webpage
4. The image is downloaded and saved
5. Close the program

UploadServer

1. Start the program
2. Import the necessary packages for TCP Sockets
3. Instantiate the class ServerSocket and Socket with required details
4. Listen and wait for the client requests
5. Accept the client as requestins
6. Using the InputStream and DataInputStream get the input from the Client side.
7. Display the image size.
8. Use the read method of the Java ImageIO class, to open/read images in a variety of formats (GIF, JPG, and PNG).
9. Receiving the image , write in the JFrame and display it
10. Stop the program

UploadClient

1. Start the Program
2. Import the necessary packages for TCP sockets
3. Instantiate the class Socket with the port number 4000 for the local host.
4. Use BufferedImage to describe an Image with an accessible buffer of image data.
5. Create a ByteArrayOutputStream to sent appropriate data to server
6. Use ImageIO class that provides methods for more advanced usages of the Image I/O API that can access the image file to be sent to Server
7. Stop the program

Ex No. 11 – Web Page Transfer using HTTP

Download

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.URL;
public class Download
{
public static void main(String args[]) throws Exception
{
try
{
String fileName="new-year-greeting-1591885220.jpg";
String website="https://assets.winni.in/sfast/fables/new-year/greetings/"+fileName;
System.out.println("Downloading file from:"+website);
URL url=new URL(website);
InputStream inputStream=url.openStream();
OutputStream outputStream=new FileOutputStream(fileName);
byte[] buffer=new byte[2048];
int length=0;
while((length=inputStream.read(buffer))!=-1)
{
System.out.println("Buffer Read of Length:"+length);
outputStream.write(buffer,0,length);
}
inputStream.close();
outputStream.close();
}
catch(Exception e)
{
System.out.println("Exception:"+e.getMessage());
}
}
}
```

UploadServer

```
import java.net.*;
import java.io.*;
import java.awt.image.*;
import javax.imageio.*;
```

```
import javax.swing.*;
class UploadServer
{
public static void main(String args[]) throws Exception
{
ServerSocket server=null;
Socket socket;
server=new ServerSocket(4000);
System.out.println("Server Waiting for Image");
socket=server.accept();
System.out.println("Client Connected");
InputStream in=socket.getInputStream();
DataInputStream dis=new DataInputStream(in);
int len=dis.readInt();
System.out.println("Image Size:"+len/1024+"KB");
byte[] data=new byte[len];
dis.readFully(data);
dis.close();
in.close();
InputStream ian=new ByteArrayInputStream(data);
BufferedImage bImage=ImageIO.read(ian);
JFrame f=new JFrame("Server");
ImageIcon icon=new ImageIcon(bImage);
JLabel l=new JLabel();
l.setIcon(icon);
f.add(l);
f.pack();
f.setVisible(true);
}
}
```

UploadClient

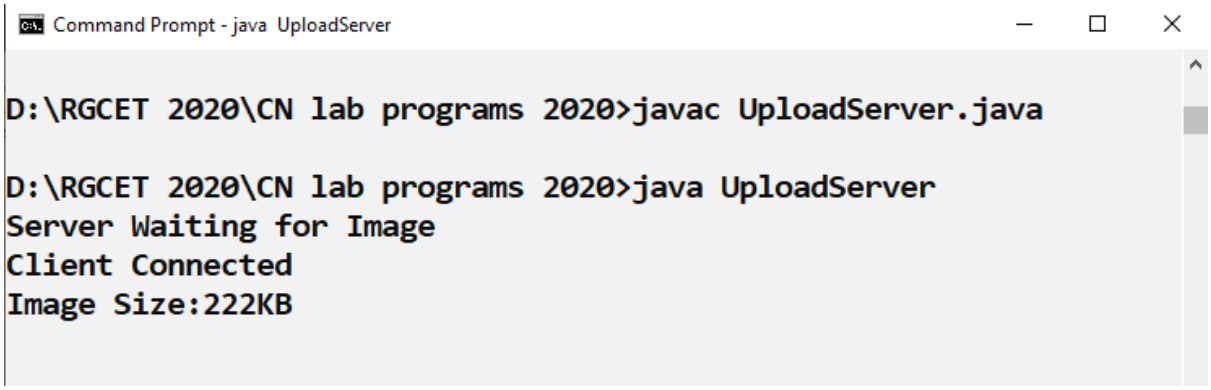
```
import javax.swing.*;
import java.net.*;
import java.awt.image.*;
import javax.imageio.*;
import java.io.*;
import java.awt.image.BufferedImage;
import java.io.ByteArrayOutputStream;
import java.io.File;
import java.io.IOException;
import javax.imageio.ImageIO;
public class UploadClient
```

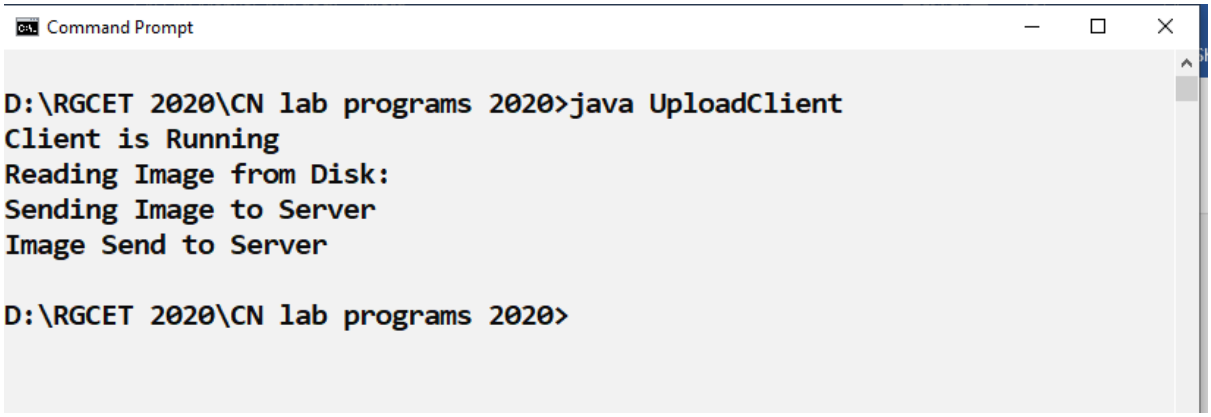
```
{
public static void main(String args[])throws Exception
{
Socket soc;
BufferedImage img=null;
soc=new Socket("localhost",4000);
System.out.println("Client is Running");
try
{
System.out.println("Reading Image from Disk:");
img=ImageIO.read(new File("new-year-greeting-1591885220.jpg"));
ByteArrayOutputStream baos=new ByteArrayOutputStream();
ImageIO.write(img,"jpg",baos);
baos.flush();
byte[] bytes=baos.toByteArray();
baos.close();
System.out.println("Sending Image to Server");
OutputStream out=soc.getOutputStream();
DataOutputStream dos=new DataOutputStream(out);
dos.writeInt(bytes.length);
dos.write(bytes,0,bytes.length);
System.out.println("Image Send to Server");
dos.close();
out.close();
}
catch(Exception e)
{
System.out.println("Exception:"+e.getMessage());
soc.close();
}
soc.close();
}
}
```

Ex No. 11 – Web Page Transfer using HTTP

OUTPUT

```
Command Prompt
D:\RGCET 2020\CN lab programs 2020>javac Download.java

D:\RGCET 2020\CN lab programs 2020>java Download
Downloading file from:https://dayfinders.com/national-day-of-hope/National-Day-of-Hope-300x200.png
Buffer Read of Length:32
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:1568
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:1856
Buffer Read of Length:32
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:1824
Buffer Read of Length:32
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:2048
Buffer Read of Length:1824
Buffer Read of Length:32
Buffer Read of Length:2048
```





Result : Thus, the Web Page transfer using HTTP is established and verified.

Ex No:12 **Implementation of ARP and RARP**

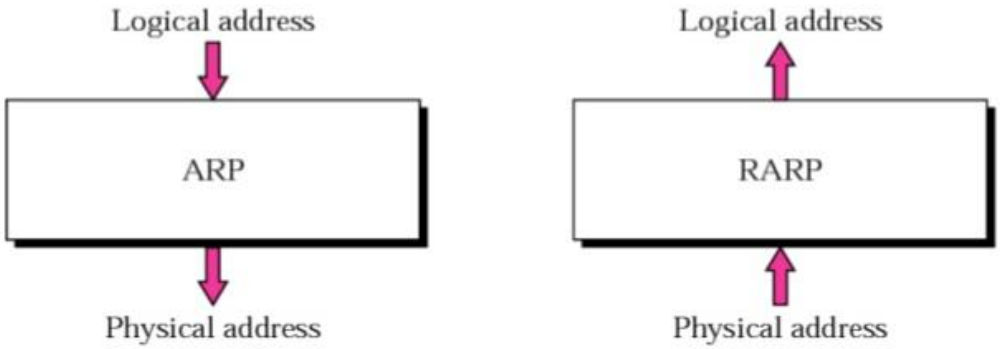
Aim : To implement the working of ARP and RARP

Address Resolution Protocol (ARP)

Address Resolution Protocol is a communication protocol used for discovering physical address associated with given network address. Typically, ARP is a network layer to data link layer mapping process, which is used to discover MAC address for given Internet Protocol Address.

Reverse Address Resolution Protocol (RARP)

Reverse ARP is a networking protocol used by a client machine in a local area network to request its Internet Protocol address (IPv4) from the gateway-router's ARP table. The network administrator creates a table in gateway-router, which is used to map the MAC address to corresponding IP address.



ALGORITHM

1. Start the program
2. Import necessary header files
3. Populate ARP table database through automatic command execution
4. ARP protocol is simulated by reading an IP address and returning the MAC address.
5. RARP protocol is simulated by reading an MAC address and returning the IP address
6. Display the values
7. Stop the program

Ex No. 12 – ARP and RARP

```
import java.io.*;
import java.util.*;
public class arp_rarp
{
private static final String Command = "arp -a";
public static void getARPTable(String cmd) throws Exception

{

File fp= new File("ARPTable.txt");
FileWriter fw=new FileWriter(fp);
BufferedWriter bw=new BufferedWriter(fw);
Process P=Runtime.getRuntime().exec(cmd);
Scanner S=new Scanner(P.getInputStream()).useDelimiter("\\A");
while(S.hasNext())
        bw.write(S.next());
bw.close();
fw.close();
}

public static void findMAC(String ip) throws Exception
{
File fp=new File("ARPTable.txt");
FileReader fr=new FileReader(fp);
BufferedReader br=new BufferedReader(fr);
String line;
while((line=br.readLine())!=null)
{
if(line.contains(ip))
{
System.out.println("Internet Address Physical Address       Type");
System.out.println(line);
break;
}
}

if(line == null)
System.out.println("Not found");
fr.close();
br.close();
}
```

```
public static void findIP(String mac)throws Exception
{
File fp=new File("ARPTable.txt");
FileReader fr=new FileReader(fp);
BufferedReader br=new BufferedReader(fr);
String line;
while((line=br.readLine())!=null)
{
if(line.contains(mac))
{
System.out.println("Internet Address  Physical Address       Type");
System.out.println(line);
break;
}
}

if(line==null)
System.out.println("not found");
fr.close();
br.close();
}

public static void main(String as[])throws Exception
{
getARPTable(Command);
Scanner S=new Scanner(System.in);
System.out.println("ARP Protocol");
System.out.println("Enter IP Address: ");
String IP=S.nextLine();
findMAC(IP);
System.out.println("RARP Protocol");
System.out.print("Enter MAC Address:");
String MAC=S.nextLine();
findIP(MAC);
}
}
```

Ex No. 12 – ARP and RARP

OUTPUT

Command Prompt

```
D:\RGCET 2020\CN lab programs 2020>javac arp_rarp.java

D:\RGCET 2020\CN lab programs 2020>java arp_rarp
ARP Protocol
Enter IP Address:
224.0.0.251
Internet Address        Physical Address        Type
  224.0.0.251           01-00-5e-00-00-fb     static
RARP Protocol
Enter MAC Address:ff-ff-ff-ff-ff-ff
Internet Address  Physical Address      Type
  192.168.1.255         ff-ff-ff-ff-ff-ff     static

D:\RGCET 2020\CN lab programs 2020>java arp_rarp
ARP Protocol
Enter IP Address:
178.16.21.14
Not found
RARP Protocol
Enter MAC Address:02-10-ff-5f-fb-00
not found
```

ARPTable.txt - Notepad

File Edit Format View Help

```
Interface: 192.168.1.87 --- 0xc
  Internet Address      Physical Address      Type
  192.168.1.1           6c-59-76-0a-ba-81     dynamic
  192.168.1.255         ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

Ln 11, Col 1 | 100% | Windows (CRLF) | UTF-8

Result : Thus, the working of ARP and RARP are simulated

Ex No:13 **Implementation of Shortest Path Routing**

Aim: To implement Shortest path routing using Dijkstra's Algorithm

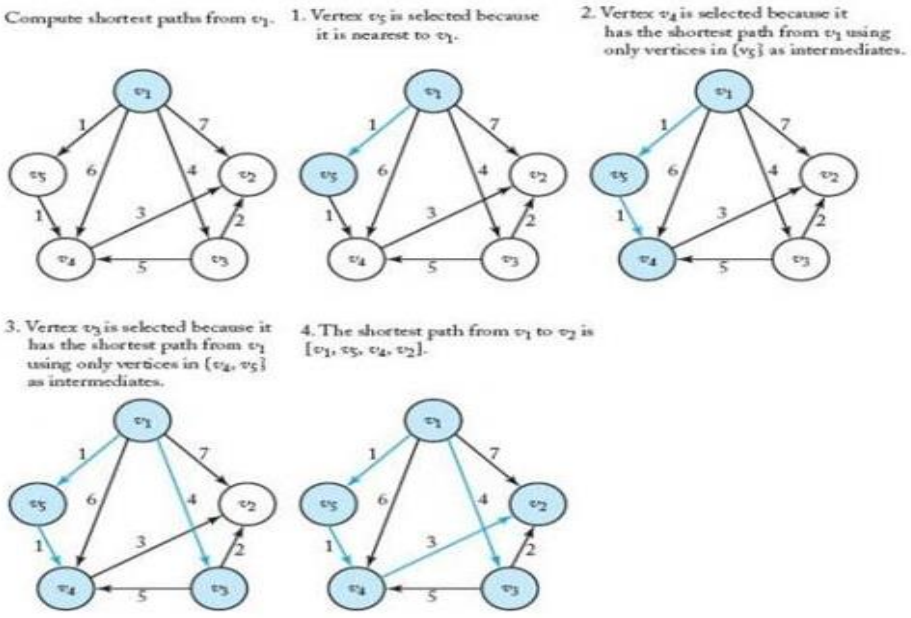
Shortest path routing

Routing decision in networks, are mostly taken on the basis of cost between source and destination. Hop count plays major role here. Shortest path is a technique which uses various algorithms to decide a path with minimum number of hops. Common shortest path algorithms are:

- Dijkstra's algorithm [Link State Routing – OSPF]
- Bellman Ford algorithm [Distance Vector Routing – RIP]
- Floyd Warshall algorithm

Algorithm Steps:

1. Start the program
2. Initialize required input / output variables and Initialize distances according to the **algorithm**.
3. Pick first node and calculate distances to adjacent nodes.
4. Pick next node with minimal distance; repeat adjacent node distance calculations.
5. Final result of shortest-path tree.
6. End the program



Ex No. 13 – Shortest path routing

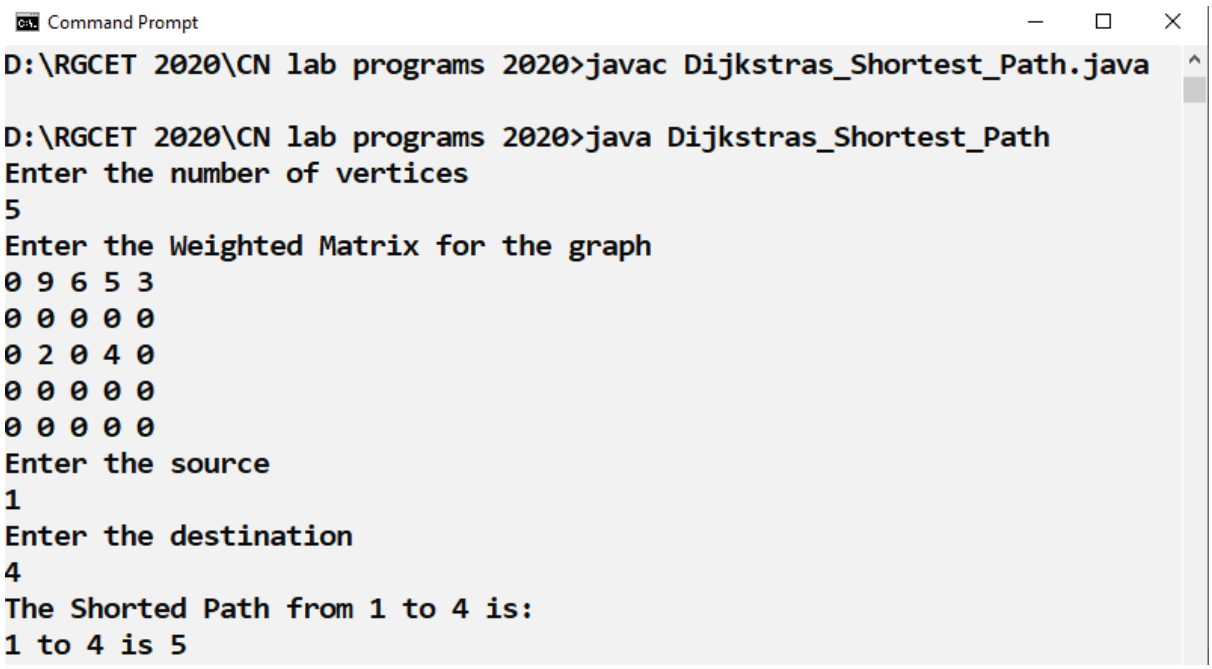
```
import java.util.HashSet;
import java.util.InputMismatchException;
import java.util.Iterator;
import java.util.Scanner;
import java.util.Set;

public class Dijkstras_Shortest_Path
{
   private int         distances[];
   private Set<Integer> settled;
   private Set<Integer> unsettled;
   private int         number_of_nodes;
   private int         adjacencyMatrix[][];

   public Dijkstras_Shortest_Path(int number_of_nodes)
   {
      this.number_of_nodes = number_of_nodes;
      distances = new int[number_of_nodes + 1];
      settled = new HashSet<Integer>();
      unsettled = new HashSet<Integer>();
      adjacencyMatrix = new int[number_of_nodes + 1][number_of_nodes + 1];
   }

   public void dijkstra_algorithm(int adjacency_matrix[][], int source)
   {
      int evaluationNode;
      for (int i = 1; i <= number_of_nodes; i++)
         for (int j = 1; j <= number_of_nodes; j++)
            adjacencyMatrix[i][j] = adjacency_matrix[i][j];

      for (int i = 1; i <= number_of_nodes; i++)
      {
         distances[i] = Integer.MAX_VALUE;
      }

      unsettled.add(source);
      distances[source] = 0;
      while (!unsettled.isEmpty())
      {
         evaluationNode = getNodeWithMinimumDistanceFromUnsettled();
```

```
            unsettled.remove(evaluationNode);
            settled.add(evaluationNode);
            evaluateNeighbours(evaluationNode);
        }
    }

    private int getNodeWithMinimumDistanceFromUnsettled()
    {
        int min;
        int node = 0;

        Iterator<Integer> iterator = unsettled.iterator();
        node = iterator.next();
        min = distances[node];
        for (int i = 1; i <= distances.length; i++)
        {
            if (unsettled.contains(i))
            {
                if (distances[i] <= min)
                {
                    min = distances[i];
                    node = i;
                }
            }
        }
        return node;
    }

    private void evaluateNeighbours(int evaluationNode)
    {
        int edgeDistance = -1;
        int newDistance = -1;

        for (int destinationNode = 1; destinationNode <= number_of_nodes;
destinationNode++)
        {
            if (!settled.contains(destinationNode))
            {
                if (adjacencyMatrix[evaluationNode][destinationNode] != Integer.MAX_VALUE)
                {
                    edgeDistance = adjacencyMatrix[evaluationNode][destinationNode];
                    newDistance = distances[evaluationNode] + edgeDistance;
                    if (newDistance < distances[destinationNode])
                    {
```

```
                distances[destinationNode] = newDistance;
            }
            unsettled.add(destinationNode);
        }
      }
   }
}

public static void main(String[] arg)
{
   int adjacency_matrix[][];
   int number_of_vertices;
   int source = 0, destination = 0;
   Scanner scan = new Scanner(System.in);
   try
   {
      System.out.println("Enter the number of vertices");
      number_of_vertices = scan.nextInt();
      adjacency_matrix = new int[number_of_vertices + 1][number_of_vertices + 1];

      System.out.println("Enter the Weighted Matrix for the graph");
      for (int i = 1; i <= number_of_vertices; i++)
      {
         for (int j = 1; j <= number_of_vertices; j++)
         {
            adjacency_matrix[i][j] = scan.nextInt();
            if (i == j)
            {
               adjacency_matrix[i][j] = 0;
               continue;
            }
            if (adjacency_matrix[i][j] == 0)
            {
               adjacency_matrix[i][j] = Integer.MAX_VALUE;
            }
         }
      }

      System.out.println("Enter the source ");
      source = scan.nextInt();

      System.out.println("Enter the destination ");
      destination = scan.nextInt();
```

```
            Dijkstras_Shortest_Path dijkstrasAlgorithm = new Dijkstras_Shortest_Path(
                  number_of_vertices);
            dijkstrasAlgorithm.dijkstra_algorithm(adjacency_matrix, source);

            System.out.println("The Shorted Path from " + source + " to " + destination + " is: ");
            for (int i = 1; i <= dijkstrasAlgorithm.distances.length - 1; i++)
            {
               if (i == destination)
                  System.out.println(source + " to " + i + " is "
                        + dijkstrasAlgorithm.distances[i]);
            }
         } catch (InputMismatchException inputMismatch)
         {
            System.out.println("Wrong Input Format");
         }
         scan.close();
      }
   }
```

Ex No. 13 – Shortest path routing



Result : Thus the Shortest path routing is implemented using Dijkstra's algorithm

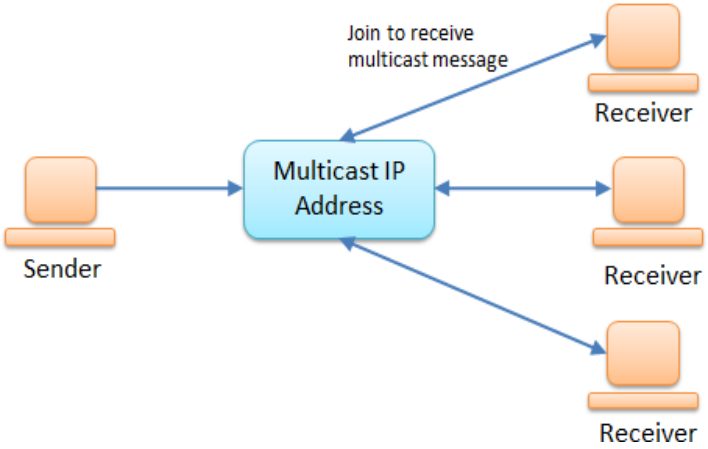
Ex No:14 **Implementation of Multicast Routing**

Aim: To implement Multicasting via Client-Server communication

Multicasting

Multicasting in computer network is a group communication, where a sender(s) send data to multiple receivers simultaneously. It supports one – to – many and many – to – many data transmission across LANs or WANs. Through the process of multicasting, the communication and processing overhead of sending the same data packet or data frame in minimized.

Multicast Addresses: The reserved class D address range: 224.0.0.0 - 239.255.255.255



Multicasting in Java

Multicasting is a type of Datagram Socket. Unlike regular Datagrams, Multicasting doesn't handle each client individually instead it sends it out to one IP Address and all subscribed clients will get the message.

The **MulticastSocket** class defined in the java.net package represents a multicast socket. Once a MulticastSocket object is created, the **joinGroup()** method is invoked to make it one of the members to receive a multicast message.

Run the Client First: The Client must subscribe to the IP before it can start receiving any packets

Algorithm

MulticastSocketServer

1. Start the program
2. Import necessary header files
3. Define and declare DatagramSockets and Datagram packets to simulate Multicast Socket and Messages
4. Register the DatagramSocket in defined IP address and Ports
5. Define required messages that should be multicasted to the clients which are joining in the given IP address and port
6. End the Program

MulticastSocketClient

1. Start the program
2. Import necessary header files
3. Define and declare MulticastSockets to simulate Multicast Clients
4. Use the objects of DatagramPackets to receive the Messages from the Server
5. Use joinGroup() method to get connected with Multicast Server
6. Receive the messages from the server
7. End the Program

Ex No. 14 – Multicast Routing

MulticastSocketServer

```
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.net.UnknownHostException;

public class MulticastSocketServer {

   final static String INET_ADDR = "224.0.0.3";
   final static int PORT = 8888;

   public static void main(String[] args) throws UnknownHostException, InterruptedException
{
      InetAddress addr = InetAddress.getByName(INET_ADDR);
      try (DatagramSocket serverSocket = new DatagramSocket())
{
      for (int i = 0; i < 5; i++)
{
      String msg = "Sent message no " + i;
      DatagramPacket msgPacket = new DatagramPacket(msg.getBytes(),
msg.getBytes().length, addr, PORT);
       serverSocket.send(msgPacket);
       System.out.println("Server sent packet with msg: " + msg);
       Thread.sleep(500);
  }
  } catch (IOException ex)
         {
         ex.printStackTrace();
         }
   }
}
```

MulticastSocketClient [Create more copies as new clients // Change Socket 1 to Socket2]

```
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.InetAddress;
import java.net.MulticastSocket;
import java.net.UnknownHostException;

public class MulticastSocketClient {
   final static String INET_ADDR = "224.0.0.3";
   final static int PORT = 8888;

   public static void main(String[] args) throws UnknownHostException {

         InetAddress address = InetAddress.getByName(INET_ADDR);
         byte[] buf = new byte[256];
            try (MulticastSocket clientSocket = new MulticastSocket(PORT)){
                  clientSocket.joinGroup(address);
             while (true) {

            DatagramPacket msgPacket = new DatagramPacket(buf, buf.length);
            clientSocket.receive(msgPacket);

            String msg = new String(buf, 0, buf.length);
            System.out.println("Socket 1 received msg: " + msg);
         }
      } catch (IOException ex) {
       ex.printStackTrace();
      }
   }
}
```

Ex No. 14 – Multicast Routing

Command Prompt

```
D:\RGCET 2020\CN lab programs 2020>javac MulticastSocketServer.java

D:\RGCET 2020\CN lab programs 2020>java MulticastSocketServer
Server sent packet with msg: Sent message no 0
Server sent packet with msg: Sent message no 1
Server sent packet with msg: Sent message no 2
Server sent packet with msg: Sent message no 3
Server sent packet with msg: Sent message no 4

D:\RGCET 2020\CN lab programs 2020>java MulticastSocketServer
Server sent packet with msg: Sent message no 0
Server sent packet with msg: Sent message no 1
Server sent packet with msg: Sent message no 2
Server sent packet with msg: Sent message no 3
Server sent packet with msg: Sent message no 4

D:\RGCET 2020\CN lab programs 2020>
```

Command Prompt - java MulticastSocketClient

```
D:\RGCET 2020\CN lab programs 2020>javac MulticastSocketClient.java
Note: MulticastSocketClient.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

D:\RGCET 2020\CN lab programs 2020>java MulticastSocketClient
Socket 1 received msg: Sent message no 0

Socket 1 received msg: Sent message no 1

Socket 1 received msg: Sent message no 2

Socket 1 received msg: Sent message no 3

Socket 1 received msg: Sent message no 4
```

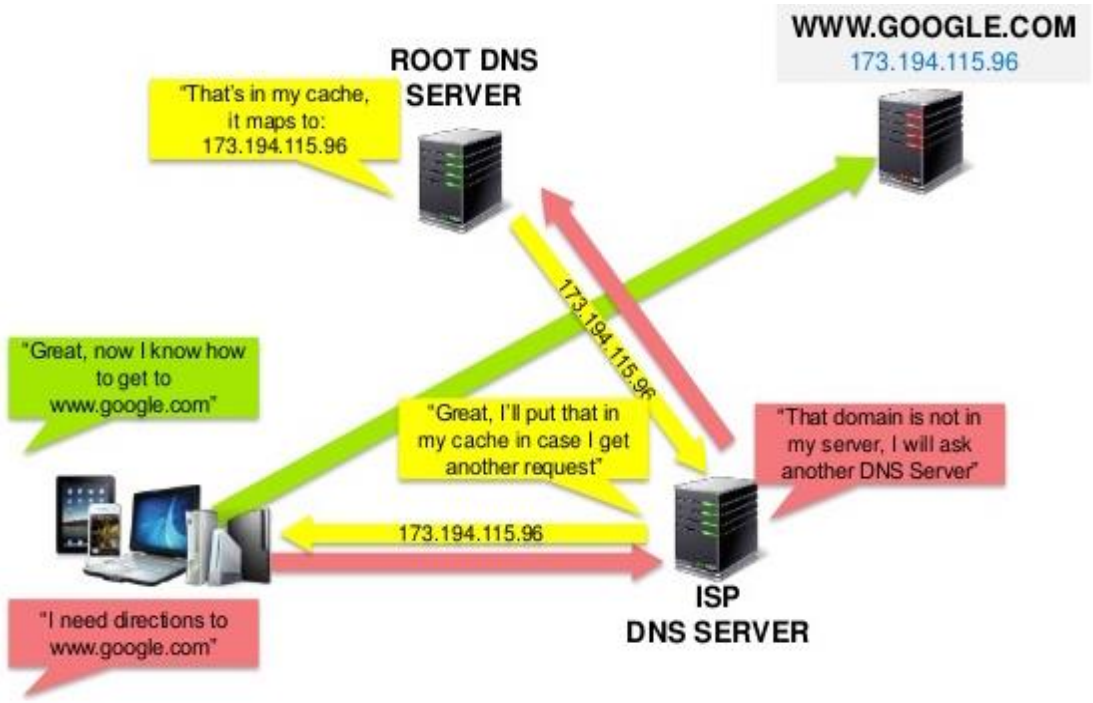
Result: Thus, Multicast routing is implemented and verified

Ex No:15 **Implementation of Domain Name Service**

Aim: To implement Domain Name service using Sockets

Domain Name Service

DNS (Domain Name Services) is one of the most important services for users on a network. DNS is a host name to IP address translation service. DNS is a distributed database implemented in a hierarchy of name servers. It is an application layer protocol for message exchange between clients and servers.



Algorithm

ServerDNS

1. Start the program.
2. Create the socket for the server.
3. Bind the socket to the port.
4. Listen for the incoming client connection.
5. Receive the IP address from the client to be resolved.
6. Get the domain name for the client.
7. Check the existence of the domain in the server.

8. If domain matches then send the corresponding address to the client.
9. Stop the program execution

ClientDNS

1. Start the Program.
2. Create the socket for the client.
3. Connect the socket to the Server.
4. Send the host name to the server to be resolved.
5. If the server corresponds then print the address and terminate the process

Ex No. 15 – Domain Name Service

Serverdns

```
import java.io.*;
import java.net.*;
import java.util.*;

class Serverdns
{
	public static void main(String args[])
	{
	try
	{
	DatagramSocket server=new DatagramSocket(1309);
	System.out.println("DNS Server waiting");
		while(true)
		{
			byte[] sendbyte=new byte[1024];
			byte[] receivebyte=new byte[1024];
				DatagramPacket receiver=new
	DatagramPacket(receivebyte,receivebyte.length);
			server.receive(receiver);
			String str=new String(receiver.getData());
			String s=str.trim();
			InetAddress addr=receiver.getAddress();
			int port=receiver.getPort();
			String ip[]={"165.165.80.80","165.165.79.1"};
			String name[]={"www.rgcetpdy.ac.in","www.google.com"};
			for(int i=0;i<ip.length;i++)
			{
				if(s.equals(ip[i]))
				{
					sendbyte=name[i].getBytes();
   DatagramPacket sender=new DatagramPacket(sendbyte,sendbyte.length,addr,port);
					server.send(sender);
System.out.println("Website name sent");
					break;
				}
				else if(s.equals(name[i]))
				{
					sendbyte=ip[i].getBytes();
```

```
        DatagramPacket sender=new DatagramPacket(sendbyte,sendbyte.length,addr,port);
System.out.println("IP address sent");
                                        server.send(sender);
                                        break;
                                }
                        }
                        break;

                }
        }
        catch(Exception e)
        {
                System.out.println(e);
        }
        }
}
```

Clientdns

```
import java.io.*;
import java.net.*;
import java.util.*;
class Clientdns
{
        public static void main(String args[])
        {
        try
        {
                DatagramSocket client=new DatagramSocket();
                InetAddress addr=InetAddress.getByName("127.0.0.1");

                byte[] sendbyte=new byte[1024];
                byte[] receivebyte=new byte[1024];
                BufferedReader in=new BufferedReader(new InputStreamReader(System.in));
                System.out.println("Enter the DOMAIN NAME or IP adress:");
                String str=in.readLine();
                sendbyte=str.getBytes();
                DatagramPacket sender=new
DatagramPacket(sendbyte,sendbyte.length,addr,1309);
                client.send(sender);
                DatagramPacket receiver=new
DatagramPacket(receivebyte,receivebyte.length);
                client.receive(receiver);
                String s=new String(receiver.getData());
```

```
                System.out.println("IP address or DOMAIN NAME: "+s.trim());
                client.close();
        }
        catch(Exception e)
        {
                System.out.println(e);
        }
        }
}
```

Ex No. 15 – Domain Name Service

Result: Thus, the DNS protocol is implemented using Sockets

Ex No:16 Throughput comparison between 802.3 and 802.11

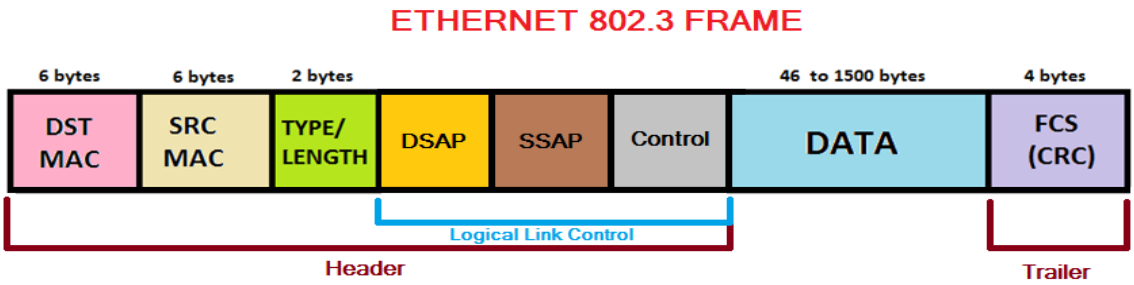
Aim : To study the throughput comparison between IEEE 802.3 and 802.11

IEEE 802 standards

- IEEE 802 specifies to a group of IEEE standards. IEEE standards 802 are used for controlling the Local Area Network and Metropolitan Area Network.
- The user layer in IEEE 802 is serviced by the two layers- the data link layer and the physical layer.
- The IEEE 802.3 and 802.11 standards are used to define CSMA/CD (Carrier Sense Multiple Access with Collision Detection) based on the Ethernet LAN (Local Area Networks) and Wireless LAN (WLAN) protocol, respectively.
- CSMA/CD Ethernet-based protocol is the most widely used for LAN and wireless standard seems to be used for mobile LAN applications provides more flexible/mobile and less expensive than wired LANs, but generally has a lower transmission rate than Ethernet LANs.
- On the other hand, the access technique used by the IEEE 802.11 Wireless LAN protocol is CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance) since the use of CSMA/CD implementation is not possible due to the waste of the energy on mobiles.

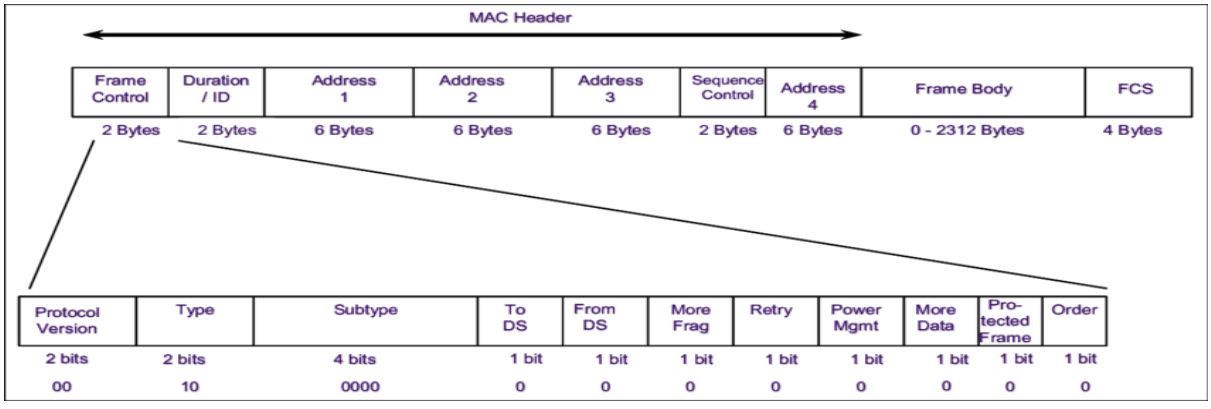
IEEE 802.3

- IEEE 802.3 was developed in 1980, by a consortium of Digital Equipment Corporation, Intel Corporation and Xerox Corporation (DIX), which have invented the native Ethernet specifications intended for 10Mbps bus-based LAN using coaxial cable and then submitted to the IEEE in 1982.
- The IEEE 802.3 document for carrier sense multiple access with multiple collision (CSMA/CD) is for use in commercial and light industrial environments. Now, when people talk about Ethernet, it typically refers to IEEE 802.3 CSMA/CD.



IEEE 802.11

- The increase use of Internet emerge the new concepts and equipment needed to provide mobile communication services within localized area. Such LAN technology provides more flexible and mobile for the users, called wireless LAN (WLAN) standardized by IEEE 802.11 standard in 1997.
- Basically, WLAN is an alternative technology to solve the problem experienced on wired LAN, such as the high installation and maintenance costs. Unlike wired LAN terminals, which are static when working on the network, one of the advantages is freedom mobility.
- Generally, wireless LAN is a LAN, which employs the infrared or microwave radio rather than coaxial cable, twisted pair or fiber optic cable as used in IEEE 802.3 standard as the transmission medium. WLAN provides the advantage of mobility for the users.



Advantages of IEEE 802.11

- The main advantage of Wireless LAN network over the CDMA/CD Ethernet based LAN network is the freedom mobility and flexibility.

- Another advantage is WLAN can be applied when the installation and maintenance cost of wired LAN infrastructure rises significantly, to provide the need of LAN extension to other building.
- However, there are five issues that decrease the attractiveness of wireless LANs, such as the transmission medium degradation due to multipath fading which in turn leads to hidden terminal problem, the security of the transmission medium, which is open to anyone within the geographical range of transmitter, and leads to the need of the encryption, the lower transmission rate of around 1 to 10Mbps and the need of the licensed when applying wireless over radio technology

Trivial Comparison	
Ethernet LAN	**IEEE 802.11 WLAN**
IEEE standard 802.3	IEEE standard 802.11
Communication medium is co-axial cable	Infrared or radio frequencies act as medium
Spread spectrum is not used	Spread spectrum is used
It uses MAC	It uses two MAC sub-layers
It uses CSMA/CD	It uses CSMA/CA
The efficiency is high	The efficiency is low
Addressing is simpler	Addressing is complicated
It has a large range	It has a short range

Throughput Comparison

A continually growing number of businesses and technology consumers exchange increasing amounts of information has result in evolving very rapidly local area networks (LANs) as the communication infrastructure that meets their fundamental needs and demands. As a consequence, to design, develop, and deploy a LAN, it is often necessary to perform an accurate measurement campaign (including interconnection topologies, transmission media, and access

protocols) so as to ensure that the LAN with its parameter settings provides the needed throughput for the intended users. Among these examinations, network access protocols play vital role in the adoption of the network. To this end, the design of an efficient and reliable Medium Access Control (MAC) protocol requires careful understanding and tuning of its various key parameters to achieve the optimum performance in any actual environment.

The two commonly MAC protocols used in LANs are the IEEE 802.3 and the IEEE 802.11 MAC protocols that are employed by Ethernet and Wireless LANs respectively. Ethernet occupies the Carrier Sense Multiple Access with Collision Detection (CSMA/CD), while wireless LAN devotes Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA). These CSMA protocols have randomness property that may direct the transmission to a possible collision which in turn requires that the stations (users) involved in the collision to abort their unsuccessful transmissions to be re-transmitted again according to a specified contention algorithm (e.g., the Truncated Binary Exponential Backoff – BEB algorithm). As a result of this randomness feature is that MAC protocol may suffer from harmful performance degradation.

The analysis is considered under the two extremely heavy load conditions – the saturation and disaster scenarios. Under saturation scenario, the objective is to examine the fundamental limit of the protocol; while under disaster scenario, the contribution is on examining the response of the protocol to a certain failure recovery.

The results present the relative estimate (or different levels of relation) between Ethernet and wireless LAN system throughputs. Results demonstrate that to achieve an acceptable performance level of the IEEE 802.3 protocol, the number of active stations in a network may be limited. On the other hand, the results present that the IEEE 802.11 MAC protocol performs reasonably well under a certain load condition with a good selection of the protocol parameters (e.g., initial backoff window size and maximum number of backoff stages). Moreover, the performance achieved by IEEE 802.11 MAC protocol which use four-way handshaking access scheme is, even, better than those achieved by the use of basic access method.

Result : Thus the Throughput of IEEE 802.3 and 802.11 is studied

Ex No:17

Key Distribution and Certification

Aim: To learn about Key Distribution and Certification

A drawback of symmetric key cryptography was the need for the two communicating parties to have agreed upon their secret key ahead of time. With public key cryptography, this *a priori* agreement on a secret value is not needed. However, public key encryption has its own difficulties, in particular the problem of obtaining someone's true public key. Both of these problems - determining a shared key for symmetric key cryptography, and securely obtaining the public key for public key cryptography - can be solved using a **trusted intermediary.** For symmetric key cryptograghy , the trusted intermediary is called a **Key Distribution Center (KDC**), which is a single, trusted network entity with whom one has established a shared secret key.

For public key cryptography, the trusted intermediary is called a **Certification Authority (CA)**. A certification authority certifies that a public key belongs to a particular entity (a person or a network entity). For a certified public key, if one can safely trust the CA that the certified the key, then one can be sure about to whom the public key belongs. Once a public key is certified, then it can be distributed from just about anywhere, including a public key server, a personal Web page or a diskette.

The Key Distribution Centre

Suppose once again that Bob and Alice want to communicate using symmetric key cryptography. They have never met (perhaps they just met in an on-line chat room) and thus have not established a shared secret key in advance. How can they now agree on a secret key, given that they can only communicate with each other over the network? A solution often adopted in practice is to use a trusted Key Distribution Center (KDC).

The KDC is a server that shares a different secret symmetric key with each registered user. This key might be manually installed at the server when a user first registers. The KDC knows the secret key of each user and each user can communicate securely with the KDC using this key.

Let's see how knowledge of this one key allows a user to securely obtain a key for communicating with any other registered user. Suppose that Alice and Bob are users of the KDC; they only know their individual key, $K_{A\text{-}KDC}$ and $K_{B\text{-}KDC}$, respectively, for communicating securely with the KDC. Alice takes the first step, and they proceed as illustrated in Figure 1.

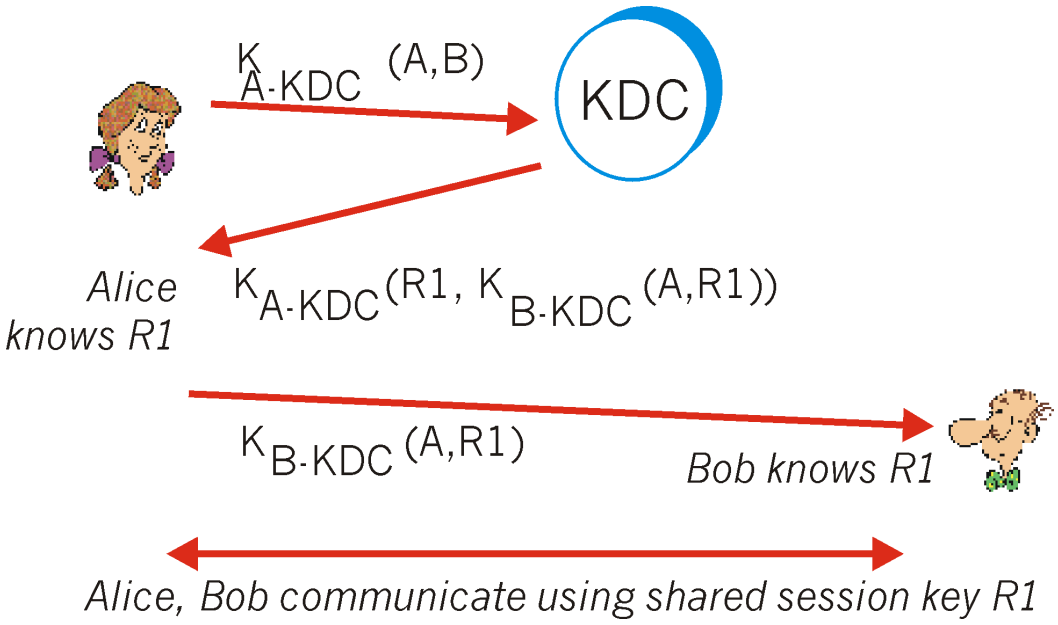


Figure 1: Setting up a one-time session key using a Key Distribution Center

- Using $K_{A\text{-}KDC}$ to encrypt her communication with the KDC, Alice sends a message to the KDC saying she (*A*) wants to communicate with Bob (*B*). We denote this message, $K_{A\text{-}KDC}$ *(A,B)*. As part of this exchange, Alice should authenticate the KDC (see homework problems), e.g., using an authentication protocol (e.g., our protocol *ap4.0)* and the shared key $K_{A\text{-}KDC}$.
- The KDC, knowing $K_{A\text{-}KDC}$, decrypts $K_{A\text{-}KDC}$ *(A,B)*. The KDC then authenticates Alice. The KDC then generates a random number, *R1*. This is the shared key value that Alice and Bob will use to perform symmetric encryption when they communicate with each other. This key is referred to as a **one-time session key** (see section 7.5.3 below), as Alice and Bob will use this key for only this one session that they are currently setting up. The KDC now needs to inform Alice and Bob of the value of *R1*. The KDC thus sends back an encrypted message to Alice containing the following:
    - *R1*, the one-time session key that Alice and Bob will use to communicate;

- a pair of values: A, and *R1*, encrypted by the KDC using Bob's key, $K_{B\text{-}KDC}$. We denote this $K_{B\text{-}KDC}(A,R1)$. It is important to note that KDC is sending Alice not only the value of *R1* for her own use, but also an encrypted version of *R1* and Alice's name encrypted using Bob's key. Alice can't decrypt this pair of values in the message (she doesn't know Bob's encryption key), but then she doesn't really need to. We'll see shortly that Alice will simply forward this encrypted pair of values to Bob (who can decrypt them).

These items are put into a message and encrypted using Alice's shared key. The message from the KDC to Alice is thus $K_{A\text{-}KDC}(R1,K_{B\text{-}KDC}(R1))$.

- Alice receives the message from the KDC, verifies the nonce, extracts *R1* from the message and saves it. Alice now knows the one-time session key, *R1*. Alice also extracts $K_{B\text{-}KDC}(A,R1)$ and forwards this to Bob.
- Bob decrypts the received message, $K_{B\text{-}KDC}(A,R1)$, using $K_{B\text{-}KDC}$ and extracts A and *R1*. Bob now knows the one-time session key, *R1*, and the person with whom he is sharing this key, *A*. Of course, he takes care to authenticate Alice using *R1* before proceeding any further.

Kerberos

Kerberos is an authentication service developed at MIT that uses symmetric key encryption techniques and a Key Distribution Center. Although it is conceptually the same as the generic KDC we described in section 7.5.1, its vocabulary is slightly different. Kerberos also contains several nice variations and extensions of the basic KDC mechanisms. Kerberos was designed to authenticate users accessing network servers and was initially targeted for use within a single administrative domain such as a campus or company.

Thus, Kerberos is framed in the language of users who want to access network services (servers) using application-level network programs such as Telnet (for remote login) and NFS (for access to remote files), rather than than human-to-human conversants who want to authenticate themselves to each other, as in our examples above. Nonetheless, the key (pun intended) underlying techniques remains the same.

The **Kerberos Authentication Server** (AS) plays the role of the KDC. The AS is the repository of not only the secret keys of all users (so that each user can communicate securely with the AS) but also information about which users have access privileges to which services on which network servers. When Alice wants to access a service on Bob (who we now think of as a server), the protocol closely follows our example in Figure 1:

- Alice contacts the Kerberos AS, indicating that she wants to use Bob. All communication between Alice and the AS is encrypted using a secret key that is shared between Alice and the AS. In Kerberos, Alice first provides her name and password to her local host. Alice's local host and the AS then determine the one-time secret session key for encrypting communication between Alice and the AS.
- The AS authenticates Alice, checks that she has access privileges to Bob, and generates a one-time symmetric session key, R1, for communication between Alice and Bob. The Authentication Server (in Kerberos parlance, now referred to as the Ticket Granting Server) sends Alice the value of R1, and also a **ticket** to Bob's services. The ticket contains Alice's name, the one-time session key, R1, and an expiration time, all encrypted using Bob's secret key (known only by Bob and the AS), as in Figure 7.5-1. Alice's ticket is valid only until its expiration time, and will be rejected by Bob is presented after that time. For Kerberos V4, the maximum lifetime of a ticket is about 21 hours. In Kerberos V5, the lifetime must expire before the end of year 9999 - a definite Y10K problem!
- Alice then sends her ticket to Bob. She also sends along an R1-encrypted timestamp that is used as a nonce. Bob decrypts the ticket using his secret key, obtains the session key, decrypts the timestamp using the just-learned session key. Bob sends back the timestamp value plus one (in Kerberos V5) or simply the timestamp itself (in Kerberos V5).

The most recent version of Kerberos (V5) provides support for multiple Authentication Servers, delegation of access rights, and renewable tickets.

Public Key Certification

One of the principle features of public key encryption is that it is possible for two entities to exchange secret messages without having to exchange secret keys. For example, when Alice wants to send a secret message to Bob, she simply encrypts the message with Bob's public key

and sends the encrypted message to Bob; she doesn't need to know Bob's secret (i.e., private) key, nor does Bob need to know her secrect key. Thus, public key cryptography obviates the need for KDC infrastructure, such as Kerberos.

Of course, with public key encryption, the communicating entities still have to exchange public keys. A user can make its public key pubicly available in many ways, e.g., by posting the key on the user's personal Web page, placing the key in a public key server, or by sending the key to a correspondent by e-mail. A Web commerce site can place its public key on its server in a manner that browsers automatically download the public key when connecting to the site. Routers can place their public keys on public key servers, thereby allowing other browsers and network entities to retrieve them.

There is, however, a subtle, yet critical, problem with public key cryptography. To gain insight to this problem, let's consider an Internet commerce example. Suppose that Alice is in the pizza delivery business and she accepts orders over the Internet. Bob, a pizza lover, sends Alice a plaintext message which includes his home address and the type of pizza he wants. In this message, Bob also includes a digital signature (e.g.,, an encrypted message digest for the original plaintext message). As discussed in Section 7.4, Alice can obtain Bob's public key (from his personal Web page, a public key server, or from an e-mail message) and verify the digital signature. In this manner Alice makes sure that Bob (rather than some adolescent prankster) indeed made the order.

This all sounds fine until clever Trudy comes along. As shown in Figure 2, Trudy decides to play a prank. Trudy sends a message to Alice in which she says she is Bob, gives Bob's home address, and orders a pizza. She also attaches a digital signature, but she attaches the signature by signing the message digest with her (i.e., Trudy's) private key. Trudy also masquerades as Bob by sending Alice Trudy's public key but saying that it belongs to Bob. In this example, also will apply Trudy's public key (thinking that it is Bob's) to the digital signature and conclude that the plaintext message was indeed created by Bob. Bob will be very surprised when the delivery person brings to his home a pizza with everything on it!

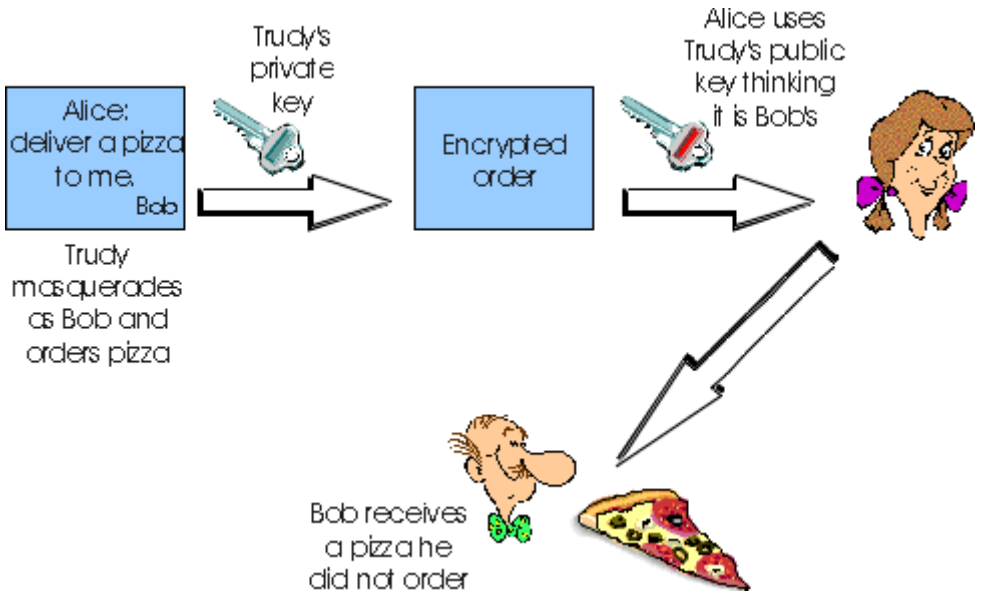


Figure 2: Trudy masquerades as Bob using public key cryptography.

We see from this example that in order for public key cryptography to be useful, entities (users, browsers, routers, etc.) need to know *for sure* that they have the public key of the entity with which they are communicating. For example, when Alice is communicating with Bob using public key cryptography, she needs to know for sure that the public key that is supposed to be Bob's is indeed Bob's.

Binding a public key to a particular entity is typically done by a **certification authority** (CA), which validates identities and issue certificates. A CA has the following roles:

- First to verify that entity (a person, a router, etc) is who it says it is. There are no mandated procedures for how certification is done. When dealing with a CA, one must trust the CA to have performed a suitably rigorous identity verification. For example, if Trudy were able to walk into Fly-by-Night Certificate Authority and simply announce "I am Alice" and receive keys associated with the identity of "Alice," then one shouldn't put much faith in public keys offered by the Fly-by-Night Certificate Authority. On the other hand, one might (or might not!) be more willing to trust a CA that is part of a federal- or state-sponsored program (e.g., [Utah 1999]). One can trust the "identify" associated with a public key only to the extent that one can trust a CA and its identity verification techniques. What a tangled web of trust we spin!
- Once the CA verifies the entity of the entity, the CA creates a **certificate** that binds the public key of the identiy to the identity. The certificate contains the public key and identifying information about the owner of the public key (for example a human name

or an IP address). The certificate is digitally signed by the CA. These steps are shown in Figure 3.

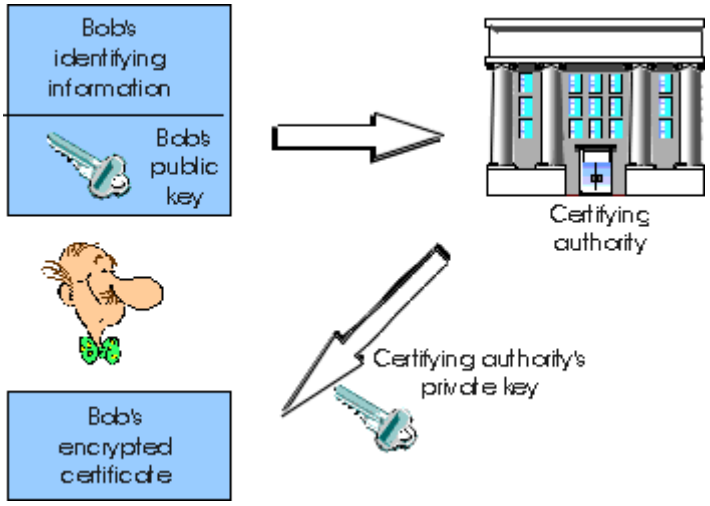


Figure 3: Bob obtains a certificate from the certification authority.

Both the International Telecommunication Union and the IETF have developed standards for Certification Authorities. ITU X.509 specifies an authentication service as well as a specific syntax for certificates. RFC 1422 describes CA-based key management for use with secure Internet e-mail. It is compatible with X.509 but goes beyond X.509 by establishing procedures and conventions for a key management architecture. Figure 4 describes some of the important field in a certificate.

Field name	**Description**
version	version number of X.509 specification
serial number	CA-issued unique identifier for a certificate
signature	specifies the algorithm used by Ca to "sign" this certificate
issuer name	identity of CA issuing this certificate, in so-called Distinguished Name(DN) [RFC 1779] format
validity period	start and end of period of validity for certificate
subject name	identity of entity whose public key is associated with this certificate, in DN format
subject public key	the subject's public key as well as an indication of the public key algorithm (and algorithm parameters) to be used with this key

Figure 4: Selected fields in a X.509 and RFC 1422 public key certificate

Result : Thus, the Key distribution and Certifications are learnt

VIVA QUESTIONS AND ANSWERS

1. Define Network?

A network is a set of devices connected by physical media links. A network is recursively is a connection of two or more nodes by a physical link or two or more networks connected by one or more nodes.

2. What is a Link?

At the lowest level, a network can consist of two or more computers directly connected by some physical medium such as coaxial cable or optical fiber. Such a physical medium is called as Link.

3. What is node?

A network can consist of two or more computers directly connected by physical medium such as coaxial cable or optical fiber. Such a physical medium is called as Links and the computer it connects is called as Nodes.

4. What is gateway or Router?

A node that is connected to two or more networks is commonly called as router or Gateway. It generally forwards message from one network to another.

5. What is point-point link?

If the physical links are limited to a pair of nodes it is said to be point to point link.

6. What is Multiple Accesses?

If the physical links are shared by more than two nodes, it is said to be Multiple Access.

7. What are the criteria necessary for an effective and efficient network?

a) Performance - It can be measured in many ways, including transmit time and response time.

b) Reliability - It is measured by frequency of failure, the time it takes a link to recover from a failure, and the network's robustness.

c) Security - Security issues include protecting data from unauthorized access and virus.

8. What is protocol?

A protocol is a set of rules that govern all aspects of information communication.

9. What are the key elements of protocols?

The key elements of protocols are

a) Syntax - It refers to the structure or format of the data, that is the order in which they are presented.

b) Semantics - It refers to the meaning of each section of bits.

c) Timing - Timing refers to two characteristics: when data should be sent and how fast they can be sent.

10. What is a peer-peer process?

The processes on each machine that communicate at a given layer are called peer-peer process.

11. Define Bandwidth and Latency?

Network performance is measured in Bandwidth (throughput) and Latency (Delay). Bandwidth of a network is given by the number of bits that can be transmitted over the network in a certain period of time. Latency corresponds to how longs it takes a message to travel from one end off a network to the other. It is strictly measured in terms of time.

12. What is Round Trip Time?

The duration of time takes to send a message from one end of a network to the other back, is called RTT.

13. What is IP address?

An Internet Protocol address (IP address) is a numerical label assigned to each device (e.g., computer, printer) participating in a computer network that uses the Internet Protocol for communication. An IP address serves two principal functions: host or network interface identification and location addressing.

14. What are IP versions?

Two versions of the Internet Protocol (IP) are in use: IP Version 4 and IP Version 6. Each version defines an IP address differently. Because of its prevalence, the generic term IP address typically still refers to the addresses defined by IPv4.

15. What is an IPv4 address?

Decomposition of an IPv4 address from dot-decimal notation to its binary value. In IPv4 an address consists of 32 bits which limits the address space to 4294967296 (2^{32}) possible unique addresses. IPv4 reserves some addresses for special purposes such as private networks (~18 million addresses) or multicast addresses (~270 million addresses).

16. What is an IPv6 address?

The rapid exhaustion of IPv4 address space prompted the Internet Engineering Task Force (IETF) to explore new technologies to expand the addressing capability in the Internet. The permanent solution was deemed to be a redesign of the Internet Protocol itself. This new generation of the Internet Protocol was eventually namedInternet Protocol Version 6 (IPv6) in 1995. The address size was increased from 32 to 128 bits (16 octets), thus providing up to 2^{128} (approximately 3.403×10^{38}) addresses. This is deemed sufficient for the foreseeable future.

17. What are Methods in ip address?

- Uses of dynamic address assignment
- Sticky dynamic IP address
- Address auto configuration
- Uses of static addressing
- Conflict
- Modifications to IP addressing?
- IP blocking and firewalls
- IP address translation

18. What is Socket?

A socket is an endpoint between two way communication.

19. What is Socket class?

A socket is simply an endpoint for communications between the machines. The Socket class can be used to create a socket.

20. What is ServerSocket class?

The ServerSocket class can be used to create a server socket. This object is used to establish communication with the clients.

21. What is Echo Command?

In echo is a command in DOS, OS/2, Microsoft Windows, Singularity, Unix and Unix-like operating systems that places a string on the computer terminal. It is a built-in command typically used in shell scripts and batch files to output status text to the screen or a file. Many shells, including Bash and zsh, implement echo as a builtin command.

22. What is ping command?

The ping command is a Command Prompt command used to test the ability of the source computer to reach a specified destination computer. The ping command is usually used as a simple way verify that a computer can communicate over the network with another computer or network device.

23. What are the Ping Related Commands?

The ping command is often used with other networking related Command Prompt commands like tracert, ipconfig, netstat, nslookup, and others.

24. Is there a continuous ping options?

ping <address> -t

Use the -t option to ping any address until you cancel it by pressing Ctrl +C.

25. IP address of ping?

The user would first start by using the ping command to ping the IP address 204.228.150.3.

26. What is meant by Talk command?

Talk is a visual communication program which copies lines from your terminal to that of another user, much like an instant messenger service.

27. Define the term unicasting, Multicasting and Broadcasting?

- If the message is sent from a source to a single destination node, it is called unicasting.
- If the message is sent to some subset of other nodes, it is called Multicasting.
- If the message is sent to all the m nodes in the networking it is called Broadcasting.

28. List the layers of OSI?

a) Physical Layer

b) Data Link Layer

c) Network Layer

d) Transport Layer

e) Session Layer

f) Presentation Layer

g) Application Layer

29. Which layers are network support layers?

a) Physical Layer

b) Data Link Layer

c) Network Layer

30. Which layers are user support layers?

a) Session Layer

b) Presentation Layer

c) Application Layer

31. What are the responsibilities of Data Link Layer?

The Data Link Layer transforms the physical layer, a raw transmission facility, to a reliable link and is responsible for node-node delivery.

a) Framing

b) Physical Addressing

c) Flow control

d) Error control

e) Access control

32. What are the responsibilities of Network Layer?

The Network Layer is responsible for the source to destination delivery of packet possibly across multiple networks

a) Logical Addressing

b) Routing

33. What are the responsibilities of Session Layer?

The session layer is the network dialog controller. It establishes, maintains and synchronizes the interaction between the communicating systems.

a) Dialog control

b) Synchronization

34. What are the responsibilities of Transport Layer?

The Transport Layer is responsible for source to destination delivery of the entire message.

a) Service-point Addressing

b) Segmentation and reassembly

c) Connection Control

d) Flow control

e) Error control

35. What are the responsibilities of Presentation Layer?

The Presentation layer is concerned with the syntax and semantics of the information exchanged between two systems.

a) Translation

b) Encryption

c) Compression

36. What are the responsibilities of Application Layer?

The Application Layer enables the user, whether human or software, to access the network. It provides user interfaces and support for services such as e-mail, shared database management and other types of distributed information services.

a) Network virtual Terminal

b) File transfer, access and Management

c) Mail Services

d) Directory Services

37. What are the types of errors?

a) Single Bit error

In a Single bit error only one bit in the data unit has changes.

b) Burst error

A Burst error means that two or more bits in the data have changed.

38. What is Redundancy?

The concept of including extra information in the transmission solely for the purpose of comparison. This technique is called redundancy.

39. What is CRC?

CRC, is the most powerful of the redundancy checking techniques, is based on binary division.

40. Define CRC generator?

A CRC generator uses a modulo-2 division. In the first step, the 4-bit division is subtracted from the first 4 bit of the dividend. Each bit of the divisor is subtracted from the corresponding bit of the dividend with disturbing the next higher bit.

41. What is the implementation of CRC long division?

The CRC code is generated through a process of long division. This can be implemented using a digital logic circuit consisting of exclusive-or (XOR) gates and a shift register.

42. What is the use of CRC in USB?

The USB specification calls for the use of Cyclic Redundancy Checksums (CRC) to protect all non-PID fields in token and data packets from errors during transmission. This paper describes the mathematical basis behind CRC in an intuitive fashion and then explains the specific implementation called for in USB.

43. Write the application of CRC?

A CRC-enabled device calculates a short, fixed-length binary sequence, known as the check value or CRC, for each block of data to be sent or stored and appends it to the data, forming a codeword.

44. What is a Polynomial Representation of Binary Words ?

To understand how a CRC coder works, let us first define the notion of the polynomial associated with a binary sequence. The polynomial is in terms of some dummy variable X, the powers of which are combined with binary (0 and 1) coefficients.

45. Define frame sequence for the data frame?

To define a code we have to state how to obtain the polynomial R(X) (that is, the appended bits) corresponding to any set of message bits or M(X). The appended bits are called the frame check sequence for the data frame of k bits.

46. Write few Importance of Error Detection?

A transmitted bit can be received in error, due to "noise" on the transmission channel. If we are dealing with voice or video data, the occurrence of errors in a small percentage of bits is quite tolerable, but in many other cases it is crucial that all bits be received intact.

47. Give the essential properties for polynomial?

A polynomial should be selected to have at least the following properties.

a) It should not be

b) It should be divisible by(x+1).

48. What is Checksum?

Checksum is used by the higher layer protocols (TCP/IP) for error detection.

49. Define Retransmission?

Retransmission is a technique in which the receiver detects the occurrence of an error and asks the sender to resent the message. Resending is repeated until a message arrives that the receiver believes is error freed.

50. What is framing?

Framing in the data link layer separates a message from one source to a destination, or from other messages to other destinations, by adding a sender address and a destination address. The destination address defines where the packet has to go and the sender address helps the recipient acknowledge the receipt

51. What is Flow control?

Flow control refers to a set of procedures used to restrict the amount of data that the sender can send before waiting for acknowledgement.

52. What is stop and wait protocol?

In Stop and wait protocol, sender sends one frame, wait until it receives confirmation from the receiver (okay to go ahead), and then sends the next fame.

53. What is sliding window?

The sliding window is an abstract concept that defines the range of sequence numbers that is the concern of the sender and receiver. In other words, he sender and receiver need to deal with only part of the possible sequence numbers.

54. What is Piggy Backing?

A technique called piggy backing is used to improve the efficiency of the bidirectional protocols. When a frame is carrying data from A to B, it can also carry control information about arrived frames from B; when a frame is carrying data from B to A, it can also carry control information about the arrived frames from A.

55. What is advantage of using piggybacking?

The principal advantage of using piggybacking over having distinct acknowl- edgement frames is a better use of the available channel bandwidth. The ack field in the frame header costs only a few bits, whereas a separate frame would need a header, the acknowledgement, and a checksum.

56. What is One-Bit Sliding Window Protocol?

Before tackling the general case, let us first examine a sliding window proto- col with a maximum window size of 1. Such a protocol uses stop-and-wait since the sender transmits a frame and waits for its acknowledgement before sending the next one.

57. Explain the Protocol Using Go Back N?

Until now we have made the tacit assumption that the transmission time required for a frame to arrive at the receiver plus the transmission time for the ac- knowledgement to come back is negligible.

58. What is meant by sleep()?

The java.lang.Thread.sleep (long millis) method causes the currently executing thread to sleep for the specified number of milliseconds, subject to the precision and accuracy of system timers and schedulers.

59. Explain pipelining?

The need for a large window on the sending side occurs whenever the product of bandwidth × round-trip-delay is large.The product of these two factors basically tells what the capacity of the pipe is, and the sender needs the ability to fill it without stopping in order to operate at peak efficiency. This technique is known as pipelining.

60. Define go back n operation?

Two basic approaches are available for dealing with errors in the presence of pipelining. One way, called go back n, is for the receiver simply to discard all subsequent frames, sending no acknowledgements for the discarded frames. This strategy corresponds to a receive window of size 1.

61. Explain selective repeat operation?

The sender, unaware of this problem, continues to send frames until the timer for frame 2 expires. Then it backs up to frame 2 and starts all over with it, sending 2, 3, 4, etc. all over again. The other general strategy for handling errors when frames are pipelined is called selective repeat.

62. How many buffers must the receiver have?

Under no conditions will it ever accept frames whose sequence numbers are below the lower edge of the window or frames whose sequence numbers are above the upper edge of the window. Consequently, the number of buffers needed is equal to the window size, not to the range of sequence numbers.

63. What is meant by acknowledgement?

An acknowledgement is a signal passed between communicating Processes or computer

To signify acknowledgement, or receipt of response, as part of a communications protocol.

64. What is TCP socket?

In TCP/IP and UDP networks, a port is an endpoint to a logical connection and the way a client program specifies a specific server program on a computer in a network. This list of well-known port numbers specifies the port used by the server process as its contact port.

65. What is UDP socket?

UDP is a connectionless protocol and the socket is created for client and server to transfer the data. Socket connection is achieved using the port number

66. What is DNS?

DNS stands for domain name system. Unique name of the host is identified with its IP address through server client communication.

67. What is raw socket?

Raw socket is similar to the TCP or UDP socket and raw socket is used to capture the packet of data and filter packet with error.

68. What is data frame?

A data frame is an aggregate of numerous, partly overlapping collections of data and metadata that have been derived from massive amounts of network activity such as Content production, consumption, and other user behavior.

69. What is subnet?

A generic term for section of a large networks usually separated by a bridge or router.

70. What are the possible ways of data exchange?

- Simplex
- Half-duplex
- Full-duplex

71. Remote Command Execution?

A variety of protocols exist primarily to allow users to execute commands on remote systems. This section describes the BSD "r" commands, rexec and rex.

72. What is Remote Code Execution?

In computer security, arbitrary code execution or remote code execution is used to describe an attacker's ability to execute any commands of the attacker's choice on a target machine or in a target process. It is commonly used in arbitrary code execution vulnerability to describe a software bug that gives an attacker a way to execute arbitrary code.

73. Bugs?

Indicating ``Login incorrect'' as opposed to ``Password incorrect'' is a security breach which allows people to probe a system for users with null passwords. A facility to allow all data and password exchanges to be encrypted should be present.

74. Applications of remote command?

- Image convertor
- Scrollar JS
- sG event
- pixels editor
- ajax talker

75. Introduction to the Java Robot Class

Baldwin shows you how to write a Java program that will programmatically invoke mouse clicks on non-Java programs, and programmatically enter text into non-Java programs.

76. Remote Screenshots?

Programmers do not like it when you stand behind them watching how they code, especially if they are not confident on how to solve a problem. Many years ago, I wrote a very simple Java program that allowed me to watch my students' screens, with their knowledge of course, without the feeling of being watched.

77. Definition of toolkit?

The Abstract Window Toolkit (AWT) is Java's original platform-dependent windowing, graphics, and user-interface widget toolkit preceding Swing. The AWT is part of the Java Foundation Classes (JFC) — the standard API for providing a graphical user interface (GUI) for a Java program.

78. What is ARP?

Address Resolution Protocol (ARP) is a network protocol, which maps a network layer protocol address to a data link layer hardware address. For example, ARP is used to resolve IP address to the corresponding Ethernet address.

79. What is the use of ARP?

A host in an Ethernet network can communicate with another host, only if it knows the Ethernet address (MAC address) of that host. The higher level protocols like IP Use a different kind of addressing scheme (like IP address) from the lower level hardware addressing scheme like MAC address. ARP is used to get the Ethernet address of a host from its IP address. ARP is extensively used by all the hosts in an Ethernet network.

80. What is an ARP cache?

ARP maintains the mapping between IP address and MAC address in a table in memory called ARP cache. The entries in this table are dynamically added and removed.

81. Define File Transfer Protocol

File Transfer Protocol (FTP), a standard Internet protocol, is the simplest way to exchange files between computers on the Internet. Like the Hypertext Transfer Protocol (HTTP), which transfers displayable Web pages and related files, and the Simple Mail Transfer Protocol (SMTP), which transfers e-mail, FTP is an application protocol that uses the Internet's TCP/IP protocols. FTP is commonly used to transfer Web page files from their creator to the computer that acts as their server for everyone on the Internet. It's also commonly used to download programs and other files to your computer from other servers.

82. Explain User Datagram Protocol, UDP.

The UDP is a connectionless, unreliable service. UDP messages can be lost and duplicated. User Data Protocol is a communication protocol. It is normally used as an alternative for TCP/IP. However there are a number of differences between them. UDP does not divide data into packets. Also, UDP does not send data packets in sequence. Hence, the application program must ensure the sequencing. UDP uses port numbers to distinguish user requests. It also has a checksum capability to verify the data.

83. What is TCP windowing concept?

TCP windowing concept is primarily used to avoid congestion in the traffic. It controls the amount of unacknowledged data a sender can send before it gets an acknowledgement back from the receiver that it has received it.

84. TCP vs. UDP.

TCP guarantees the delivery of data. UDP on the other hand, does not guarantee delivery of data. TCP delivers messages in the order they were sent. UDP has no ordering mechanisms. In TCP data is sent as a stream while UDP sends data as individual packets. UDP is faster than TCP. TCP is a connection oriented protocol while UDP is connectionless.

85. What is Connection-oriented?

Communication includes the steps of setting up a call from one computer to another, transmitting/receiving data, and then releasing the call, just like a voice phone call. However, the network connecting the computers is a packet switched network, unlike the phone system's circuit switched network. Connection-oriented communication is done in one of two ways over a packet switched network: with and without virtual circuits.

86. What is Connectionless?

Communication is just packet switching where no call establishment and release occur. A message is broken into packets, and each packet is transferred separately. Moreover, the packets can travel different route to the destination since there is no connection. Connectionless service is typically provided by the **UDP (User Datagram Protocol)**, which we will examine later. The packets transferred using UDP are also called **datagrams**.

87. Define datagram socket

A **datagram socket** is a type of connectionless network **socket**, which is the point for sending or receiving of packet delivery services. Each packet sent or received on a **datagram socket** is individually addressed and routed.

88. Define datagram packet

Datagram packets are used to implement a connectionless packet delivery service. Each message is routed from one machine to another based solely on information contained within that packet. Multiple packets sent from one machine to another might be routed differently, and might arrive in any order. Packet delivery is not guaranteed.

89. What is SAP?

Series of interface points that allow other computers to communicate with the other layers of network protocol stack.

90. How Gateway is different from routers?

A gateway operates at the upper levels of the OSI model and translates information between two completely different network architectures or data formats.

91. What is attenuation?

The degeneration of a signal over distance on a network cable is called attenuation.

92. What is MAC address?

The address for a device as it is identified at the Media Access Control (MAC) layer in the network architecture. MAC address is usually stored in ROM on the network adapter card and is unique.

93. What is difference between ARP and RARP?

The address resolution protocol is used to associate the 32 bit IP address with the 48 bit physical address, used by a host or a router to find the physical address of another host on its network by sending a ARP query packet that includes the IP address of the receiver.

The reverse address resolution protocol allows a host to discover its Internet address when it knows only its physical address.

94. What is multicast routing?

Sending a message to a group is called multicasting, and its routing algorithm is called multicast routing.

95. What are the important topologies for networks?

- Bus topology: In this each computer is directly connected to primary network cable in a single line. Advantage of bus topology is an Inexpensive and easy to install.
- Star topology: In this all computers are connected using a central hub. Advantage of star topology is easy to install and also easy to trouble shoot physical problems.
- Ring topology: In this all computers are connected in loop. Advantage of ring topology is all the computers have equal access to network media.

96. What is the range of address in the classes of internet address?

- Class A – 0.0.0.0 to 127.255.255.255
- Class B – 128.0.0.0 to 191.255.255.255
- Class C– 192.0.0.0 to 223.255.255.255
- Class D– 224.0.0.0 to 239.255.255.255
- Class E– 240.0.0.0 to 247.255.255.255

97. What is called multipoint communication?

Some applications require data to be delivered from a sender to multiple receivers. Examples of such applications include audio and video broadcasts, real-time delivery of stock quotes, and teleconferencing applications. A service where data is delivered from a sender to multiple receivers is called multipoint communication or multicast.

98. Explain multicast group or host group?

Multicast can be thought of as a generalization of unicast and broadcast. In multicast, data is transmitted to a set of hosts that have indicated interest in receiving the data, referred to as a multicast group or host group.

99. Define IGMP?

IP multicast involves both hosts and routers. In IPv4, support of IP multicast is optional, but almost all hosts and most routers support multicast. Hosts that are members of a multicast group exchange Internet Group Management Protocol (IGMP) messages with routers.

100. Explain IP Multicast service?

A central notion of IP multicast is the IP multicast address. A multicast address is an IP address in the range from 224.0.0.0 to 239.255.255.255. Each multicast address specifies a multicast group.

101. Define multicast packet?

A multicast packet is an IP datagram that has an IP multicast address as destination address. A multicast packet is delivered to all hosts that are members the multicast group designated by the IP multicast address.

102. Explain IP Multicast Addresses?

The range of IP multicast addresses, 224.0.0.0 to 239.255.255.255, corresponds to the Class D addresses in the class-based IP address scheme. These are the addresses that start with "1110" (Figure 10.4). This leaves 28 bits to identify a multicast group.

103. What is Hypertext Transfer Protocol (HTTP)?

Hypertext Transfer Protocol (HTTP) is a networking protocol for distributed, collaborative, hypermedia information systems. HTTP is the foundation of data communication for the World Wide Web.

104. What is the port number of HTTP?

The port number of HTTP is 80.

105. What is autonomous system?

It is a collection of routers under the control of a single administrative authority and that uses a common Interior Gateway protocol.

106. What is Source Routing?

Source routing is similar in concept to virtual circuit routing. It is implemented as under:

- Initially, a path between nodes wishing to communicate is found out, either by flooding or by any other suitable method.

- This route is then specified in the header of each packet routed between these two nodes. A route may also be specified partially, or in terms of some intermediate hops.

107. What is minimum number of hops?

- **Minimum number of hops:** If each link is given a unit cost, the shortest path is the one with minimum number of hops. Such a route is easily obtained by a breadth first search method. This is easy to implement but ignores load, link capacity etc.

108. What is IGP (Interior Gateway Protocol)?

It is any routing protocol used within an autonomous system.

109. What is region?

When hierarchical routing is used, the routers are dividing into what we will call regions, with each router knowing all the detail about how to route packets to destinations within its own region, but knowing nothing about the internal structure of other regions.

110. What is EGP (Exterior Gateway Protocol)?

It is the protocol the routers in neighboring autonomous systems use to identify the set of networks that can be reached within or via each autonomous system.

111. What is BGP (Border Gateway protocol)?

It is a protocol used to advertise the set of networks that can be reached with in an autonomous system. BGP enables this information to be shared with the autonomous system. This is newer than EGP (Exterior Gateway Protocol).

112. What is Kerberos?

It is an authentication service developed at the Massachusetts Institute of Technology. Kerberos used encryption to prevent intruders from discovering passwords and gaining unauthorized access to files.

113. What is OSPF?

It is an Internet routing protocol that scales well, can route traffic along multiple paths, and used knowledge of an Internet's topology to make accurate routing decisions.